

ISIS-1611: Inteligencia Artificial

Notas del Curso Semestre 2026-19

Universidad de los Andes

Basado en: *Artificial Intelligence: A Modern Approach*

Russell, Norvig & Ming-Wei Chang (4^a ed.)

Carla González

16 de julio de 2026

Índice general

I	Búsqueda	5
1.	Semana 1: Introducción y Búsqueda Desinformada	6
1.1.	Contexto e Historia de la IA	6
1.1.1.	Impacto de la IA en el mundo real	6
1.1.2.	¿Qué es la Inteligencia Artificial?	6
1.1.3.	Historia de la IA	6
1.2.	Agentes Inteligentes	6
1.2.1.	¿Qué es un Agente?	6
1.2.2.	Racionalidad	7
1.2.3.	Descripción PEAS	7
1.2.4.	Naturaleza de los Ambientes	7
1.2.5.	Tipos de Agentes	8
1.3.	Búsqueda Desinformada I: Formulación del Problema	8
1.3.1.	Formulación de un Problema de Búsqueda	8
1.3.2.	Nodos vs. Estados	9
1.3.3.	Medidas de Desempeño de Algoritmos	9
1.4.	Búsqueda Desinformada II: Algoritmos	9
1.4.1.	Breadth-First Search (BFS)	9
1.4.2.	Depth-First Search (DFS)	9
1.4.3.	Depth-Limited Search y Iterative Deepening (IDDFS)	10
1.4.4.	Uniform-Cost Search (UCS)	10
1.4.5.	Comparativa Búsqueda Desinformada	10
1.5.	Búsqueda Informada: Heurísticas	10
1.5.1.	Función Heurística	10
1.5.2.	Greedy Best-First Search	10
1.5.3.	A*	11
2.	Semana 2: Búsqueda Informada y Ambientes Complejos	12
2.1.	Búsqueda Informada Avanzada	12
2.1.1.	Búsqueda con Memoria Acotada	12
2.1.2.	Heurísticas: Dominancia y Creación	13
2.2.	Búsqueda en Ambientes Complejos	13
2.2.1.	Búsqueda Local	13
2.2.2.	Hill Climbing (Ascenso de Colina)	13
2.2.3.	Simulated Annealing	14
2.2.4.	Algoritmos Genéticos	14
2.3.	Ejercicios de Búsqueda	14
2.3.1.	Guía de Ejercicios — Resumen de Bloques	14

II	Razonamiento bajo Incertidumbre	16
3.	Semana 3: Búsqueda Adversaria y Razonamiento bajo Incertidumbre	17
3.1.	Búsqueda Adversaria y Juegos I	17
3.1.1.	Juegos como Problemas de Búsqueda	17
3.1.2.	Minimax	17
3.2.	Búsqueda Adversaria y Juegos II	18
3.2.1.	Poda Alfa-Beta (α - β)	18
3.2.2.	Juegos con Incertidumbre: Expectiminimax	18
3.2.3.	Funciones de Evaluación	19
3.2.4.	Monte Carlo Tree Search (MCTS)	19
3.3.	Razonamiento bajo Incertidumbre I	19
3.3.1.	Probabilidad como Base del Razonamiento	19
3.3.2.	Regla de Bayes	19
3.3.3.	Distribuciones de Probabilidad	20
3.4.	Razonamiento bajo Incertidumbre II	20
3.4.1.	Inferencia por Enumeración	20
3.4.2.	Variables y Modelos	20
4.	Semana 4: Redes Bayesianas, MDPs y CSP	22
4.1.	Redes Bayesianas	22
4.1.1.	Semántica	22
4.1.2.	Inferencia Exacta	22
4.1.3.	Inferencia Aproximada	22
4.2.	Procesos de Decisión de Markov (MDPs)	23
4.2.1.	Propiedad de Markov	23
4.2.2.	Política y Utilidad	24
4.2.3.	Ecuación de Bellman	24
4.2.4.	Algoritmos de Solución	24
4.3.	Ejercicios: Razonamiento bajo Incertidumbre	24
4.4.	Problemas de Satisfacción de Restricciones (CSP)	25
4.4.1.	Tipos de Restricciones	25
4.4.2.	Búsqueda con Retroceso (Backtracking)	25
5.	Semana 5: CSP Avanzado y Agentes Lógicos	27
5.1.	Problemas de Satisfacción de restricciones	27
5.1.1.	Diagrama de Arbol	27
5.1.2.	k-consistencia	28
5.1.3.	Forward Checking	28
5.1.4.	Estructura del grafo de restricciones	28
5.1.5.	Búsqueda local para CSP: Min-conflicts	29
5.2.	Ejercicio: Coloreado del mapa de Australia	29
5.3.	Agentes Lógicos: Introducción	30
5.3.1.	Agentes basados en conocimiento	30
5.3.2.	Lógica proposicional	31
5.3.3.	Formas normales: CNF y DNF	31
5.3.4.	El mundo del Wumpus	31

III Lógica y Representación del Conocimiento 33

6. Semana 6: Lógica y Representación del Conocimiento 34

6.1. Agentes Lógicos II: Inferencia Proposicional	34
6.1.1. Reglas de Inferencia	34
6.1.2. Resolución	34
6.1.3. DPLL y WalkSAT	34
6.1.4. Ejemplo: Refutación por Resolución	34
6.2. Lógica de Primer Orden (LPO)	35
6.2.1. Sintaxis	35
6.2.2. Semántica	35
6.2.3. Cuantificadores	36
6.2.4. Ejemplo: Modelo en LPO	36
6.3. LPO e Inferencia I	36
6.3.1. Bases de Conocimiento en LPO	36
6.3.2. Unificación	36
6.3.3. Inferencia con Cuantificadores	37
6.3.4. Forma Normal de Clausura (CNF en LPO)	37
6.3.5. Ejemplo: Unificación	37
6.4. Inferencia en LPO II	37
6.4.1. Encadenamiento hacia adelante (Forward Chaining)	37
6.4.2. Encadenamiento hacia atrás (Backward Chaining)	38
6.4.3. Resolución en LPO	38
6.4.4. Ejemplo: El caso de Nono (¿Es West un criminal?)	38
6.5. Ejercicios: Guía de Agentes Lógicos (con solución paso a paso)	39
6.5.1. Bloque 1: Conceptos Fundamentales de Agentes Lógicos	39
6.5.2. Bloque 2: Validez y Satisfacibilidad	41
6.5.3. Bloque 3: Modelamiento de Lenguaje Natural — El Problema de la Man- sión Embrujada	43
6.5.4. Bloque 4: Conversión a FNC y Resolución	44
6.5.5. Bloque 5: Cláusulas de Horn, Cláusulas Definidas y Encadenamiento	46
6.6. Ejercicios: Guía de Lógica de Primer Orden (con solución paso a paso)	49
6.6.1. Bloque 1: Conceptos Fundamentales de LPO	49
6.6.2. Bloque 2: Traducción — Español a Lógica de Primer Orden	50
6.6.3. Bloque 3: LPO a Español y Análisis de Validez	51
6.6.4. Bloque 4: Construcción de Base de Conocimiento — Dominio de Parentesco	52
6.6.5. Bloque 5: Unificación y Demostración por Resolución	53

IV Planeación 56

7. Semana 7: Inferencia en LPO y Planeación 57

7.1. Inferencia en LPO: Cierre	57
7.1.1. Lógica Proposicional: Entailment y Model Checking	57
7.1.2. Cláusulas de Horn y Encadenamiento hacia Adelante	57
7.1.3. Encadenamiento hacia Atrás	58
7.1.4. Sustitución, Unificación y Occurs Check	59
7.1.5. Forma Clausal y Resolución por Refutación en LPO	59
7.2. Ejercicios: Guía de Lógica e Inferencia (con solución paso a paso)	60
7.2.1. Bloque 1: Preguntas Conceptuales (Verdadero / Falso)	60
7.2.2. Ejercicio 1: El apagón en Paloquemao	60
7.2.3. Ejercicio 2: Café de especialidad en el Eje Cafetero	61

7.2.4.	Ejercicio 3: La comparsa del Carnaval de Barranquilla	62
7.2.5.	Ejercicio 4: Estación de anillamiento en el páramo	63
7.2.6.	Ejercicio 5: El caso de la pieza extraviada	65
7.3.	Planeación: Definición y Algoritmos Generales	67
7.3.1.	¿Qué es la Planeación Clásica?	67
7.3.2.	PDDL: Representación de Estados	68
7.3.3.	PDDL: Schema de Acción	68
7.3.4.	Ejemplos de Dominios Clásicos	69
7.3.5.	Búsqueda hacia Adelante en el Espacio de Estados (Progresión)	70
7.3.6.	Búsqueda hacia Atrás en el Espacio de Estados (Regresión)	71
7.3.7.	Planeación como Satisfacibilidad Booleana (SATPLAN)	72
7.3.8.	Otras Estrategias de Planeación	72
7.4.	Planeación: Heurísticas y Planeación Jerárquica	73
7.4.1.	Repaso: Admisibilidad y Monotonidad de Heurísticas	73
7.4.2.	Heurísticas en Planeación: Relajación de Problemas	74
7.4.3.	Heurística de Ignorar Precondiciones	74
7.4.4.	Heurística de Ignorar Listas de Borrado (Delete Lists)	75
7.4.5.	Podado Independiente de Dominio	76
7.4.6.	Planeación Jerárquica (HTN)	78
7.5.	Planeación en Ambientes No Deterministas	84
7.5.1.	Ambientes No Deterministas y Condicionales	84
7.5.2.	Planeación Parcialmente Observable	84
7.5.3.	Planeación Continua y de Ejecución	84
V	IA Moderna, Aplicaciones y Ética	85
8.	Semana 8: Aplicaciones, Ética y Repaso Final	86
8.1.	Aplicaciones en IA	86
8.1.1.	Machine Learning	86
8.1.2.	Deep Learning	86
8.1.3.	Áreas de Aplicación	86
8.2.	Ética en IA y Aplicaciones Futuras	87
8.2.1.	Riesgos y Desafíos Éticos	87
8.2.2.	Marcos Regulatorios	87
8.2.3.	Niveles de Uso de IA Generativa (Uniandes)	87
8.2.4.	Futuro de la IA	87
8.3.	Repaso para Examen Final	87
8.3.1.	Resumen de Temas	87

Parte I

Búsqueda

Capítulo 1

Semana 1: Introducción y Búsqueda Desinformada

1.1. Contexto e Historia de la IA

1.1.1. Impacto de la IA en el mundo real

La IA impacta múltiples sectores: economía, política, ley y derecho, trabajo, ciencia, salud y educación.

1.1.2. ¿Qué es la Inteligencia Artificial?

Existen cuatro enfoques para definir la IA:

Pensamiento humano Modelos cognitivos	Pensamiento racional Leyes del pensamiento (lógica)
Actuar humanamente Test de Turing	Actuar racionalmente Agentes racionales

Nota 1.1.1. La IA moderna se enfoca en **agentes racionales**: entidades que perciben su entorno y actúan para maximizar su medida de desempeño esperada.

1.1.3. Historia de la IA

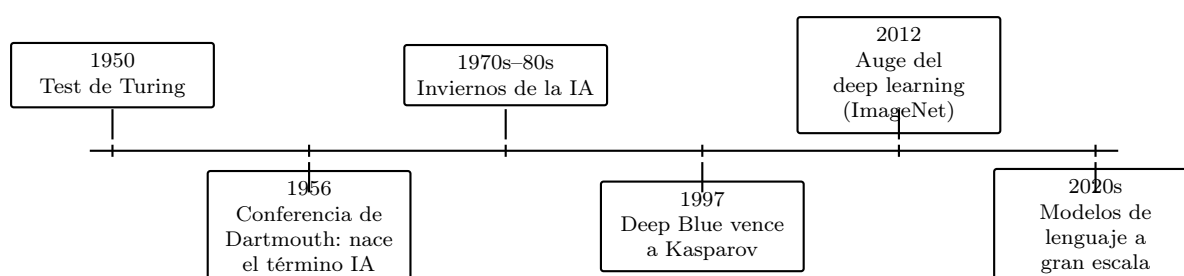


Figura 1.1: Línea de tiempo con algunos hitos de la historia de la IA.

1.2. Agentes Inteligentes

1.2.1. ¿Qué es un Agente?

Definición 1.2.1 (Agente). Un agente es cualquier entidad que **percibe** su ambiente mediante **sensores** y **actúa** sobre él mediante **actuadores**.

La función del agente mapea percepciones a acciones: $f : P^* \rightarrow A$

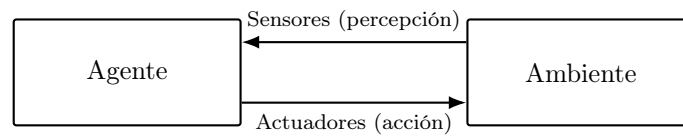


Figura 1.2: Ciclo agente-ambiente: el agente percibe el ambiente a través de sensores y actúa sobre él mediante actuadores, en un ciclo continuo.

Ejemplo 1.2.1 (PEAS de un taxi autónomo). La descripción PEAS de un taxi autónomo es:

- **Performance:** seguridad, tiempo de viaje, cumplir normas de tránsito, comodidad del pasajero.
- **Environment:** calles, otros vehículos, peatones, señales de tránsito, clima.
- **Actuators:** dirección, acelerador, freno, señales, bocina.
- **Sensors:** cámaras, GPS, velocímetro, sensores de proximidad (LIDAR/radar).

1.2.2. Racionalidad

Un agente racional selecciona la acción que maximiza su **medida de desempeño** esperada, dado su conocimiento actual.

Depende de:

1. La medida de desempeño
2. El conocimiento previo del entorno
3. Las acciones disponibles
4. La secuencia de percepciones hasta el momento

1.2.3. Descripción PEAS

Para caracterizar el ambiente de una tarea se usa PEAS:

- **Performance** (Medida de desempeño)
- **Environment** (Entorno)
- **Actuators** (Actuadores)
- **Sensors** (Sensores)

1.2.4. Naturaleza de los Ambientes

Dimensión	Valores posibles
Observabilidad	Completamente observable / Parcialmente observable
Agentes	Un solo agente / Multiagente
Determinismo	Determinista / Estocástico
Episodicidad	Episódico / Secuencial
Dinamismo	Estático / Dinámico
Tiempo	Discreto / Continuo
Conocimiento	Conocido / Desconocido

1.2.5. Tipos de Agentes

1. **Agente de reflejo simple:** actúa según la percepción actual (reglas condición-acción).
2. **Agente de reflejo basado en modelos:** mantiene un estado interno del mundo.
3. **Agente basado en objetivos:** considera metas para elegir acciones.
4. **Agente basado en utilidad:** maximiza una función de utilidad.
5. **Agente que aprende:** mejora su desempeño con la experiencia.

1.3. Búsqueda Desinformada I: Formulación del Problema

1.3.1. Formulación de un Problema de Búsqueda

Definición 1.3.1 (Problema de búsqueda). Un problema de búsqueda se define con cinco componentes:

1. **Estado inicial:** s_0
2. **Modelo de transición:** $\text{RESULT}(s, a) \rightarrow s'$
3. **Función de costo de acción:** $c(s, a, s')$
4. **Test de meta:** $\text{IS-GOAL}(s) \rightarrow \{T, F\}$
5. **Espacio de estados:** grafo de todos los estados alcanzables

La **solución** es una secuencia de acciones que lleva del estado inicial a un estado meta. Una **solución óptima** minimiza el costo total del camino.

Ejemplo 1.3.1 (Mapa de Rumania). Un viajero está en la ciudad de Arad y debe llegar a Bucarest.

- **Estado inicial:** *Arad*.
- **Modelo de transición:** $\text{RESULT}(\text{Arad}, \text{ir a Zerind}) = \text{Zerind}$, y así para cada carretera.
- **Costo de acción:** la distancia en kilómetros de cada carretera.
- **Test de meta:** estado = *Bucarest*.
- **Espacio de estados:** el grafo de ciudades rumanas conectadas por carreteras.

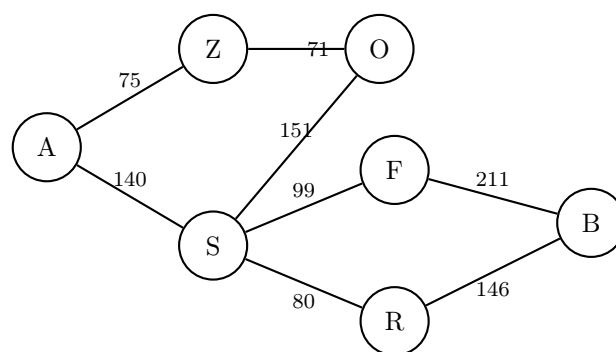


Figura 1.3: Fragmento del espacio de estados del mapa de Rumania: nodos = ciudades, aristas = carreteras con su costo (distancia en km). Este grafo es la representación del *problema*; el árbol de búsqueda se construye explorándolo a partir de *Arad*.

1.3.2. Nodos vs. Estados

Un **nodo** en el árbol de búsqueda contiene:

- El estado que representa
- El nodo padre
- La acción que generó el nodo
- El costo del camino $g(n)$
- La profundidad

1.3.3. Medidas de Desempeño de Algoritmos

Criterio	Descripción
Completitud	¿Encuentra solución si existe?
Optimalidad	¿Encuentra la solución de menor costo?
Complejidad en tiempo	Nodos generados/expandidos
Complejidad en espacio	Nodos en memoria simultáneamente

Los parámetros usuales son b (factor de ramificación), d (profundidad de la solución más superficial) y m (profundidad máxima del árbol).

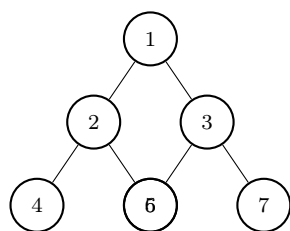
1.4. Búsqueda Desinformada II: Algoritmos

1.4.1. Breadth-First Search (BFS)

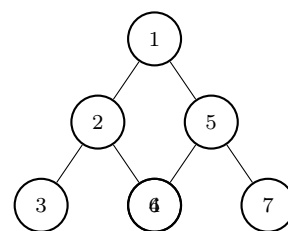
- Expande el nodo más **superficial** primero (FIFO).
- **Completo**: sí (si b finito).
- **Óptimo**: sí, si todos los costos de paso son iguales.
- Tiempo: $\mathcal{O}(b^d)$ Espacio: $\mathcal{O}(b^d)$

1.4.2. Depth-First Search (DFS)

- Expande el nodo más **profundo** primero (LIFO).
- **Completo**: no en árbol infinito; sí en grafo finito.
- **Óptimo**: no.
- Tiempo: $\mathcal{O}(b^m)$ Espacio: $\mathcal{O}(bm)$



BFS: orden de expansión 1,2,3,4,5,6,7 (nivel por nivel).



DFS: orden de expansión 1,2,3,4,5,6,7 (baja hasta el fondo antes de retroceder).

Figura 1.4: Mismo árbol de búsqueda, dos órdenes de expansión distintos según la estructura de datos de la frontera (cola FIFO en BFS, pila LIFO en DFS).

1.4.3. Depth-Limited Search y Iterative Deepening (IDDFS)

IDDFS ejecuta DFS con límite de profundidad creciente $l = 0, 1, 2, \dots$

- Combina las ventajas de BFS (completo, óptimo si costos iguales) con las de DFS (espacio lineal).
- Tiempo: $\mathcal{O}(b^d)$ Espacio: $\mathcal{O}(bd)$

1.4.4. Uniform-Cost Search (UCS)

- Expande el nodo con menor **costo de camino** $g(n)$ (cola de prioridad).
- Equivale a BFS cuando todos los costos son iguales.
- **Completo**: sí (si costos $> \epsilon > 0$).
- **Óptimo**: sí.
- Tiempo y espacio: $\mathcal{O}(b^{1+\lceil C^*/\epsilon \rceil})$, donde C^* es el costo óptimo.

Ejemplo 1.4.1 (UCS sobre un grafo con costos). Usando el grafo de Rumania de la clase anterior, para ir de *Arad* a *Bucarest*, UCS explora en este orden (por costo acumulado $g(n)$ creciente): *Arad*(0), *Zerind*(75), *Sibiu*(140), *Oradea*(146), *Rimnicu*(220), *Fagaras*(239), *Bucarest* vía *Rimnicu* $\rightarrow$ *Pitesti* $\rightarrow$ *Bucarest* con costo 418, que resulta más barato que ir vía *Fagaras* (costo 450), aunque *Fagaras* se alcance primero.

1.4.5. Comparativa Búsqueda Desinformada

Algoritmo	Completo	Óptimo	Tiempo	Espacio
BFS	Sí	Sí*	b^d	b^d
DFS	No	No	b^m	bm
IDDFS	Sí	Sí*	b^d	bd
UCS	Sí	Sí	$b^{C^*/\epsilon}$	$b^{C^*/\epsilon}$

*Óptimo si costos de paso son iguales

1.5. Búsqueda Informada: Heurísticas

1.5.1. Función Heurística

Definición 1.5.1 (Heurística). $h(n)$ es una función que estima el costo desde el nodo n hasta la meta más cercana. Debe ser **admisible**: nunca sobreestima el costo real.

Nota 1.5.1. Una heurística es **admisible** si $h(n) \leq h^*(n)$ para todo n , donde $h^*(n)$ es el costo real al llegar a la meta.

1.5.2. Greedy Best-First Search

Expande el nodo que parece más cercano a la meta según $h(n)$.

- Función de evaluación: $f(n) = h(n)$
- No es **óptimo** ni completo en general.
- Rápido en la práctica, puede quedar atrapado en mínimos locales.

1.5.3. A*

Combina el costo real del camino y la heurística:

$$f(n) = g(n) + h(n)$$

- $g(n)$: costo del camino desde inicio hasta n
- $h(n)$: estimación del costo desde n hasta la meta
- **Completo y óptimo** si h es admisible (búsqueda en árbol) o consistente (búsqueda en grafo).

Definición 1.5.2 (Consistencia / Monotonía). h es consistente si para todo nodo n y sucesor n' :

$$h(n) \leq c(n, a, n') + h(n')$$

La consistencia implica admisibilidad.

Ejemplo 1.5.1 (Heurísticas comunes en una cuadrícula). Para un agente que se mueve en una cuadrícula 2D desde (x_1, y_1) hasta (x_2, y_2) :

- **Distancia Manhattan:** $h(n) = |x_1 - x_2| + |y_1 - y_2|$. Admisible si el agente solo se mueve en horizontal/vertical (no sobreestima, porque es exactamente el número mínimo de pasos sin obstáculos).
- **Distancia euclidiana:** $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. Admisible si se permite movimiento diagonal, ya que nunca es mayor que el camino real.

Usar Manhattan cuando solo hay 4 movimientos posibles evita sobreestimar; usarla con movimiento diagonal permitiría violar la admisibilidad, porque el camino real podría ser más corto que la suma de catetos.

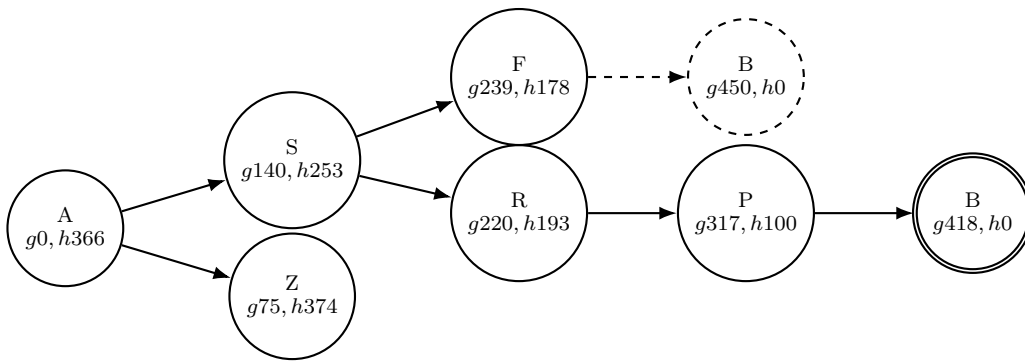


Figura 1.5: Taza de A* sobre el mapa de Rumania (heurística: distancia en línea recta a Bucarest). A* siempre expande el nodo de menor $f(n) = g(n) + h(n)$ de la frontera. Aunque Fagaras (F) llega primero a Bucarest con $f = 450$ (nodo punteado: se genera pero **se descarta**), Rimnicu (R) tiene $f = 220 + 193 = 413$, menor que $f(F) = 239 + 178 = 417$, así que A* explora primero $R \rightarrow$ Pitesti (P) con $f = 317 + 100 = 417$. Al final, la meta alcanzada por Pitesti tiene $f = 418$, menor que los 450 de la rama de Fagaras, así que esa es la **solución óptima** (nodo de doble borde).

Capítulo 2

Semana 2: Búsqueda Informada y Ambientes Complejos

Nota 2.0.1 (Eventos esta semana). ■ Publicación **Taller 1**

- **Examen 1** (al final de la semana)

2.1. Búsqueda Informada Avanzada

2.1.1. Búsqueda con Memoria Acotada

Iterative Deepening A* (IDA*)

Versión de A* que usa IDDFS con el valor de corte $f(n) = g(n) + h(n)$.

- Espacio: $\mathcal{O}(bd)$
- Completo y óptimo (con h admisible)

Recursive Best-First Search (RBFS)

Mantiene el mejor valor alternativo f en cada nivel de recursión.

- Espacio: $\mathcal{O}(bd)$
- Puede reexpandir nodos (regenera sub-árboles olvidados)

SMA* (Simplified Memory-Bounded A*)

Usa toda la memoria disponible; descarta el nodo con mayor f cuando la memoria está llena.

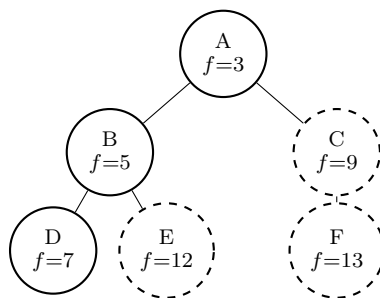


Figura 2.1: IDA* con umbral inicial $f \leq 7$ (el valor de h en la raíz): en la primera iteración se expanden A , B y D (todos con $f \leq 7$), y se podan C , E y F (nodos punteados, con $f > 7$). Si no se halla la meta, el umbral sube al menor f podado (aquí 9) y se repite la búsqueda desde cero.

2.1.2. Heurísticas: Dominancia y Creación

Definición 2.1.1 (Dominancia). h_2 domina a h_1 si $h_2(n) \geq h_1(n)$ para todo n y ambas son admisibles. Una heurística dominante expande menos nodos.

Formas de construir heurísticas admisibles:

1. **Relajación del problema:** eliminar restricciones del problema original.
2. **Bases de datos de patrones:** precomputar costos exactos de subproblemas.
3. **Máximo de heurísticas:** $h(n) = \max(h_1(n), h_2(n))$

Ejemplo 2.1.1 (8-puzzle: dos heurísticas admisibles). Sea n un estado del 8-puzzle donde las fichas 1, 2 y 3 no están en su casilla objetivo.

- $h_1(n)$ = número de **fichas fuera de lugar**. Si 3 fichas están mal ubicadas, $h_1(n) = 3$.
- $h_2(n)$ = suma de **distancias Manhattan** de cada ficha a su casilla objetivo. Si esas 3 fichas están, respectivamente, a 1, 2 y 3 movimientos de su lugar, $h_2(n) = 1 + 2 + 3 = 6$.

Ambas son admisibles (relajaciones del problema: h_1 ignora la restricción de que una ficha se mueva solo a una casilla adyacente vacía; h_2 ignora que solo puede moverse una ficha a la vez). Como $h_2(n) \geq h_1(n)$ siempre, h_2 **domina** a h_1 y expande menos nodos en A*/IDA*.

2.2. Búsqueda en Ambientes Complejos

2.2.1. Búsqueda Local

A diferencia de la búsqueda sistemática, la búsqueda local opera sobre un **estado actual** sin mantener el camino recorrido. Útil cuando el objetivo es el estado mismo (no el camino).

2.2.2. Hill Climbing (Ascenso de Colina)

Hill climbing mantiene un único estado actual y se mueve al vecino con mejor valor, deteniéndose cuando ningún vecino mejora (un óptimo local respecto al movimiento permitido).

Problemas:

- Máximos locales (no globales)
- Mesetas (*plateaus*) y crestas

Variantes: Hill climbing estocástico, reinicio aleatorio, *simulated annealing*.

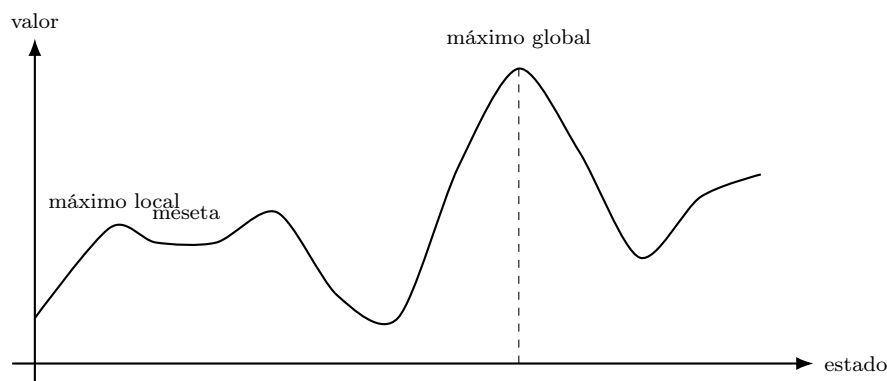


Figura 2.2: Paisaje de valores de estados: hill climbing puede quedar atrapado en el máximo local de la izquierda o en la meseta, sin llegar nunca al máximo global de la derecha, porque solo acepta movimientos que mejoran el valor actual.

2.2.3. Simulated Annealing

Permite movimientos malos con probabilidad $e^{\Delta E/T}$ donde T decrece con el tiempo. Garantiza convergencia al óptimo global si T baja suficientemente despacio.

Ejemplo 2.2.1 (Aceptar un movimiento malo). Si un movimiento empeora el valor en $\Delta E = -2$ y la temperatura actual es $T = 1$, la probabilidad de aceptarlo de todos modos es $e^{\Delta E/T} = e^{-2/1} \approx 0,135$ (13.5 %). Con $T = 4$ (temperatura más alta, típico al inicio), esa misma probabilidad sube a $e^{-2/4} \approx 0,607$ (60.7 %): al principio el algoritmo explora más libremente escapando de máximos locales; al final ($T \rightarrow 0$) se comporta como hill climbing puro.

2.2.4. Algoritmos Genéticos

Inspirados en la evolución biológica. Operan sobre una **población** de individuos (estados codificados como cadenas).

Operadores:

1. **Selección**: individuos con mayor aptitud (*fitness*) tienen más probabilidad de reproducirse.
2. **Cruzamiento** (*crossover*): combina dos individuos para producir descendencia.
3. **Mutación**: cambia aleatoriamente genes con baja probabilidad.

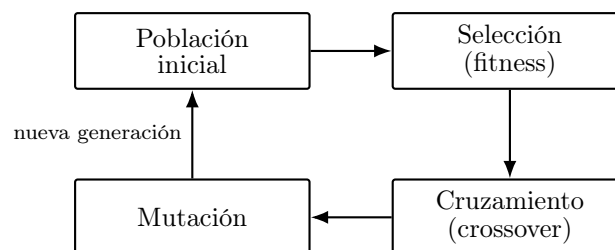


Figura 2.3: Ciclo de un algoritmo genético: de la población actual se seleccionan padres según su aptitud, se cruzan para producir hijos, se mutan con baja probabilidad, y la nueva generación reemplaza a la anterior.

Ejemplo 2.2.2 (Cruzamiento de un 8-reinas codificado). Si dos individuos del problema de las 8 reinas se codifican como cadenas de 8 dígitos (la fila de la reina en cada columna), por ejemplo 32752411 y 24748552, un cruzamiento de un punto tras la posición 3 produce el hijo 327 (del primero) + 48552 (del segundo) = 32748552, combinando la parte inicial de un padre con la parte final del otro.

2.3. Ejercicios de Búsqueda

Nota 2.3.1. Esta clase es de ejercicios y repaso para el **Examen 1**. Ver la *Guía de Ejercicios: Búsqueda Desinformada e Informada*.

2.3.1. Guía de Ejercicios — Resumen de Bloques

Bloque 1: Agentes Inteligentes

Caracterizar escenarios con PEAS, identificar tipo de agente y entorno.

Bloque 2: Formulación de Problemas

Definir estado inicial, modelo de transición, costo y test de meta para problemas concretos.

Bloque 3: Búsqueda No Informada

Aplicar BFS, DFS y UCS sobre grafos; analizar completitud y optimalidad.

Bloque 4: Búsqueda Informada y Heurísticas

Aplicar Greedy, A*, RBFS, SMA*; verificar admisibilidad y consistencia de heurísticas.

Ejercicio 2.3.1. Dado el grafo del Bloque 3 (Guía), aplica BFS, DFS y UCS. Indica el orden de expansión y el camino solución encontrado.

Solución:

Parte II

Razonamiento bajo Incertidumbre

Capítulo 3

Semana 3: Búsqueda Adversaria y Razonamiento bajo Incertidumbre

Nota 3.0.1 (Eventos esta semana). ■ **Entrega Taller 1**

3.1. Búsqueda Adversaria y Juegos I

3.1.1. Juegos como Problemas de Búsqueda

Un **juego** de dos jugadores, suma cero, de información perfecta se define como:

- S_0 : estado inicial
- $\text{PLAYER}(s)$: qué jugador mueve en estado s
- $\text{ACTIONS}(s)$: movimientos legales
- $\text{RESULT}(s, a)$: estado resultante
- $\text{IS-TERMINAL}(s)$: test de fin de juego
- $\text{UTILITY}(s, p)$: valor numérico del estado terminal para el jugador p

3.1.2. Minimax

MAX maximiza, MIN minimiza la utilidad.

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{si IS-TERMINAL}(s) \\ \max_a \text{MINIMAX}(\text{RESULT}(s, a)) & \text{si PLAYER}(s) = \text{MAX} \\ \min_a \text{MINIMAX}(\text{RESULT}(s, a)) & \text{si PLAYER}(s) = \text{MIN} \end{cases}$$

- Completo (en árbol finito) y óptimo contra un oponente óptimo.
- Tiempo: $\mathcal{O}(b^m)$ Espacio: $\mathcal{O}(bm)$

Ejemplo 3.1.1 (Minimax sobre un árbol de profundidad 2). En el árbol de la Figura siguiente, cada hoja es un estado terminal con su utilidad para MAX. Los valores se calculan de abajo hacia arriba: cada nodo MIN toma el mínimo de sus hijos, y la raíz (MAX) toma el máximo de los valores que le devuelven los nodos MIN.

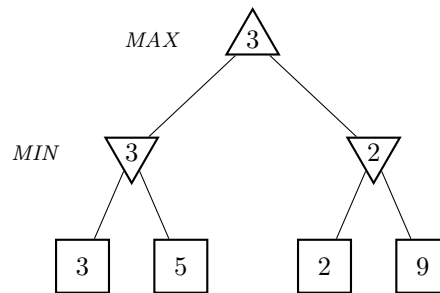


Figura 3.1: Árbol minimax: los nodos MIN eligen el menor de sus hijos ($\min(3, 5) = 3$ y $\min(2, 9) = 2$); la raíz MAX elige el mayor de esos valores ($\max(3, 2) = 3$), así que la rama izquierda es la jugada óptima para MAX.

3.2. Búsqueda Adversaria y Juegos II

3.2.1. Poda Alfa-Beta (α - β)

Elimina ramas del árbol que no pueden afectar la decisión final.

- α : mejor valor que MAX puede garantizar (lo que MAX ya tiene)
- β : mejor valor que MIN puede garantizar (lo que MIN ya tiene)
- Poda: si $v \geq \beta$ en nodo MAX $\Rightarrow$ cortar; si $v \leq \alpha$ en nodo MIN $\Rightarrow$ cortar.

Con orden perfecto de movimientos: $\mathcal{O}(b^{m/2})$ (duplica la profundidad manejable).

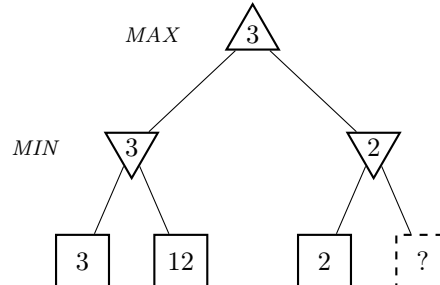


Figura 3.2: Poda alfa-beta: tras explorar la rama izquierda, MAX ya garantiza al menos $\alpha = 3$. Al explorar la rama derecha, MIN encuentra un hijo con valor 2, así que ya sabe que esa rama vale $\leq 2 < \alpha$: como MAX nunca elegiría un valor menor que 3 pudiendo garantizarlo, el segundo hijo de B (nodo punteado) se **poda** sin evaluarlo, sin cambiar la decisión final.

3.2.2. Juegos con Incertidumbre: Expectiminimax

Para juegos con elementos aleatorios (dados) se agregan **nodos azar**:

$$\text{EXPECTIMINIMAX}(s) = \sum_r P(r) \cdot \text{EXPECTIMINIMAX}(\text{RESULT}(s, r))$$

Ejemplo 3.2.1 (Nodo de azar con una moneda). Si en un nodo de azar una moneda justa decide entre dos resultados con utilidades 8 y 2, el valor del nodo de azar es $0,5 \cdot 8 + 0,5 \cdot 2 = 5$, sin importar si el nodo padre es MAX o MIN: el valor esperado se calcula *antes* de que el jugador siguiente decida.

3.2.3. Funciones de Evaluación

En juegos reales no se puede explorar hasta el final. Se usa una **función de evaluación heurística** $EVAL(s)$ que estima la utilidad desde un estado no terminal.

Características de una buena función de evaluación:

- Debe concordar con la utilidad real en estados terminales.
- Rápida de calcular.
- Debe reflejar las probabilidades reales de ganar.

3.2.4. Monte Carlo Tree Search (MCTS)

Estrategia basada en simulaciones aleatorias (*rollouts*). Cuatro fases: selección, expansión, simulación, retropropagación.

3.3. Razonamiento bajo Incertidumbre I

3.3.1. Probabilidad como Base del Razonamiento

Cuando el entorno es **parcialmente observable** o **estocástico**, se usa probabilidad para representar la incertidumbre.

Definición 3.3.1 (Probabilidad a priori y condicional). La **probabilidad a priori** (o incondicional) $P(A)$ representa el grado de creencia sin ninguna evidencia adicional.

La **probabilidad condicional** $P(A | B) = \frac{P(A \wedge B)}{P(B)}$ representa el grado de creencia dado que B es verdadero.

3.3.2. Regla de Bayes

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)}$$

- $P(A)$: probabilidad a priori de A (hipótesis)
- $P(B | A)$: verosimilitud de la evidencia B dado A
- $P(B)$: probabilidad marginal de B (normalizador)
- $P(A | B)$: probabilidad a posteriori

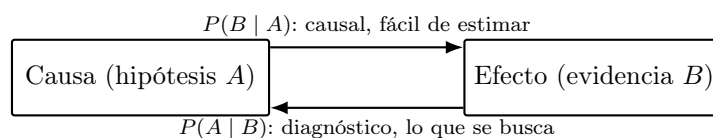


Figura 3.3: La regla de Bayes permite invertir el sentido del razonamiento: de un modelo *causal* $P(\text{efecto} | \text{causa})$, que suele conocerse de antemano (estadísticas, estudios clínicos), se obtiene la probabilidad *diagnóstica* $P(\text{causa} | \text{efecto})$, que es la que realmente interesa inferir.

Ejemplo 3.3.1 (Diagnóstico médico). Sea $M = \text{tiene meningitis}$ y $R = \text{tiene rigidez de cuello}$. Se sabe que $P(R | M) = 0,7$ (la meningitis causa rigidez de cuello el 70 % de las veces), $P(M) = 1/50000$ (prevalencia a priori) y $P(R) = 0,01$ (rigidez de cuello en la población general). Por la regla de Bayes:

$$P(M | R) = \frac{P(R | M) \cdot P(M)}{P(R)} = \frac{0,7 \times \frac{1}{50000}}{0,01} = 0,0014$$

Aunque la rigidez de cuello es un síntoma fuerte de meningitis (70 %), la probabilidad posterior sigue siendo baja porque la meningitis es muy rara a priori ($P(M)$ pequeño) y la rigidez de cuello es relativamente común por otras causas ($P(R)$ no tan pequeño).

3.3.3. Distribuciones de Probabilidad

Para variables aleatorias discretas, la **distribución conjunta completa** especifica $P(X_1, X_2, \dots, X_n)$ para todos los valores posibles. De ella se puede derivar cualquier probabilidad por marginalización y condicionamiento.

Marginalización: $P(X) = \sum_y P(X, Y = y)$

Independencia: $X \perp Y \Leftrightarrow P(X, Y) = P(X) \cdot P(Y)$

Independencia condicional: $X \perp Y | Z \Leftrightarrow P(X | Y, Z) = P(X | Z)$

3.4. Razonamiento bajo Incertidumbre II

3.4.1. Inferencia por Enumeración

Para cualquier consulta $P(X | \mathbf{e})$:

$$P(X | \mathbf{e}) = \alpha \sum_{\mathbf{y}} P(X, \mathbf{e}, \mathbf{y})$$

donde $\mathbf{y}$ son las variables ocultas y α es el normalizador.

3.4.2. Variables y Modelos

- **Variable aleatoria:** puede ser discreta o continua.
- **Distribución de probabilidad:** asigna probabilidades a todos los valores.
- **Tabla de probabilidad condicional (CPT):** especifica $P(X | \text{Parents}(X))$.

Ejemplo 3.4.1 (Enumeración sobre Dolor de muela, Caries y Detector). Sea la distribución conjunta completa $P(\text{Caries}, \text{Dolor}, \text{Detector})$:

	Dolor		¬Dolor	
	Detector	¬Detector	Detector	¬Detector
Caries	0.108	0.012	0.072	0.008
¬Caries	0.016	0.064	0.144	0.576

Cuadro 3.1: Distribución conjunta completa (suma de las 8 celdas = 1).

Para calcular $P(\text{Caries} | \text{Dolor})$ por enumeración, se **suma sobre la variable oculta** Detector, dejando fija la evidencia Dolor:

$$P(\text{Caries} | \text{Dolor}) = \alpha \sum_{\text{det}} P(\text{Caries}, \text{Dolor}, \text{det}) = \alpha (0,108 + 0,012) = \alpha (0,12)$$

De forma análoga, $P(\neg \text{Caries} \mid \text{Dolor}) = \alpha(0,016 + 0,064) = \alpha(0,08)$. Normalizando con $\alpha = 1/(0,12+0,08) = 5$: $P(\text{Caries} \mid \text{Dolor}) = 0,6$ y $P(\neg \text{Caries} \mid \text{Dolor}) = 0,4$. Este es el patrón general de inferencia por enumeración: fijar la evidencia, sumar sobre las variables ocultas, y normalizar al final.

Capítulo 4

Semana 4: Redes Bayesianas, MDPs y CSP

4.1. Redes Bayesianas

Definición 4.1.1 (Red Bayesiana). Una red bayesiana es un grafo acíclico dirigido (DAG) donde:

- Cada nodo representa una variable aleatoria.
- Los arcos representan dependencias directas.
- Cada nodo X_i tiene una CPT: $P(X_i \mid \text{Parents}(X_i))$.

4.1.1. Semántica

La distribución conjunta se factoriza como:

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Parents}(X_i))$$

4.1.2. Inferencia Exacta

- **Enumeración:** suma sobre todas las variables ocultas. Exponencial en el peor caso.
- **Eliminación de variables:** orden de eliminación importa para la eficiencia.

4.1.3. Inferencia Aproximada

- **Muestreo directo:** muestrea según el orden topológico de la red.
- **Muestreo por rechazo:** descarta muestras inconsistentes con la evidencia.
- **Ponderación por verosimilitud:** fija la evidencia, pondera las muestras.
- **MCMC / Gibbs Sampling:** muestrea cada variable condicionada al resto.

Ejemplo 4.1.1 (Red de alarma). Un sistema de alarma antirrobo se activa por un robo o (raramente) por un temblor. Dos vecinos, John y Mary, llaman si oyen la alarma, pero a veces confunden el teléfono con la alarma o no la oyen.

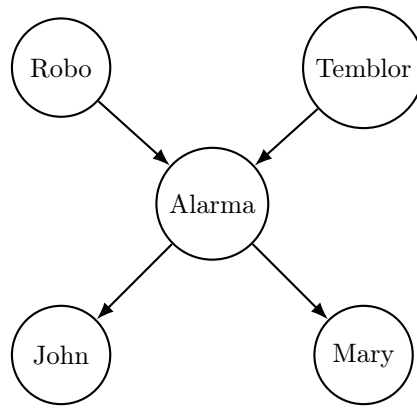


Figura 4.1: Red bayesiana Robo-Temblor-Alarma-John-Mary. Robo y Temblor son independientes a priori, pero se vuelven **dependientes** al condicionar sobre Alarma (*explaining away*); John y Mary son condicionalmente independientes dado Alarma.

$P(\text{Robo})$	$P(\text{Temblo})$	Robo	Temblo	$P(\text{Alarma})$
		V	V	0.95
		V	F	0.94
0.001	0.002	F	V	0.29
		F	F	0.001

Alarma	$P(\text{John})$
V	0.90
F	0.05

Alarma	$P(\text{Mary})$
V	0.70
F	0.01

Cuadro 4.1: CPTs de la red: la de Alarma tiene $2^2 = 4$ filas porque tiene 2 padres; John y Mary tienen 1 padre cada uno (Alarma), así que sus CPTs tienen 2 filas.

4.2. Procesos de Decisión de Markov (MDPs)

Definición 4.2.1 (MDP). Un MDP se define por la tupla (S, A, P, R, γ) :

- S : conjunto de estados
- A : conjunto de acciones
- $P(s' \mid s, a)$: modelo de transición (probabilístico)
- $R(s, a, s')$: función de recompensa
- $\gamma \in [0, 1]$: factor de descuento

4.2.1. Propiedad de Markov

$$P(s_{t+1} \mid s_t, a_t) = P(s_{t+1} \mid s_0, a_0, \dots, s_t, a_t)$$

El futuro solo depende del estado presente, no del historial.

4.2.2. Política y Utilidad

- **Política** $\pi(s)$: función que asigna una acción a cada estado.
- **Utilidad** (retorno descontado): $U = \sum_{t=0}^{\infty} \gamma^t R_t$
- **Política óptima** π^* : maximiza la utilidad esperada.

4.2.3. Ecuación de Bellman

$$U^*(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U^*(s')]$$

4.2.4. Algoritmos de Solución

- **Value Iteration**: actualiza $U(s)$ iterativamente hasta convergencia.
- **Policy Iteration**: alterna entre evaluación de política y mejora de política.

Ejemplo 4.2.1 (Grid world 4×3). El agente se mueve en una cuadrícula de 4×3 celdas. La celda $(2, 2)$ es un muro (no se puede entrar). Los estados terminales son $(4, 3)$ con recompensa $+1$ y $(4, 2)$ con recompensa -1 ; cada movimiento a un estado no terminal cuesta $R(s) = -0,04$. Las acciones son deterministas en este ejemplo simplificado ($\gamma = 1$).

			+1
			-1
inicio			

Figura 4.2: Grid world clásico 4×3 : la celda gris en $(2, 2)$ es un muro; las metas están en $(4, 3)$ ($+1$) y $(4, 2)$ (-1). Cada paso no terminal cuesta $R(s) = -0,04$, lo que incentiva llegar rápido a la meta positiva.

Ejemplo 4.2.2 (Un paso de Value Iteration). Supóngase que, en una iteración anterior, $U(3, 3) = 0,85$ (celda justo a la izquierda de la meta $+1$) y que desde $(3, 2)$ la acción *arriba* tiene efecto determinista hacia $(3, 3)$. Aplicando la ecuación de Bellman:

$$U(3, 2) \leftarrow R(3, 2) + \gamma \cdot U(3, 3) = -0,04 + 1 \cdot 0,85 = 0,81$$

En cada iteración, los valores altos cerca de la meta $+1$ se van propagando hacia atrás, celda por celda, hasta que todos los $U(s)$ convergen.

4.3. Ejercicios: Razonamiento bajo Incertidumbre

Nota 4.3.1. Repaso de probabilidad, redes bayesianas y MDPs.

Ejercicio 4.3.1. Dada la red bayesiana del ejemplo (Robo-Temblor-Alarma-John-Mary), calcula $P(\text{Robo} \mid \text{John} = V, \text{Mary} = V)$ usando enumeración.

Solución: Por la regla de la cadena sobre el orden topológico de la red:

$$P(b \mid j, m) = \alpha \sum_e P(b)P(e) \sum_a P(a \mid b, e) P(j \mid a) P(m \mid a)$$

Para $b = V$ (usando las CPTs de la clase anterior):

$$(e=V): 0,002 \cdot [0,95 \cdot 0,9 \cdot 0,7 + 0,05 \cdot 0,05 \cdot 0,01] = 0,002 \cdot 0,598525 = 0,0011971$$

$$(e=F): 0,998 \cdot [0,94 \cdot 0,9 \cdot 0,7 + 0,06 \cdot 0,05 \cdot 0,01] = 0,998 \cdot 0,59223 = 0,5910575$$

Sumando y multiplicando por $P(b) = 0,001$: $P(b, j, m) = 0,001 \cdot 0,5922546 = 0,00059225$.

Repitiendo el mismo cálculo para $b = F$ (con $P(a \mid \neg b, e)$ de la CPT) se obtiene $P(\neg b, j, m) = 0,00149188$.

Normalizando con $\alpha = 1/(0,00059225 + 0,00149188) \approx 479,8$:

$$P(\text{Robo} \mid j, m) \approx 0,284 \quad P(\neg \text{Robo} \mid j, m) \approx 0,716$$

A pesar de que John y Mary llamaron, la probabilidad de robo sigue siendo menor que 0,5: la tasa base de robos es tan baja ($P(\text{Robo}) = 0,001$) que dos llamadas, aunque son evidencia fuerte, no son suficientes para hacerlo más probable que improbable.

4.4. Problemas de Satisfacción de Restricciones (CSP)

Definición 4.4.1 (CSP). Un CSP se define por:

- $X = \{X_1, \dots, X_n\}$: variables
- $D = \{D_1, \dots, D_n\}$: dominios de cada variable
- C : conjunto de restricciones sobre subconjuntos de variables

La **solución** es una asignación completa y consistente.

4.4.1. Tipos de Restricciones

- **Unarias:** $X_i \in \{v_1, v_2\}$
- **Binarias:** $X_i \neq X_j$
- **Globales:** involucran más de 2 variables (ej. AllDiff)

4.4.2. Búsqueda con Retroceso (Backtracking)

Asigna valores a variables una por una; retrocede cuando se viola una restricción.

Heurísticas para mejorar el backtracking:

1. **MRV** (Minimum Remaining Values): elegir la variable con menos valores posibles.
2. **Degree heuristic**: variable con más restricciones sobre variables no asignadas.
3. **Least Constraining Value**: valor que deja más opciones para las demás variables.

Ejemplo 4.4.1 (Coloreado de un mapa). Cuatro regiones W, X, Y, Z deben colorearse con {rojo, verde, azul} de modo que regiones vecinas tengan colores distintos. Las adyacencias son: $W-X$, $W-Y$, $X-Y$, $X-Z$, $Y-Z$.

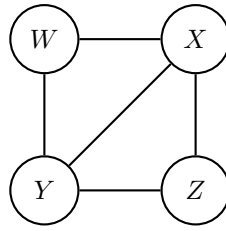


Figura 4.3: Grafo de restricciones del CSP: cada nodo es una variable (región) y cada arista es una restricción binaria de desigualdad con su vecina. X y Y tienen grado 3 (las más restringidas); son buenas candidatas para asignar primero según la heurística de grado.

Capítulo 5

Semana 5: CSP Avanzado y Agentes Lógicos

Nota 5.0.1 (Eventos esta semana). ■ Entrega Taller 2

- Examen 2

5.1. Problemas de Satisfacción de restricciones

5.1.1. Diagrama de Arbol

En la clase anterior se definió un CSP como una tripla (*variables, dominios, restricciones*) y se vió la **busqueda backtracking**, que es como un DFS + ordenamiento de variables + falla inmediata ante violación; junto con dos mejoras clave:

- **Filtrado** mediante *consistenciadearcos*; revisar si para cada valor de X, existe algún valor compatible en Y que detecta fallas inevitables antes que la simple revisión forward.
- **Ordenamiento** MRV (*Minimum Remaining Values*) para variables y LCV (*Least constraining value*) para valores

Ejemplo 5.1.1. Con dominios iniciales rojo,verde,azul y restricciones de desigualdad entre regiones vecinas, al forzar consistencia de arcos sobre el grafo WA NT SA Q NSW V se observa el efecto en cadena: Si el dominio de X pierde un valor, todos los vecinos de X se deben volver a

Arco revisado	Resultado
V → NSW	Consistente
SA → NSW	Consistente
NSW → SA	Inconsistente, se elimina azul del dominio de NSW
V → NSW	Como NSW cambi+o, se vuelve a ervisar se elimina rojo de V

Cuadro 5.1: Efecto en cadena al forzar consistencia de arcos.

revisar, porque una restriccion que antes era satisfacible puede dejar de serlo

Forzar consistencia de arcos sobre todo el CSP **no garantiza** encontrar la solución. al terminar el proceso puede que a quede exactamente una solución, b queden multiples soluciones sin decidir entre ellas o c el problema no tenga soluci´n y el algoritmo no lo sepa. Por eso la consistencia se arcos se corre dentro de una busqueda backtracking no en lugar de ella.

5.1.2. k-consistencia

La consistencia de arcos es apenas un caso particular de una idea más general, ir aumentando el grado de consistencia que se exige, de nodos a arcos, de arcos a trios de variables y así sucesivamente.

Definición 5.1.1. Un CSP es k -consistente si, para cualquier conjunto de $k-1$ variables y cualquier asignación consistente a esas $k-1$ variables, siempre existe un valor consistente para extender la asignación a una k -ésima variable cualquiera.

Nivel	Que garantiza
1-consistencia (consistencia de nodos)	Cada dominio ya respeta las restricciones unarias del nodo
2-consistencia (consistencia de arcos)	Toda asignación consistente a un nodo se extiende al otro
3-consistencia (consistencia de caminos)	Todo par de nodos consistentes se puede extender a un tercero
k -consistencia	Se cumple simultáneamente para $1, 2, \dots, k$

A mayor k , el filtrado poda más el árbol de búsqueda, pero también es **mas caro de computar**. No hay almuerzo gratis, es un tradeoff entre tiempo invertido en propagación de restricciones y tiempo invertido en backtracking.

5.1.3. Forward Checking

Es una alternativa más barata que forzar consistencia de arcos completa. Cada vez que se asigna un valor a X_i , se recorren las variables **vecinas no asignadas** de X_i y se elimina de sus dominios cualquier valor inconsistente con la asignación recién hecha.

Definición 5.1.2 (Forward Checking). Al asignar $X_i \leftarrow v$, para cada variable X_j vecina de X_i y aún sin asignar, eliminar de D_j todo valor que no sea consistente con $X_i = v$. Si algún D_j queda vacío, se detecta el fallo inmediatamente sin necesidad de llegar a esa variable.

La diferencia con la consistencia de arcos (AC-3) es que forward checking **solo revisa los arcos que salen de la variable recién asignada**, una sola vez. No propaga el efecto en cadena hacia el resto del grafo como sí lo hace AC-3 (que vuelve a revisar los vecinos de un vecino cuando su dominio cambia). Por eso forward checking es más barato pero detecta menos inconsistencias por adelantado; es un punto intermedio entre backtracking simple (0 chequeo hacia adelante) y AC-3 completo (propagación total).

5.1.4. Estructura del grafo de restricciones

Cuando el grafo de restricciones de un CSP es un **árbol** (sin ciclos), el problema se puede resolver en tiempo $\mathcal{O}(nd^2)$, sin ningún backtracking.

Definición 5.1.3 (CSP con estructura de árbol). 1. Elegir una variable como raíz y ordenar el resto en un orden topológico $X_1, \dots, X_n$ tal que cada variable aparezca después de su padre en el árbol.

2. Recorrer las variables **de atrás hacia adelante** (j de n a 2) y forzar consistencia de arco entre X_j y su padre $\text{Padre}(X_j)$, eliminando del dominio del padre los valores que ya no tengan soporte.

3. Recorrer las variables **de adelante hacia atrás** (j de 1 a n) asignando a cada X_j cualquier valor de su dominio consistente con el valor ya asignado a $\text{Padre}(X_j)$.

Como el grafo quedó dirigido y arco-consistente antes de asignar, el segundo recorrido nunca falla: nunca hay que retroceder.

La mayoría de los CSP reales no son árboles, pero se pueden **acercar** a esa estructura:

- **Cutset conditioning**: encontrar un conjunto pequeño de variables (*cutset*) tal que, al eliminarlas del grafo, lo que queda es un árbol. Se prueban todas las asignaciones consistentes posibles para el cutset y, para cada una, se resuelve el árbol restante en $\mathcal{O}(nd^2)$. Si el cutset tiene tamaño c , el costo total es $\mathcal{O}(d^c \cdot (n - c)d^2)$: exponencial en c , pero manejable si c es pequeño respecto a n .
- **Tree decomposition**: agrupar variables en subproblemas conectados en forma de árbol (cada subproblema se resuelve por separado) para luego combinar soluciones consistentes entre subproblemas vecinos.

5.1.5. Búsqueda local para CSP: Min-conflicts

A diferencia de backtracking (que construye la asignación variable por variable), min-conflicts parte de una **asignación completa** (posiblemente inconsistente) y la va reparando.

Definición 5.1.4 (Min-conflicts). 1. Generar una asignación inicial completa, asignando a cada variable un valor cualquiera de su dominio (puede violar restricciones).

2. Repetir hasta encontrar una solución o alcanzar un número máximo de pasos:

- a) Elegir al azar una variable X_i que esté en conflicto (viola alguna restricción).
- b) Reasignarle el valor de su dominio que **minimice el número de conflictos** con las demás variables (rompiendo empates al azar).

Ejemplo 5.1.2 (Min-conflicts en n-reinas). Con las 8 reinas ya puestas una por columna (asignación inicial con conflictos), en cada paso se elige una reina que ataque a otra y se mueve dentro de su columna a la fila con menos reinas atacándola. En la práctica, resuelve instancias de miles de reinas en muy pocos pasos.

Es muy eficiente en la práctica incluso para CSPs grandes, pero **no garantiza completitud**: puede quedar atascado y nunca encontrar una solución aunque exista (no hay noción de retroceso ni de dominios que se agotan).

5.2. Ejercicio: Coloreado del mapa de Australia

Ejercicio 5.2.1. Aplicar backtracking con MRV (y desempate por grado) y propagación tipo forward checking al problema de colorear el mapa de Australia con 3 colores {rojo, verde, azul}, exigiendo que regiones vecinas tengan colores distintos. Regiones: WA, NT, SA, Q, NSW, V, T (Tasmania es una isla, no tiene vecinos). Adyacencias: WA-NT, WA-SA, NT-SA, NT-Q, SA-Q, SA-NSW, SA-V, Q-NSW, NSW-V.

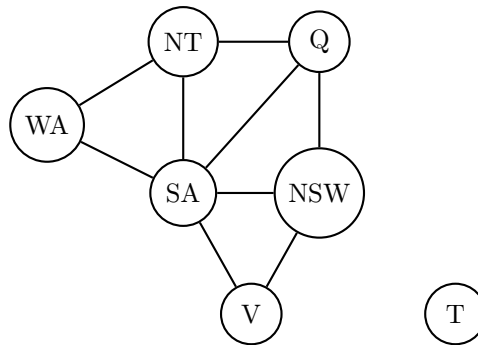


Figura 5.1: Grafo de restricciones del mapa de Australia: SA es vecina de 5 regiones (grado 5), la más restringida del grafo; T (Tasmania) no tiene aristas porque es una isla sin vecinos.

Solución:

Todas las variables (salvo T) parten con $|D| = 3$: hay empate en MRV, así que se desempata con la **heurística de grado** (más restricciones sobre variables sin asignar todavía). SA tiene grado 5, el más alto de todo el grafo, así que se asigna primero.

Paso	Asignación	Efecto de la propagación
1	SA = rojo	Se elimina rojo de WA, NT, Q, NSW, V
2	Q = verde	Se elimina verde de NT y de NSW
3	NT = azul	Se elimina azul de WA
4	WA = verde	Sin vecinos nuevos por revisar
5	NSW = azul	Se elimina azul de V
6	V = verde	Sin vecinos nuevos por revisar
7	T = rojo	T no tiene restricciones, cualquier color sirve

Cuadro 5.2: Traza de backtracking + MRV + forward checking sobre el mapa de Australia.

En cada paso, después de asignar, la variable que queda con el dominio más pequeño (a veces de tamaño 1) es la siguiente elegida por MRV, así que no hace falta romper más empates. La propagación evita que en este caso se necesite retroceder: ningún dominio queda vacío en el camino.

Solución final: SA = rojo, Q = verde, NT = azul, WA = verde, NSW = azul, V = verde, T = rojo (o cualquier color, T no tiene vecinos).

5.3. Agentes Lógicos: Introducción

5.3.1. Agentes basados en conocimiento

Un agente basado en conocimiento mantiene una **base de conocimiento** (KB): un conjunto de oraciones en algún lenguaje de representación que expresan hechos sobre el mundo. El agente decide qué acción tomar consultando (ASK) e incorporando percepciones nuevas (TELL) a la KB, usando un **motor de inferencia**.

Definición 5.3.1 (Agente basado en conocimiento). En cada ciclo el agente: (1) le dice (TELL) a la KB lo que percibió, (2) le pregunta (ASK) cuál es la mejor acción dado lo que sabe, y (3) le dice a la KB qué acción ejecutó. Todo el conocimiento sobre el dominio vive en la KB, no en el código del agente, así que se puede razonar y no solo reaccionar.

$KB \vdash \alpha$ (la KB **deriva** α sintácticamente, con un procedimiento de inferencia)

Un procedimiento de inferencia es **sólido** (*sound*) si solo deriva oraciones que realmente son consecuencia lógica de la KB, y es **completo** si es capaz de derivar toda oración que sea consecuencia lógica.

5.3.2. Lógica proposicional

Sintaxis: oraciones atómicas (símbolos proposicionales, cada uno verdadero o falso) combinadas con conectivos lógicos $\neg$ (negación), $\wedge$ (conjunción), $\vee$ (disyunción), $\Rightarrow$ (implicación), $\Leftrightarrow$ (bicondicional), para formar oraciones complejas.

Semántica: un **modelo** es una asignación de verdadero/falso a cada símbolo proposicional. Dado un modelo, el valor de verdad de cualquier oración se calcula recursivamente con las tablas de verdad de los conectivos.

- Una oración es **válida** (tautología) si es verdadera en **todos** los modelos (ej. $P \vee \neg P$).
- Una oración es **satisfacible** si es verdadera en **al menos un** modelo.
- Una oración es **insatisfacible** si no es verdadera en ningún modelo (ej. $P \wedge \neg P$).
- **Consecuencia lógica (entailment):** $KB \models \alpha$ si en todo modelo donde KB es verdadera, α también lo es. Equivalentemente, $KB \models \alpha$ si y solo si $KB \wedge \neg \alpha$ es insatisfacible.

Ejemplo 5.3.1 (Model checking). Para decidir si $KB \models \alpha$ por enumeración de modelos: se listan todas las combinaciones de verdad de los símbolos que aparecen en KB y en α (tabla de verdad completa), y se revisa que en toda fila donde KB sea verdadera, α también lo sea. Es correcto pero exponencial en el número de símbolos.

5.3.3. Formas normales: CNF y DNF

- **CNF** (Forma Normal Conjuntiva): conjunción de cláusulas, donde cada cláusula es una disyunción de literales. Es la forma que usan la mayoría de los algoritmos de inferencia (resolución, DPLL, etc.).
- **DNF** (Forma Normal Disyuntiva): disyunción de términos, donde cada término es una conjunción de literales.

Pasos para convertir cualquier oración a CNF:

1. Eliminar $\Leftrightarrow$: reemplazar $\alpha \Leftrightarrow \beta$ por $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.
2. Eliminar $\Rightarrow$: reemplazar $\alpha \Rightarrow \beta$ por $\neg \alpha \vee \beta$.
3. Meter las negaciones hacia adentro usando De Morgan ($\neg(\alpha \wedge \beta) \equiv \neg \alpha \vee \neg \beta$, $\neg(\alpha \vee \beta) \equiv \neg \alpha \wedge \neg \beta$) y eliminar dobles negaciones.
4. Distribuir $\vee$ sobre $\wedge$ hasta que quede una conjunción de disyunciones de literales.

5.3.4. El mundo del Wumpus

Ambiente clásico para introducir agentes lógicos: una cueva en cuadrícula (típicamente 4×4), con un Wumpus que mata al agente si entra en su casilla, pozos que lo hacen caer, y oro que debe recoger y sacar. El agente empieza en (1, 1) y solo percibe su casilla actual.

- **Percepciones:** *stench* (hedor, si el Wumpus está en una casilla adyacente), *breeze* (brisa, si hay un pozo adyacente), *glitter* (brillo, si hay oro en la casilla), *bump* (chocó con pared), *scream* (grito, al matar al Wumpus con una flecha).

- **Acciones:** avanzar, girar izquierda/derecha, agarrar, disparar, subir (para salir de la cueva).

seguro	?		
inicio	seguro brisa	?	

Figura 5.2: Estado de conocimiento tras percibir en (1,1) sin hedor ni brisa (por lo que (1,2) y (2,1) son seguras), y luego en (2,1) brisa sin hedor: hay un pozo en (2,2) o en (3,1), pero no se sabe cuál con la información acumulada hasta ahora.

Ejemplo 5.3.2 (Inferencia en el mundo del Wumpus). Si en (1,1) el agente no percibe ni hedor ni brisa, la regla $Breeze_{(x,y)} \Leftrightarrow \bigvee Pit_{vecino}$ (y la equivalente para *Stench* y el Wumpus) permite concluir que **ninguna** casilla vecina, (1,2) y (2,1), tiene pozo ni Wumpus: son seguras para avanzar. Si luego en (2,1) el agente sí percibe brisa (pero no hedor), infiere que hay un pozo en (3,1) o en (2,2), pero **no sabe en cuál** de los dos sin más información; ese es justamente el tipo de razonamiento bajo información parcial que la KB tiene que resolver combinando todas las percepciones acumuladas, no solo la última.

Parte III

Lógica y Representación del Conocimiento

Capítulo 6

Semana 6: Lógica y Representación del Conocimiento

6.1. Agentes Lógicos II: Inferencia Proposicional

6.1.1. Reglas de Inferencia

Regla	Forma
Modus Ponens	$\alpha \Rightarrow \beta, \alpha \vdash \beta$
Modus Tollens	$\alpha \Rightarrow \beta, \neg\beta \vdash \neg\alpha$
Resolución	$(\ell_1 \vee \dots \vee \ell_k), (\neg\ell_1 \vee m_1 \vee \dots) \vdash (m_1 \vee \dots)$

6.1.2. Resolución

Algoritmo de inferencia completo para lógica proposicional en CNF:

1. Convertir $KB \wedge \neg\alpha$ a CNF.
2. Aplicar resolución hasta obtener la cláusula vacía (contradicción) o no poder seguir.
3. Si se obtiene contradicción $\Rightarrow KB \models \alpha$.

6.1.3. DPLL y WalkSAT

- **DPLL**: backtracking completo sobre asignaciones de verdad, con propagación de cláusulas unitarias.
- **WalkSAT**: búsqueda local incompleta, eficaz en la práctica para instancias satisfacibles.

6.1.4. Ejemplo: Refutación por Resolución

Ejemplo 6.1.1 (Refutación). Sea $KB = \{P, P \Rightarrow Q, Q \Rightarrow R\}$ y se quiere probar $KB \models R$. Se convierte $KB \wedge \neg R$ a CNF: $\{P, \neg P \vee Q, \neg Q \vee R, \neg R\}$, y se aplica resolución hasta obtener la cláusula vacía $\square$ (una contradicción), lo que confirma el entailment.

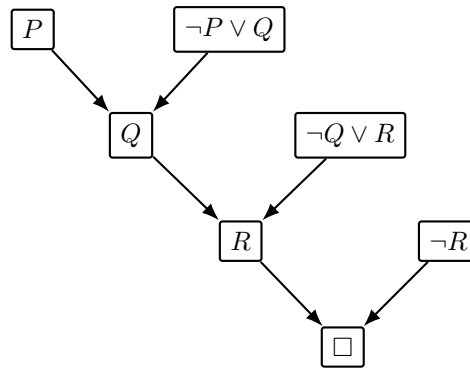


Figura 6.1: Árbol de refutación por resolución: cada nodo se obtiene combinando los dos que apuntan hacia él; llegar a la cláusula vacía $\square$ demuestra que $KB \wedge \neg R$ es insatisfacible, es decir, $KB \models R$.

Ejercicio 6.1.1. Sea $KB = \{A \vee B, \neg A \vee C, \neg B \vee C, \neg C \vee D\}$. Se quiere probar $KB \models D$ por refutación.

1. Escriba la cláusula que se agrega a KB al negar la meta.
2. Aplique resolución sobre $A \vee B$ y $\neg A \vee C$, y por separado sobre $\neg B \vee C$, para derivar una cláusula con únicamente C .
3. Combine el resultado anterior con $\neg C \vee D$ y con la meta negada para llegar a la cláusula vacía.
4. Dibuje el árbol de refutación completo, análogo al de la Figura anterior.

6.2. Lógica de Primer Orden (LPO)

6.2.1. Sintaxis

Elementos de LPO:

- **Constantes:** *Juan, UNIANDES, ...*
- **Variables:** *x, y, z, ...*
- **Predicados:** *Estudiante(x), Mayor(x, y)*
- **Funciones:** *Padre(x), Suma(x, y)*
- **Cuantificadores:** $\forall$ (universal), $\exists$ (existencial)
- **Conectivos:** $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

6.2.2. Semántica

Un **modelo** en LPO especifica:

- El dominio de objetos
- La interpretación de constantes, predicados y funciones

6.2.3. Cuantificadores

$$\forall x P(x) \equiv \neg \exists x \neg P(x)$$

$$\exists x P(x) \equiv \neg \forall x \neg P(x)$$

Nota 6.2.1. $\forall x (P(x) \Rightarrow Q(x))$ es diferente de $\forall x (P(x) \wedge Q(x))$. El cuantificador universal normalmente usa $\Rightarrow$; el existencial usa $\wedge$.

6.2.4. Ejemplo: Modelo en LPO

Ejemplo 6.2.1 (Modelo simple). Sea el dominio $D = \{\text{Juan, Maria, Pedro}\}$ y el predicado binario $\text{Padre}(x, y)$ (x es padre de y). La interpretación de la Figura siguiente satisface $\text{Padre}(\text{Juan, Maria})$ y $\text{Padre}(\text{Juan, Pedro})$, y ninguna otra pareja.

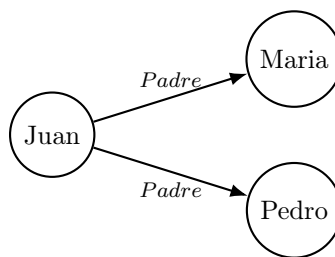


Figura 6.2: Modelo con dominio $\{\text{Juan, Maria, Pedro}\}$: cada flecha es una pareja (x, y) para la cual $\text{Padre}(x, y)$ es verdadero en este modelo. Cambiar las flechas cambia el modelo sin cambiar la sintaxis de la oración.

Ejercicio 6.2.1. Usando los predicados $\text{Estudiante}(x)$, $\text{Curso}(y)$, $\text{Aprueba}(x, y)$ y $\text{Certificado}(x, y)$, traduzca a LPO:

1. Todo estudiante que aprueba un curso recibe un certificado de ese curso.
2. Existe al menos un estudiante que no ha aprobado ningún curso.
3. Ningún estudiante aprueba todos los cursos.

Para cada oración, indique si el cuantificador principal es $\forall$ o $\exists$ y justifique si el conectivo correcto dentro de su alcance es $\Rightarrow$ o $\wedge$.

6.3. LPO e Inferencia I

6.3.1. Bases de Conocimiento en LPO

Las bases de conocimiento en LPO pueden expresar relaciones complejas de forma compacta, a diferencia de la lógica proposicional.

6.3.2. Unificación

Para aplicar reglas de inferencia en LPO se necesita **unificar** expresiones:

$$\text{UNIFY}(\alpha, \beta) = \theta \text{ tal que } \text{SUBST}(\theta, \alpha) = \text{SUBST}(\theta, \beta)$$

Algoritmo: recorre la estructura de ambas expresiones, construyendo sustituciones. Devuelve *fallo* si no hay unificador.

6.3.3. Inferencia con Cuantificadores

- **Instanciación universal:** $\forall x \alpha(x) \vdash \alpha(c)$ para cualquier constante c .
- **Instanciación existencial:** $\exists x \alpha(x) \vdash \alpha(k)$ donde k es una nueva constante de Skolem.
- **Skolemización:** eliminar cuantificadores existenciales introduciendo constantes/funciones de Skolem.

6.3.4. Forma Normal de Clausura (CNF en LPO)

Pasos para convertir a CNF:

1. Eliminar bicondicionales e implicaciones.
2. Mover $\neg$ hacia adentro (De Morgan, reglas de cuantificadores).
3. Estandarizar variables.
4. Skolemizar.
5. Eliminar cuantificadores universales.
6. Distribuir $\vee$ sobre $\wedge$.

6.3.5. Ejemplo: Unificación

Ejemplo 6.3.1 (Unificación de dos átomos). Para unificar $Padre(x, Juan)$ con $Padre(Pedro, y)$, el algoritmo recorre ambas expresiones posición por posición: en la primera posición x debe sustituirse por Pedro; en la segunda, y debe sustituirse por Juan. El unificador más general es $\theta = \{x/Pedro, y/Juan\}$.

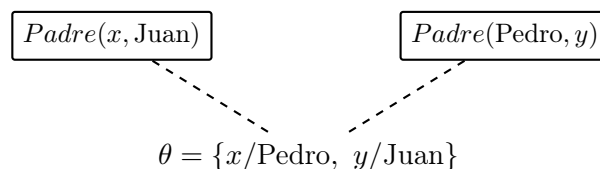


Figura 6.3: Unificación de $Padre(x, Juan)$ y $Padre(Pedro, y)$: aplicar θ a ambas expresiones produce la misma expresión, $Padre(Pedro, Juan)$.

- Ejercicio 6.3.1.**
1. Encuentre, si existe, el unificador más general de $Quiere(x, Helado(x))$ y $Quiere(Juan, Helado(Vainilla))$.
 2. Encuentre el unificador de $Hermano(x, y)$ y $Hermano(Juan, y)$. ¿Por qué es necesario estandarizar variables antes de unificar dos cláusulas que provienen de la misma regla?
 3. Convierta a CNF: $\forall x (Estudiante(x) \Rightarrow \exists y (Curso(y) \wedge Inscrito(x, y)))$, indicando en qué paso aparece la función de Skolem y qué representa.

6.4. Inferencia en LPO II

6.4.1. Encadenamiento hacia adelante (Forward Chaining)

Aplica reglas de la forma $P_1 \wedge \dots \wedge P_n \Rightarrow Q$ para derivar nuevos hechos hasta alcanzar la meta.

- Completo para bases de conocimiento de **cláusulas de Horn**.
- Dirigido por los datos.

6.4.2. Encadenamiento hacia atrás (Backward Chaining)

Parte de la meta y trabaja hacia atrás buscando hechos o submetas.

- Completo para cláusulas de Horn.
- Base de Prolog.
- Puede ciclar si no se controla.

6.4.3. Resolución en LPO

Generaliza la resolución proposicional usando unificación:

$$\frac{\ell_1 \vee \dots \vee \ell_k \quad \neg m_1 \vee \dots \vee \neg m_j}{\text{SUBST}(\theta, \ell_1 \vee \dots \vee m_j)}$$

donde $\theta = \text{UNIFY}(\ell_i, m_k)$.

- **Refutación por resolución:** completa para LPO (Teorema de Completitud de Gödel).

6.4.4. Ejemplo: El caso de Nono (¿Es West un criminal?)

Ejemplo 6.4.1 (Base de conocimiento). 1. $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

2. $\text{Owns}(\text{Nono}, M1)$

3. $\text{Missile}(M1)$

4. $\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$

5. $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$

6. $\text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$

7. $\text{American}(\text{West})$

8. $\text{Enemy}(\text{Nono}, \text{America})$

Meta: probar $\text{Criminal}(\text{West})$.

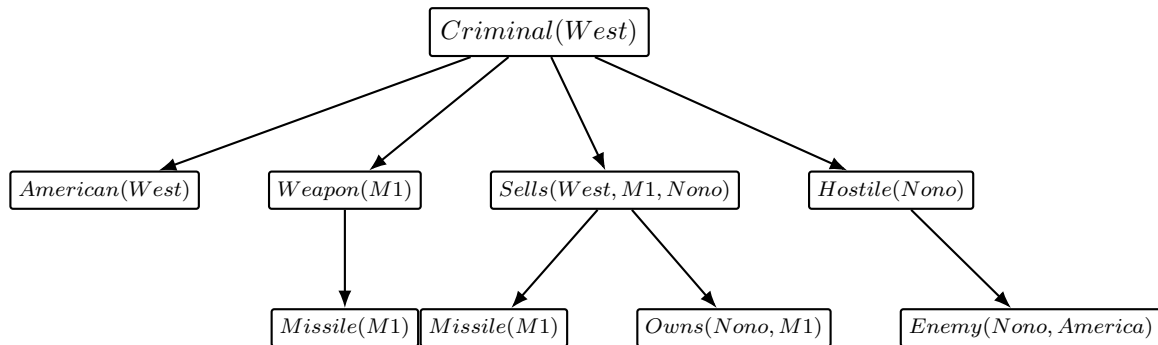


Figura 6.4: Árbol de encadenamiento hacia atrás para $\text{Criminal}(\text{West})$: los cuatro hijos de la raíz son las **cuatro submetas** que exige la conjunción de la regla 1 (todas deben probarse), y $\text{Missile}(M1)$ y $\text{Owns}(\text{Nono}, M1)$ son las dos submetas que exige la regla 4. Cada hoja es un hecho que ya está en la KB (reglas 2, 3, 7 y 8).

Submeta	Regla utilizada	Qué se obtiene
<i>Criminal(West)</i>	Regla 1	Se descompone en 4 submetas (AND)
<i>American(West)</i>	Regla 7 (hecho)	Submeta resuelta directamente
<i>Weapon(M1)</i>	Regla 5	Se reduce a la submeta <i>Missile(M1)</i>
<i>Missile(M1)</i>	Regla 3 (hecho)	Submeta resuelta directamente
<i>Sells(West, M1, Nono)</i>	Regla 4	Se reduce a <i>Missile(M1) ∧ Owns(Nono, M1)</i>
<i>Owns(Nono, M1)</i>	Regla 2 (hecho)	Submeta resuelta directamente
<i>Hostile(Nono)</i>	Regla 6	Se reduce a <i>Enemy(Nono, America)</i>
<i>Enemy(Nono, America)</i>	Regla 8 (hecho)	Submeta resuelta directamente

Cuadro 6.1: Traza del encadenamiento hacia atrás: en cada fila se indica qué regla dispara la reducción y qué submeta nueva (o hecho) se obtiene, hasta que todas las hojas quedan probadas y *Criminal(West)* queda demostrado.

Ejercicio 6.4.1. 1. Usando la misma KB, construya la traza de **encadenamiento hacia adelante**: parta de los hechos (reglas 2, 3, 7, 8) y muestre, en una tabla como la anterior, en qué orden se disparan las reglas 5, 6, 4 y 1 hasta derivar *Criminal(West)*.

2. Compare las dos trazas: ¿por qué el encadenamiento hacia atrás solo explora los hechos relevantes para la meta, mientras que el hacia adelante puede derivar hechos que nunca se usan?
3. Dado el hecho adicional *Enemy(Nono, UK)* (en vez de *America*), ¿sigue siendo posible probar *Criminal(West)* con esta KB? Justifique cuál regla falla.

Ejercicio 6.4.2 (Resolución en LPO). Sean las cláusulas

$$\neg \text{Ama}(x, \text{Padre}(x)) \vee \text{Feliz}(x) \quad \text{y} \quad \text{Ama}(\text{Juan}, \text{Padre}(\text{Juan})).$$

1. Encuentre el unificador θ entre los literales complementarios.
2. Aplique $\text{SUBST}(\theta, \cdot)$ a la cláusula resolvente resultante.
3. ¿Qué hecho queda demostrado?

6.5. Ejercicios: Guía de Agentes Lógicos (con solución paso a paso)

Esta sección resuelve, de principio a fin, la *Guía de Ejercicios: Agentes Lógicos*. Cada respuesta se justifica explícitamente; en los casos falsos se da la versión corregida, y en los casos de tablas de verdad o resolución se muestra cada paso.

6.5.1. Bloque 1: Conceptos Fundamentales de Agentes Lógicos

Parte A — Consecuencia Lógica y Modelos

1. “Una base de conocimiento KB implica lógicamente una sentencia α (escrito $KB \models \alpha$) si y solo si α es verdadera en todo modelo en el que KB es verdadera.”

Verdadero. Es exactamente la definición semántica de consecuencia lógica: un modelo de KB es una interpretación donde toda sentencia de KB es verdadera, y $KB \models \alpha$ exige que α también lo sea en *todos* esos modelos.

2. “Si $KB \models \alpha$, entonces α debe ser verdadera en todos los mundos posibles, sin importar lo que diga KB .”
Falso. α solo tiene que ser verdadera en los modelos donde KB también es verdadera; en los modelos donde KB es falsa, α puede ser falsa sin violar $KB \models \alpha$.
Corrección: Si $KB \models \alpha$, entonces α es verdadera en todo modelo en el que KB es verdadera (no en todo modelo posible sin restricción).
3. “Una sentencia es válida (tautología) si es verdadera en algunos los modelos posibles.”
Falso. Eso describe *satisfacibilidad*, no validez.
Corrección: Una sentencia es válida (tautología) si es verdadera en **todos** los modelos posibles.
4. “Una sentencia es satisfacible si existe al menos un modelo en el que es verdadera.”
Verdadero. Es la definición estándar de satisfacibilidad.
5. “Si una sentencia es insatisfacible, entonces su negación es satisfacible.”
Verdadero. Si α es insatisfacible, es falsa en todo modelo, por lo tanto $\neg\alpha$ es verdadera en todo modelo (es decir, $\neg\alpha$ es válida). Toda sentencia válida es, en particular, satisfacible (verdadera en al menos un modelo: todos ellos).

Parte B — Algoritmos de Inferencia y Verificación de Modelos

1. “El algoritmo *TT-Implica?* enumera todos los modelos posibles para verificar si $KB \models \alpha$, siendo correcto y completo pero exponencial en el número de símbolos.”
Verdadero. Con n símbolos proposicionales hay 2^n asignaciones de verdad posibles; *TT-Implica?* las recorre todas, por lo que es $\mathcal{O}(2^n)$, pero al ser una enumeración exhaustiva es correcto (sound) y completo.
2. “Un procedimiento de prueba es correcto (sound) si sólo deriva sentencias que son consecuencia lógica de KB .”
Verdadero. Esa es la definición de *soundness*: nunca deriva una sentencia falsa a partir de premisas verdaderas. (La *completitud* es la propiedad complementaria: deriva *todas* las sentencias que son consecuencia lógica.)
3. “Modus Ponens establece: si $\alpha \Rightarrow \beta$ está en KB , y β es verdadera, entonces podemos inferir α .”
Falso. Esto es la falacia de *afirmar el consecuente*, una regla inválida.
Corrección: Modus Ponens establece que si $\alpha \Rightarrow \beta$ está en KB y α es verdadera, entonces podemos inferir β .
4. “La regla de resolución establece que de las cláusulas $(\ell_1 \vee \dots \vee \ell_k \vee m)$ y $(\neg m \vee n_1 \vee \dots \vee n_j)$ se puede derivar $(\ell_1 \vee \dots \vee \ell_k \vee n_1 \vee \dots \vee n_j)$.”
Verdadero. Es exactamente la regla de resolución binaria: se cancela el par complementario $m/\neg m$ y se unen el resto de los literales de ambas cláusulas.
5. “El algoritmo *DPLL* es una mejora sobre la enumeración básica de tablas de verdad para verificar satisfacibilidad.”
Verdadero. *DPLL* añade propagación de cláusulas unitarias, eliminación de literales puros y poda temprana (*early termination*) sobre el backtracking de fuerza bruta, evitando explorar ramas completas del árbol de asignaciones cuando ya se sabe que fallarán.
6. “El encadenamiento hacia adelante está orientado a metas: trabaja hacia atrás desde una consulta para determinar si KB la soporta.”
Falso. Esa descripción corresponde al encadenamiento **hacia atrás**.
Corrección: El encadenamiento hacia adelante está orientado a **datos**: parte de los hechos

conocidos en KB y dispara reglas repetidamente hasta derivar la consulta (o hasta que no se puedan agregar más hechos).

Parte C — Reglas de Inferencia

1. De $(P \Rightarrow Q)$ y P , concluir Q .
Nombre: Modus Ponens. **Válida:** Sí. Si $P \Rightarrow Q$ es verdadera y P es verdadera, la única fila de la tabla de verdad compatible con $P \Rightarrow Q$ verdadera y P verdadera tiene Q verdadera.
2. De $(P \Rightarrow Q)$ y $\neg Q$, concluir $\neg P$.
Nombre: Modus Tollens. **Válida:** Sí. Si P fuera verdadera, $P \Rightarrow Q$ obligaría a Q verdadera, contradiciendo $\neg Q$; por lo tanto P debe ser falsa.
3. De P y Q , concluir $(P \wedge Q)$.
Nombre: Introducción de la conjunción (And-Introduction). **Válida:** Sí, por definición de $\wedge$.
4. De $(P \vee Q)$ y $\neg P$, concluir Q .
Nombre: Silogismo disyuntivo (resolución unitaria). **Válida:** Sí. Como P es falsa, para que $P \vee Q$ sea verdadera, Q debe serlo.
5. De $(P \Rightarrow Q)$ y $(Q \Rightarrow R)$, concluir $(P \Rightarrow R)$.
Nombre: Silogismo hipotético (transitividad de la implicación). **Válida:** Sí (se demuestra formalmente en el Bloque 4, Problema 4.1.3, convirtiendo la sentencia a FNC).
6. De $(P \Rightarrow Q)$ y $\neg P$, concluir $\neg Q$.
Nombre: Negación del antecedente (falacia). **Válida:** No, es **inválida**. **Contraejemplo:** $P = F$, $Q = T$: $P \Rightarrow Q$ es verdadera y $\neg P$ es verdadera, pero $\neg Q$ es falsa.

6.5.2. Bloque 2: Validez y Satisfacibilidad

Problema 2.1 — Validez y Satisfacibilidad (Ej. 13, Cap. 7 AIMA)

1. $\frac{Humo \Rightarrow Humo.}{Humo \quad Humo \Rightarrow Humo}$

T	T
F	T

Todas las filas son verdaderas $\Rightarrow$ **Válida** (instancia del esquema $\alpha \Rightarrow \alpha$, siempre tautológico).
2. $\frac{Humo \Rightarrow Fuego.}{Humo \quad Fuego \quad Humo \Rightarrow Fuego}$

T	T	T
T	F	F
F	T	T
F	F	T

Hay filas verdaderas y una falsa $\Rightarrow$ **Satisfacible, pero no válida** (contingente).
3. $(Humo \Rightarrow Fuego) \Rightarrow (\neg Humo \Rightarrow \neg Fuego)$.

<i>Humo</i>	<i>Fuego</i>	$H \Rightarrow F$	$\neg H$	$\neg F$	$(H \Rightarrow F) \Rightarrow (\neg H \Rightarrow \neg F)$
T	T	T	F	F	T
T	F	F	F	T	T
F	T	T	T	F	F
F	F	T	T	T	T

La fila $Humo = F, Fuego = T$ da Falso, y las demás dan Verdadero $\Rightarrow$ **Satisfacible, pero no válida** (contingente).

4. $Humo \vee Fuego \vee \neg Fuego$.

El subtérmino $Fuego \vee \neg Fuego$ ya es una tautología (verdadero sin importar el valor de $Fuego$), así que toda la disyunción es verdadera en las 4 filas posibles de $(Humo, Fuego) \Rightarrow$ **Válida**.

5. $((Humo \wedge Calor) \Rightarrow Fuego) \Leftrightarrow ((Humo \Rightarrow Fuego) \vee (Calor \Rightarrow Fuego))$.

<i>H</i>	<i>C</i>	<i>Fu</i>	$H \wedge C$	$A = (H \wedge C) \Rightarrow Fu$	$H \Rightarrow Fu$	$C \Rightarrow Fu$	<i>B</i> (su OR)	$A \Leftrightarrow B$
T	T	T	T		T	T	T	T
T	T	F	T		F	F	F	T
T	F	T	F		T	T	T	T
T	F	F	F		F	T	T	T
F	T	T	F		T	T	T	T
F	T	F	F		T	F	T	T
F	F	T	F		T	T	T	T
F	F	F	F		T	T	T	T

Las 8 filas dan Verdadero $\Rightarrow$ **Válida**. (Se puede confirmar algebraicamente: $(H \wedge C) \Rightarrow Fu \equiv \neg H \vee \neg C \vee Fu$, y $(H \Rightarrow Fu) \vee (C \Rightarrow Fu) \equiv (\neg H \vee Fu) \vee (\neg C \vee Fu) \equiv \neg H \vee \neg C \vee Fu$: ambos lados son *idénticos*, por lo que el bicondicional es una tautología.)

Problema 2.2 — Verificación de Consecuencia Lógica

1. $A \Leftrightarrow B \models \neg A \vee B$?

<i>A</i>	<i>B</i>	$A \Leftrightarrow B$	$\neg A \vee B$
T	T	T	T
T	F	F	F
F	T	F	T
F	F	T	T

En las dos filas donde $A \Leftrightarrow B$ es verdadera (fila 1 y fila 4), $\neg A \vee B$ también lo es. **Correcta**. Justificación algebraica: $A \Leftrightarrow B \equiv (A \Rightarrow B) \wedge (B \Rightarrow A)$, y $A \Rightarrow B \equiv \neg A \vee B$; por lo tanto $A \Leftrightarrow B$ implica cada uno de sus conjuntos, en particular $\neg A \vee B$.

2. $(A \vee B) \wedge (\neg C \vee \neg D \vee E) \models (A \vee B)$?

Correcta, y de forma trivial: cualquier modelo que satisface una conjunción $X \wedge Y$ satisface, en particular, cada conjunto por separado (X e Y); aquí $X = (A \vee B)$. No hace falta enumerar las $2^5 = 32$ filas: es una instancia directa de la regla de *eliminación de la conjunción* ($X \wedge Y \models X$), válida para cualquier X, Y .

3. $(A \vee B) \wedge (\neg C \vee \neg D \vee E) \models (A \vee B) \wedge (\neg D \vee E)$?

No es correcta. Contraejemplo: $A = T, B = F, C = F, D = T, E = F$.

Verificación paso a paso:

- $A \vee B = T \vee F = T$.
- $\neg C \vee \neg D \vee E = \neg F \vee \neg T \vee F = T \vee F \vee F = T$.

- Lado izquierdo (premisa) = $T \wedge T = T$: el modelo satisface la premisa.
- $\neg D \vee E = \neg T \vee F = F \vee F = F$.
- Lado derecho (conclusión) = $(A \vee B) \wedge (\neg D \vee E) = T \wedge F = F$.

La premisa es verdadera y la conclusión es falsa en este modelo, así que la premisa **no** implica la conclusión. El error es que $\neg C \vee \neg D \vee E$ puede ser verdadera "gracias a" $\neg C$ sin decir nada sobre $\neg D \vee E$.

6.5.3. Bloque 3: Modelamiento de Lenguaje Natural — El Problema de la Mansión Embrujada

Parte A — Definición de Símbolos Proposicionales Además de $F_{x,y}$, $T_{x,y}$, $Frio_{x,y}$ y $Crujido_{x,y}$ (dados en el enunciado), se necesitan símbolos para registrar el estado epistémico del robot:

- $Segura_{x,y}$ = “El robot ha *probado* que la habitación (x, y) es segura (no tiene Fantasma ni Trampa)”.
- $Visitada_{x,y}$ = “El robot ya visitó la habitación (x, y) ”.

Parte B — Codificación de las Reglas

1. La entrada es segura:

$$\neg F_{1,1} \wedge \neg T_{1,1}$$

2. Frío en $(1, 1)$ implica Fantasma en una adyacente:

$$Frio_{1,1} \Leftrightarrow (F_{2,1} \vee F_{1,2})$$

3. Crujido en $(1, 1)$ implica Trampa en una adyacente:

$$Crujido_{1,1} \Leftrightarrow (T_{2,1} \vee T_{1,2})$$

4. Axioma de Frío para $(2, 2)$, adyacente a $(1, 2)$, $(3, 2)$ y $(2, 1)$:

$$Frio_{2,2} \Leftrightarrow (F_{1,2} \vee F_{3,2} \vee F_{2,1})$$

5. A lo sumo un Fantasma en toda la mansión (6 habitaciones: $(1, 1)$ a $(3, 2)$). Usando implicaciones, se afirma que si una habitación tiene el Fantasma, **ninguna otra** lo tiene:

$$\bigwedge_{(x,y) \neq (x',y')} (F_{x,y} \Rightarrow \neg F_{x',y'})$$

Por ejemplo, para $(1, 1)$:

$$F_{1,1} \Rightarrow (\neg F_{2,1} \wedge \neg F_{3,1} \wedge \neg F_{1,2} \wedge \neg F_{2,2} \wedge \neg F_{3,2})$$

y una implicación análoga se escribe partiendo de cada una de las otras 5 habitaciones (en total $\binom{6}{2} = 15$ pares de restricción, cada uno equivalente a la cláusula $\neg F_{x,y} \vee \neg F_{x',y'}$).

Parte C — Razonamiento a partir de las Percepciones Adyacencias en la cuadrícula 3×2 : $(1,1)-(2,1)-(3,1)$ (fila inferior) y $(1,2)-(2,2)-(3,2)$ (fila superior), con $(1,1)-(1,2)$, $(2,1)-(2,2)$ y $(3,1)-(3,2)$ uniendo las dos filas.

1. **¿Puede el robot concluir que $(2,2)$ contiene una Trampa?**

Razonamiento (paso a paso):

- a) En $(1,1)$ el robot percibe “Sin Crujido”. Por el axioma de la Parte B, ítem 3, $Crujido_{1,1} \Leftrightarrow (T_{2,1} \vee T_{1,2})$; como $Crujido_{1,1}$ es falso, se concluye $\neg T_{2,1} \wedge \neg T_{1,2}$ (ni $(2,1)$ ni $(1,2)$ tienen Trampa). Junto con $\neg F_{2,1} \wedge \neg F_{1,2}$ (del axioma de Frío) y la entrada segura, esto confirma que $(2,1)$ y $(1,2)$ son seguras — de ahí que el robot pudiera moverse a ambas.
- b) El axioma de Crujido para $(1,2)$ (adyacente a $(1,1)$ y $(2,2)$) es:
$$Crujido_{1,2} \Leftrightarrow (T_{1,1} \vee T_{2,2})$$
- c) El robot percibe Crujido en $(1,2)$, luego $Crujido_{1,2}$ es verdadero, así que $T_{1,1} \vee T_{2,2}$ es verdadero (Modus Ponens sobre el bicondicional).
- d) Se sabe que $\neg T_{1,1}$ (la entrada es segura, Parte B ítem 1).
- e) Por silogismo disyuntivo sobre $T_{1,1} \vee T_{2,2}$ y $\neg T_{1,1}$, se concluye $T_{2,2}$.

Conclusión: Sí, el robot puede concluir con certeza que hay una Trampa en $(2,2)$.

2. **¿Puede el robot concluir que $(3,1)$ es segura para entrar?**

Razonamiento (paso a paso):

- a) $(3,1)$ es adyacente a $(2,1)$ y $(3,2)$. El robot solo tiene percepciones en $(1,1)$, $(2,1)$ y $(1,2)$; nunca ha estado en $(3,2)$, así que no hay información directa sobre esa adyacencia.
- b) En $(2,1)$ el robot solo percibió Crujido (no Frío). Por el axioma de Frío para $(2,1)$: $Frio_{2,1} \Leftrightarrow (F_{1,1} \vee F_{3,1} \vee F_{2,2})$. Como $Frio_{2,1}$ es falso, se concluye $\neg F_{1,1} \wedge \neg F_{3,1} \wedge \neg F_{2,2}$: en particular $\neg F_{3,1}$ (no hay Fantasma en $(3,1)$).
- c) Por el axioma de Crujido para $(2,1)$: $Crujido_{2,1} \Leftrightarrow (T_{1,1} \vee T_{3,1} \vee T_{2,2})$. Como $Crujido_{2,1}$ es verdadero y $\neg T_{1,1}$ (entrada segura), se obtiene $T_{3,1} \vee T_{2,2}$.
- d) Del ítem anterior de este mismo bloque ya se sabe $T_{2,2}$ es verdadero. Esto por sí solo **ya satisface** la disyunción $T_{3,1} \vee T_{2,2}$, sin necesidad de que $T_{3,1}$ sea verdadero. Es decir, la KB *no fuerza* un valor de verdad para $T_{3,1}$: es consistente tanto con $T_{3,1} = Verdadero$ como con $T_{3,1} = Falso$.

Conclusión: No. Aunque se sabe que $(3,1)$ no tiene Fantasma ($\neg F_{3,1}$), el estado de la Trampa en $(3,1)$ queda **indeterminado** (el Crujido percibido en $(2,1)$ ya queda explicado por la Trampa en $(2,2)$). Como la seguridad exige certeza sobre *ambos* peligros, el robot no puede probar que $(3,1)$ sea segura y, según la regla del enunciado, no puede entrar todavía.

6.5.4. Bloque 4: Conversión a FNC y Resolución

Problema 4.1 — Conversión a FNC

1. $P \Leftrightarrow Q$

Paso a paso:

$$\begin{aligned} P \Leftrightarrow Q &\equiv (P \Rightarrow Q) \wedge (Q \Rightarrow P) && \text{(def. de } \Leftrightarrow \text{)} \\ &\equiv (\neg P \vee Q) \wedge (\neg Q \vee P) && \text{(eliminar } \Rightarrow \text{)} \end{aligned}$$

Ya es conjunción de disyunciones de literales: no hace falta distribuir más.

FNC: $(\neg P \vee Q) \wedge (\neg Q \vee P)$

2. $\neg(P \vee Q) \Rightarrow R$ **Paso a paso:**

$$\begin{aligned}
\neg(P \vee Q) \Rightarrow R &\equiv \neg\neg(P \vee Q) \vee R && \text{(eliminar } \Rightarrow \text{)} \\
&\equiv (P \vee Q) \vee R && \text{(doble negación)} \\
&\equiv P \vee Q \vee R && \text{(ya es una sola cláusula)}
\end{aligned}$$

FNC: $P \vee Q \vee R$ 3. $(A \Rightarrow B) \wedge (B \Rightarrow C) \Rightarrow (A \Rightarrow C)$ **Paso a paso:**

$$\begin{aligned}
&\equiv \neg[(\neg A \vee B) \wedge (\neg B \vee C)] \vee (\neg A \vee C) && \text{(eliminar las tres } \Rightarrow \text{)} \\
&\equiv [\neg(\neg A \vee B) \vee \neg(\neg B \vee C)] \vee (\neg A \vee C) && \text{(De Morgan)} \\
&\equiv (A \wedge \neg B) \vee (B \wedge \neg C) \vee \neg A \vee C && \text{(De Morgan otra vez, doble negación)}
\end{aligned}$$

Distribuir $\vee$ sobre $\wedge$, primero $(A \wedge \neg B) \vee (B \wedge \neg C)$:

$$(A \vee B) \wedge (A \vee \neg C) \wedge (\neg B \vee B) \wedge (\neg B \vee \neg C)$$

Como $\neg B \vee B$ es tautología, se elimina esa cláusula (no aporta restricción):

$$(A \vee B) \wedge (A \vee \neg C) \wedge (\neg B \vee \neg C)$$

Ahora se distribuye ese resultado con $\vee (\neg A \vee C)$, cláusula por cláusula:

$$(A \vee B \vee \neg A \vee C) \wedge (A \vee \neg C \vee \neg A \vee C) \wedge (\neg B \vee \neg C \vee \neg A \vee C)$$

Cada una de las tres cláusulas contiene un literal y su negación ($A, \neg A$ en la primera y segunda; $C, \neg C$ en la segunda y tercera), por lo que **las tres son tautológicas** y se pueden eliminar todas.

Respuesta: La FNC resultante no tiene cláusulas (todas son tautologías), lo cual significa que la sentencia original es verdadera en *todo* modelo: es una **validez** (el silogismo hipotético / transitividad de la implicación, coherente con la Parte C, ítem 5, del Bloque 1).

FNC: $\top$ (conjunción vacía de cláusulas, todas tautológicas)**Problema 4.2 — Validez, FNC y Prueba por Resolución (Ej. 13, Cap. 7 AIMA)**

Sentencia: $[(Comida \Rightarrow Fiesta) \vee (Bebidas \Rightarrow Fiesta)] \Rightarrow [(Comida \wedge Bebidas) \Rightarrow Fiesta]$. Se abrevia $Co = Comida$, $Be = Bebidas$, $Fi = Fiesta$.

1. **Clasificación por enumeración:**

Co	Be	Fi	$Co \Rightarrow Fi$	$Be \Rightarrow Fi$	LHS(or)	$Co \wedge Be$	RHS	Total
T	T	T	T	T	T	T	T	T
T	T	F	F	F	F	T	F	T
T	F	T	T	T	T	F	T	T
T	F	F	F	T	T	F	T	T
F	T	T	T	T	T	F	T	T
F	T	F	T	F	T	F	T	T
F	F	T	T	T	T	F	T	T
F	F	F	T	T	T	F	T	T

Clasificación: Válida.

Justificación: Las 8 filas dan verdadero. Es interesante notar que la única fila en la que el consecuente (RHS) es falso es $Co=T, Be=T, Fi=F$; pero en esa misma fila el antecedente (LHS) también es falso, así que la implicación completa es $F \Rightarrow F = T$.

2. FNC de cada lado:

FNC lado izquierdo:

$$\begin{aligned}(Co \Rightarrow Fi) \vee (Be \Rightarrow Fi) &\equiv (\neg Co \vee Fi) \vee (\neg Be \vee Fi) && \text{(eliminar } \Rightarrow \text{)} \\ &\equiv \neg Co \vee \neg Be \vee Fi && \text{(ya es una sola cláusula; } Fi \text{ se repite)}\end{aligned}$$

FNC lado izquierdo: $\neg Co \vee \neg Be \vee Fi$

FNC lado derecho:

$$\begin{aligned}(Co \wedge Be) \Rightarrow Fi &\equiv \neg(Co \wedge Be) \vee Fi && \text{(eliminar } \Rightarrow \text{)} \\ &\equiv \neg Co \vee \neg Be \vee Fi && \text{(De Morgan)}\end{aligned}$$

FNC lado derecho: $\neg Co \vee \neg Be \vee Fi$

Confirmación: Ambos lados se reducen exactamente a la *misma* cláusula $\neg Co \vee \neg Be \vee Fi$. Como la sentencia completa tiene la forma $X \Rightarrow X$ (con X idéntico a ambos lados), es una tautología por construcción — esto confirma directamente el resultado de validez obtenido por enumeración en (1), y de forma más eficiente.

3. Prueba por resolución. Para probar que la sentencia es válida (es decir, $\models LHS \Rightarrow RHS$), se niega la conclusión y se busca una contradicción entre LHS y $\neg RHS$.Cláusulas FNC de $(KB \wedge \neg \text{conclusión})$:

- $C_1: \neg Co \vee \neg Be \vee Fi$ (de LHS)
- $\neg RHS = \neg(\neg Co \vee \neg Be \vee Fi) \equiv Co \wedge Be \wedge \neg Fi$ (De Morgan), lo que produce tres cláusulas unitarias:
- $C_2: Co$
- $C_3: Be$
- $C_4: \neg Fi$

Paso de resolución 1: Resolver C_1 ($\neg Co \vee \neg Be \vee Fi$) con C_2 (Co) sobre el par complementario $Co/\neg Co \Rightarrow$ se deriva $C_5: \neg Be \vee Fi$.

Paso de resolución 2: Resolver C_5 ($\neg Be \vee Fi$) con C_3 (Be) sobre el par $Be/\neg Be \Rightarrow$ se deriva $C_6: Fi$.

Paso de resolución 3: Resolver C_6 (Fi) con C_4 ($\neg Fi$) sobre el par $Fi/\neg Fi \Rightarrow$ se deriva la cláusula vacía $\perp$.

Conclusión: Como se derivó $\perp$, el conjunto $\{LHS, \neg RHS\}$ es insatisfacible, por lo tanto $LHS \models RHS$, es decir, la sentencia original es **válida**. Esto confirma, por tercera vía, el resultado de (1) y (2).

6.5.5. Bloque 5: Cláusulas de Horn, Cláusulas Definidas y Encadenamiento

Parte A — Clasificación de Cláusulas Sección A.1

#	Cláusula	Clasificación (justificación)
1	$A \vee \neg B \vee \neg C$	Definida, Horn (1 literal positivo: A)
2	$\neg A \vee \neg B$	Horn, Objetivo (0 literales positivos)
3	A	Definida, Horn, Unitaria (1 literal, positivo)
4	$A \vee B \vee \neg C$	No-Horn (2 literales positivos: A, B)
5	$\neg A \vee \neg B \vee \neg C$	Horn, Objetivo (0 literales positivos)
6	$B \wedge C \Rightarrow A \equiv \neg B \vee \neg C \vee A$	Definida, Horn (1 positivo: A)
7	$Verdadero \Rightarrow A \equiv A$	Definida, Horn, Unitaria (equivalente a la cláusula 3)

Sección A.2 — Clasificación desde lenguaje natural (Ej. 17, Cap. 7)

Enunciado: “Una persona que es radical (R) es elegible (E) si es conservadora (C), pero de lo contrario no es elegible.” Esto se lee como: *si* R , entonces (E sii C); la oración no afirma nada sobre las personas no radicales.

1. Análisis de las tres opciones:

- **Opción 1** ($(R \wedge E) \Leftrightarrow C$): **Incorrecta.** Fuerza C a ser falso cada vez que R es falso (porque si $R=F$, entonces $R \wedge E = F$, y el bicondicional exige $C = F$), es decir, afirma que ninguna persona no-radical puede ser conservadora — algo que el enunciado original nunca dice.
- **Opción 2** ($R \Rightarrow (E \Leftrightarrow C)$): **Correcta.** Traduce exactamente “si radical, entonces elegible sii conservador”, y es vacuamente verdadera (no afirma nada) cuando R es falso, tal como exige el enunciado.
- **Opción 3** ($R \Rightarrow ((C \Rightarrow E) \vee \neg E)$): **Incorrecta.** Simplificando el consecuente: $(C \Rightarrow E) \vee \neg E \equiv (\neg C \vee E) \vee \neg E \equiv \neg C \vee (E \vee \neg E) \equiv \neg C \vee \top \equiv \top$. El consecuente es una tautología, así que toda la fórmula se reduce a $R \Rightarrow \top \equiv \top$: es siempre verdadera y no expresa ninguna restricción real.

2. Conversión a forma clausular de la opción correcta (Opción 2):

$$\begin{aligned}
 R \Rightarrow (E \Leftrightarrow C) &\equiv \neg R \vee (E \Leftrightarrow C) \\
 &\equiv \neg R \vee [(\neg E \vee C) \wedge (\neg C \vee E)] && \text{(bicondicional)} \\
 &\equiv (\neg R \vee \neg E \vee C) \wedge (\neg R \vee \neg C \vee E) && \text{(distribuir } \vee \text{ sobre } \wedge)
 \end{aligned}$$

Respuesta: Dos cláusulas: $(\neg R \vee \neg E \vee C)$ y $(\neg R \vee \neg C \vee E)$.

Respuesta (Horn): Sí, ambas son cláusulas de Horn. La primera tiene exactamente un literal positivo (C), la segunda tiene exactamente un literal positivo (E); ambas son de hecho **definidas** ($R \wedge E \Rightarrow C$ y $R \wedge C \Rightarrow E$ respectivamente).

3. ¿Podría un algoritmo de encadenamiento resolver esta KB?

Respuesta: Sí. Como la representación correcta (Opción 2) se descompone en dos cláusulas definidas (de Horn), tanto el encadenamiento hacia adelante como el encadenamiento hacia atrás son correctos y completos para procesarla, siempre que el resto de la KB (hechos sobre R , C , E para individuos concretos) también esté en forma de Horn.

Parte B — Encadenamiento Hacia Adelante KB: $H_1 : A$, $H_2 : B$, $R_1 : A \wedge B \Rightarrow C$, $R_2 : C \Rightarrow D$, $R_3 : B \wedge D \Rightarrow E$, $R_4 : E \Rightarrow F$, $R_5 : A \wedge F \Rightarrow G$.

1. Traza:

Paso	Regla Aplicada	Nuevo Hecho Derivado
1	H_1 (hecho inicial)	A
2	H_2 (hecho inicial)	B
3	$R_1: A \wedge B \Rightarrow C$ (A, B ya conocidos)	C
4	$R_2: C \Rightarrow D$ (C ya conocido)	D
5	$R_3: B \wedge D \Rightarrow E$ (B, D ya conocidos)	E
6	$R_4: E \Rightarrow F$ (E ya conocido)	F
7	$R_5: A \wedge F \Rightarrow G$ (A, F ya conocidos)	G

2. **¿Es G consecuencia lógica? Sí.** El encadenamiento hacia adelante es correcto (sound) y completo para KB de Horn; como derivó G en el paso 7 mediante una cadena de reglas cuyas premisas estaban satisfechas en cada momento, se concluye $KB \models G$.
3. **Literales que son consecuencia lógica de la KB:** A, B, C, D, E, F, G — todos los hechos derivados por la traza, ya que el algoritmo terminó sin poder agregar nada más y ningún otro símbolo aparece en la KB.

Parte C — Encadenamiento Hacia Atrás

1. Árbol de prueba Y-O para la consulta G :

```

G                                     (meta)
'-- R5: A y F => G                   (nodo-Y: se requieren ambas ramas)
  |-- A                             (hecho, hoja OK)
  '-- F
    '-- R4: E => F
      '-- E
        '-- R3: B y D => E           (nodo-Y: se requieren ambas ramas)
          |-- B                     (hecho, hoja OK)
          '-- D
            '-- R2: C => D
              '-- C
                '-- R1: A y B => C   (nodo-Y: se requieren ambas ramas)
                  |-- A             (hecho, hoja OK, ya probado)
                  '-- B             (hecho, hoja OK, ya probado)

```

Cada nodo etiquetado con una regla es un nodo “Y” (AND): todas sus ramas hijas deben probarse. La recursión termina exitosamente cuando cada hoja es un hecho de la KB (A o B).

2. Comparación:

Pasos hacia adelante: 7 hechos derivados (incluye los 2 hechos iniciales), en un único recorrido que expande *toda* la KB sin importar la consulta.

Pasos hacia atrás: Se exploran únicamente las submetas necesarias para G : A, F, E, B, D, C (mismos 7 nodos en este caso particular, porque esta KB es una cadena lineal donde cada hecho es relevante para G), pero solo se expanden las reglas cuya *cabeza* coincide con la meta actual, nunca reglas irrelevantes.

Preferencia: En esta KB lineal ambos métodos exploran esencialmente el mismo número de nodos, así que no hay ventaja clara. En general, se prefiere **hacia adelante** cuando se quieren derivar *todas* las consecuencias de la KB (o la KB es pequeña y densa en reglas relevantes), y **hacia atrás** cuando solo interesa responder una consulta puntual en una KB grande con muchos hechos/reglas irrelevantes para esa consulta (evita trabajo innecesario).

Parte D — Completitud y Limitaciones

1. ¿Puede el encadenamiento hacia adelante manejar $(A \vee B \Rightarrow C)$?

Respuesta: No directamente en esa forma sintáctica, porque el algoritmo espera premisas unidas por $\wedge$ (todas deben cumplirse a la vez), no por $\vee$. Sin embargo, al convertir la cláusula a FNC se obtiene $(A \vee B) \Rightarrow C \equiv \neg(A \vee B) \vee C \equiv (\neg A \wedge \neg B) \vee C \equiv (\neg A \vee C) \wedge (\neg B \vee C)$, es decir, dos cláusulas de Horn separadas y equivalentes: $A \Rightarrow C$ y $B \Rightarrow C$. Una vez descompuesta así, el encadenamiento hacia adelante sí puede procesarla (disparando C si se conoce A o si se conoce B , por separado).

2. KB con solo cláusulas no-Horn — ¿qué algoritmos sirven?

Respuesta: Al menos dos: (1) **TT-Implica?** (enumeración de tablas de verdad), correcto y completo para cualquier KB proposicional aunque exponencial; (2) **Refutación por resolución**, correcta y completa para cualquier conjunto de cláusulas en FNC, sin requerir estructura de Horn. (También podría usarse DPLL para decidir la satisfacibilidad de $KB \wedge \neg\alpha$.)

3. Complejidad temporal: encadenamiento hacia adelante (Horn) vs. TT-Implica? (general).

Respuesta: El encadenamiento hacia adelante sobre una KB de Horn es **lineal** en el tamaño de la KB, $\mathcal{O}(n)$: cada cláusula se examina y dispara a lo sumo una vez, llevando un contador de premisas pendientes por cláusula, sin necesidad de retroceder ni de "adivinar" valores de verdad. TT-Implica? sobre una KB proposicional general es **exponencial**, $\mathcal{O}(2^n)$ en el número de símbolos, porque enumera todas las asignaciones de verdad posibles sin asumir ninguna estructura.

Respuesta (por qué): La diferencia se debe a que las cláusulas de Horn (a lo sumo un literal positivo) tienen una estructura dirigida (premisas $\Rightarrow$ conclusión) que permite propagar hechos ya establecidos de forma monótona y determinista, sin bifurcaciones; una KB general (con cláusulas no-Horn) no tiene esa estructura, y en el peor caso hay que probar combinaciones de valores de verdad para decidir la consecuencia lógica.

6.6. Ejercicios: Guía de Lógica de Primer Orden (con solución paso a paso)

Esta sección resuelve, de principio a fin, la *Guía de Ejercicios: Lógica de Primer Orden*.

6.6.1. Bloque 1: Conceptos Fundamentales de LPO

1. “Una sentencia válida en LPO es verdadera en toda interpretación posible, independientemente de la asignación a variables libres.”

Verdadero. Por definición, una **sentencia** (a diferencia de una fórmula abierta) no tiene variables libres, así que la cláusula “independientemente de la asignación a variables libres” se cumple trivialmente (no hay ninguna que asignar); lo que exige la validez es que sea verdadera en *todo* modelo/interpretación posible.

2. “Si una fórmula Φ es satisfacible, entonces su negación ($\neg\Phi$) debe ser insatisfacible.”

Falso. Satisfacibilidad de Φ solo significa que Φ es verdadera en *al menos un* modelo; nada impide que también exista otro modelo donde Φ (y por lo tanto $\neg\Phi$) sea verdadera.

Contraejemplo: $\Phi = P(x)$. Es satisfacible (verdadera en una interpretación donde P es siempre verdadero). Pero $\neg\Phi = \neg P(x)$ también es satisfacible (verdadera en una interpretación donde P es siempre falso): no es insatisfacible.

Corrección: Solo si Φ es **válida** se garantiza que $\neg\Phi$ es insatisfacible.

3. “La sentencia $\forall x P(x) \rightarrow \exists x P(x)$ es válida en LPO, siempre que el dominio sea no vacío.”

Verdadero. Si el dominio no es vacío, existe al menos un objeto d ; si $\forall x P(x)$ es verdadera, en particular $P(d)$ es verdadera, lo cual basta para que $\exists x P(x)$ sea verdadera. (Precisamente por esto LPO exige dominios no vacíos: con dominio vacío, $\forall x P(x)$ sería vacuamente verdadera mientras que $\exists x P(x)$ sería falsa, y la implicación fallaría.)

4. “Las fórmulas $\forall x \exists y R(x, y)$ y $\exists y \forall x R(x, y)$ son lógicamente equivalentes.”

Falso. Solo se cumple una dirección: $\exists y \forall x R(x, y) \models \forall x \exists y R(x, y)$ (si hay un y que sirve para todo x , ese mismo y sirve como testigo para cada x por separado), pero no la recíproca.

Contraejemplo: Dominio = enteros, $R(x, y) \equiv “x < y”$. $\forall x \exists y (x < y)$ es verdadera (para todo entero siempre hay uno mayor). Pero $\exists y \forall x (x < y)$ es falsa (ningún entero es mayor que *todos* los enteros).

5. “La fórmula $\forall x(\Phi \vee \Psi)$ es lógicamente equivalente a $(\forall x \Phi \vee \forall x \Psi)$ para cualesquiera Φ y Ψ .”

Falso. Solo se cumple una dirección: $(\forall x \Phi \vee \forall x \Psi) \models \forall x(\Phi \vee \Psi)$, pero no la recíproca (a diferencia de $\wedge$, que sí distribuye en ambas direcciones sobre $\forall$).

Contraejemplo: Dominio = $\{1, 2\}$, $\Phi(x) = “x \text{ es impar}”$ (verdadero solo para $x=1$), $\Psi(x) = “x \text{ es par}”$ (verdadero solo para $x=2$). $\forall x(\Phi \vee \Psi)$ es verdadera (todo elemento es par o impar), pero $\forall x \Phi$ es falsa (falla en $x=2$) y $\forall x \Psi$ es falsa (falla en $x=1$), así que $\forall x \Phi \vee \forall x \Psi$ es falsa.

6.6.2. Bloque 2: Traducción — Español a Lógica de Primer Orden

Vocabulario dado: $Ocupacion(p, o)$, $Cliente(p_1, p_2)$ (“ p_1 es cliente de p_2 ”), $Jefe(p_1, p_2)$, constantes de ocupación *Medico*, *Cirujano*, *Abogado*, *Actor*, *Mecanico*, constantes *Emily*, *Joe*, *Enfermero(p)*, $LeAgradada(p_1, p_2)$ (“a p_1 le agrada p_2 ”).

1. “Emily es cirujana o abogada.”

$$Ocupacion(Emily, Cirujano) \vee Ocupacion(Emily, Abogado)$$

2. “Joe es actor, pero también ejerce otra ocupación.”

$$Ocupacion(Joe, Actor) \wedge \exists o (Ocupacion(Joe, o) \wedge o \neq Actor)$$

Justificación: “otra” ocupación requiere afirmar la *existencia* de una ocupación distinta de *Actor*; por eso el cuantificador existencial se combina con $\wedge$ (existe un objeto que cumple ser ocupación de Joe **y** ser distinto de Actor), y se necesita explícitamente la desigualdad $o \neq Actor$, si no la misma ocupación *Actor* satisfaría trivialmente el existencial.

3. “Todos los cirujanos son médicos.”

$$\forall p (Ocupacion(p, Cirujano) \Rightarrow Ocupacion(p, Medico))$$

Justificación: Enunciado universal sobre una categoría $\Rightarrow$ se usa $\forall$ con $\Rightarrow$, tal como sugiere el consejo de la guía.

4. “Existe un abogado todos cuyos clientes son médicos.”

$$\exists p_1 (Ocupacion(p_1, Abogado) \wedge \forall p_2 (Cliente(p_2, p_1) \Rightarrow Ocupacion(p_2, Medico)))$$

Justificación: El cuantificador principal es existencial (“existe un abogado”), así que se combina con $\wedge$ para afirmar sus dos propiedades (ser abogado, y la condición sobre sus clientes). Dentro, “todos sus clientes” es un $\forall$ interno con $\Rightarrow$. Nótese el orden de argumentos: $Cliente(p_2, p_1)$ significa “ p_2 es cliente de p_1 ”, que es lo que se necesita (clientes *del abogado* p_1).

5. “Algún mecánico les agrada a todos los enfermeros.”

$$\exists p_1 (Ocupacion(p_1, Mecanico) \wedge \forall p_2 (Enfermero(p_2) \Rightarrow LeAgradada(p_2, p_1)))$$

Justificación: De nuevo $\exists$ con $\wedge$ para el mecánico, y $\forall$ con $\Rightarrow$ para “todos los enfermeros”. Cuidado con el orden de *LeAgradada*: como $LeAgradada(p_1, p_2)$ significa “a p_1 le agrada p_2 ”, y se quiere decir que *a los enfermeros les agrada el mecánico*, se escribe $LeAgradada(p_2, p_1)$ (enfermero, mecánico), no al revés.

6.6.3. Bloque 3: LPO a Español y Análisis de Validez

Parte A — Traducción al español

1. $\forall x, y, l \text{ HablaIdioma}(x, l) \wedge \text{HablaIdioma}(y, l) \Rightarrow \text{Entiende}(x, y) \wedge \text{Entiende}(y, x)$
Español: “Si dos personas hablan el mismo idioma, entonces se entienden mutuamente (cada una entiende a la otra).”
2. $\exists y \forall x \text{ Entiende}(x, y)$
Español: “Existe una persona a quien todo el mundo entiende.”
3. $\forall x \exists y (\text{Amigos}(x, y) \wedge \forall z (\text{Amigos}(x, z) \rightarrow \text{Entiende}(z, y)))$
Español: “Toda persona tiene un amigo tal que *todos* los amigos de esa persona entienden a ese amigo en particular.”

Parte B — Validez, satisfacibilidad, insatisfacibilidad

4. $(\exists x x=x) \Rightarrow (\forall y \exists z z \neq y)$
Clasificación: Satisfacible, pero no válida (contingente).
Justificación: El antecedente $\exists x x=x$ es verdadero en *todo* modelo (por reflexividad de la igualdad, y porque los dominios en LPO siempre son no vacíos), así que la verdad de la implicación depende enteramente del consecuente $\forall y \exists z z \neq y$ (“todo elemento tiene otro distinto de él”), que es verdadero en cualquier dominio con **2 o más** elementos, pero **falso** en un dominio de un solo elemento $D = \{a\}$ (para $y = a$ no existe ningún $z \in D$ con $z \neq a$).
 Con $|D| \geq 2$: antecedente T , consecuente $T \Rightarrow$ implicación T (satisfacible).
 Con $|D| = 1$: antecedente T , consecuente $F \Rightarrow$ implicación F (no válida).
5. $\forall x (P(x) \vee \neg P(x))$
Clasificación: Válida.
Justificación: Para cualquier interpretación de P y cualquier objeto del dominio, $P(x) \vee \neg P(x)$ es una instancia del principio del tercero excluido, verdadera sin importar el valor de verdad de $P(x)$. Al ser verdadera para todo x en todo modelo, la sentencia universal también lo es.
6. $\forall x (P(x) \wedge Q(x)) \leftrightarrow (\forall x P(x) \wedge \forall x Q(x))$
Clasificación: Válida.
Justificación: A diferencia de $\forall/\exists$, el cuantificador universal **sí distribuye completamente** sobre $\wedge$ en ambas direcciones: “todo objeto cumple P y Q ” equivale exactamente a “todo objeto cumple P ” y “todo objeto cumple Q ”, para cualquier interpretación de P, Q y cualquier dominio — no existe contraejemplo posible.
7. $\forall x (P(x) \vee Q(x)) \leftrightarrow (\forall x P(x) \vee \forall x Q(x))$
Clasificación: Satisfacible, pero no válida (contingente).
Justificación: Solo la dirección $(\forall x P(x) \vee \forall x Q(x)) \models \forall x (P(x) \vee Q(x))$ es válida en general; la recíproca falla.
Contraejemplo (para mostrar que no es válida): Dominio $\{1, 2\}$, $P(x) = “x \text{ impar}”$, $Q(x) = “x \text{ par}”$. $\forall x (P \vee Q)$ es verdadero, pero $\forall x P$ y $\forall x Q$ son ambos falsos, así que el lado derecho es falso: el bicondicional es falso en este modelo (no es válida).
Modelo donde es verdadera (para mostrar que es satisfacible, no insatisfacible): Cualquier interpretación donde $P(x)$ sea verdadero para todo x (p. ej. $P(x) \equiv (x=x)$): ambos lados del bicondicional son verdaderos, así que el bicondicional completo es verdadero.

6.6.4. Bloque 4: Construcción de Base de Conocimiento — Dominio de Parentesco

Predicados base: $Padre(p, q)$ (“ p es progenitor de q ”), $Hombre(p)$, $Mujer(p)$, $Casado(p, q)$.

Parte A — Axiomas

1. **Madre**(m, h): m es la madre de h .

$$Madre(m, h) \Leftrightarrow Padre(m, h) \wedge Mujer(m)$$

2. **PadreBio**(f, h): f es el padre biológico de h .

$$PadreBio(f, h) \Leftrightarrow Padre(f, h) \wedge Hombre(f)$$

3. **Hermano**(x, y): x e y son hermanos (comparten al menos un progenitor, $x \neq y$).

$$Hermano(x, y) \Leftrightarrow x \neq y \wedge \exists p (Padre(p, x) \wedge Padre(p, y))$$

4. **Abuelo**(g, h): g es abuelo o abuela de h .

$$Abuelo(g, h) \Leftrightarrow \exists p (Padre(g, p) \wedge Padre(p, h))$$

5. **Abuela**(g, h): g es abuela de h .

$$Abuela(g, h) \Leftrightarrow Abuelo(g, h) \wedge Mujer(g)$$

Parte B — Hechos del árbol genealógico

$Casado(Jorge, Mama)$

$Padre(Jorge, Isabel)$, $Padre(Jorge, Margarita)$, $Padre(Mama, Isabel)$, $Padre(Mama, Margarita)$
 $Padre(Isabel, Carlos)$, $Padre(Isabel, Ana)$, $Padre(Isabel, Andres)$, $Padre(Isabel, Eduardo)$
 $Padre(Felipe, Carlos)$, $Padre(Felipe, Ana)$, $Padre(Felipe, Andres)$, $Padre(Felipe, Eduardo)$
 $Padre(Carlos, Guillermo)$, $Padre(Carlos, Enrique)$, $Padre(Diana, Guillermo)$, $Padre(Diana, Enrique)$
 $Padre(Ana, Pedro)$, $Padre(Ana, Zara)$, $Padre(Marcos, Pedro)$, $Padre(Marcos, Zara)$

$Hombre(Jorge)$, $Hombre(Felipe)$, $Hombre(Carlos)$, $Hombre(Marcos)$, $Hombre(Andres)$,
 $Hombre(Eduardo)$, $Hombre(Guillermo)$, $Hombre(Enrique)$, $Hombre(Pedro)$

$Mujer(Mama)$, $Mujer(Isabel)$, $Mujer(Margarita)$, $Mujer(Diana)$, $Mujer(Ana)$, $Mujer(Zara)$

Nota: Solo se codifica $Casado(Jorge, Mama)$ porque es el único matrimonio que el enunciado afirma explícitamente; no se asumen matrimonios no declarados (p. ej. Isabel–Felipe).

Parte C — Consultas

1. ¿Quiénes son los nietos de Isabel?

Se instancia el axioma $Abuelo(g, h) \Leftrightarrow \exists p (Padre(g, p) \wedge Padre(p, h))$ con $g = Isabel$. Los hijos de Isabel son $\{Carlos, Ana, Andres, Eduardo\}$ (progenitor directo); se revisa cuáles de ellos tienen hijos propios en la KB:

- $Padre(Isabel, Carlos) \wedge Padre(Carlos, Guillermo) \Rightarrow Abuelo(Isabel, Guillermo)$
- $Padre(Isabel, Carlos) \wedge Padre(Carlos, Enrique) \Rightarrow Abuelo(Isabel, Enrique)$

- $\text{Padre}(\text{Isabel}, \text{Ana}) \wedge \text{Padre}(\text{Ana}, \text{Pedro}) \Rightarrow \text{Abuelo}(\text{Isabel}, \text{Pedro})$
- $\text{Padre}(\text{Isabel}, \text{Ana}) \wedge \text{Padre}(\text{Ana}, \text{Zara}) \Rightarrow \text{Abuelo}(\text{Isabel}, \text{Zara})$
- *Andres y Eduardo* no tienen hijos registrados en la KB, así que no aportan nietos.

Respuesta: Los nietos de Isabel son **Guillermo, Enrique, Pedro y Zara**.

2. ¿Son Guillermo y Zara hermanos?

Se evalúa $\text{Hermano}(\text{Guillermo}, \text{Zara}) \Leftrightarrow \text{Guillermo} \neq \text{Zara} \wedge \exists p(\text{Padre}(p, \text{Guillermo}) \wedge \text{Padre}(p, \text{Zara}))$.

Progenitores de Guillermo: $\{\text{Carlos}, \text{Diana}\}$. Progenitores de Zara: $\{\text{Ana}, \text{Marcos}\}$. La intersección $\{\text{Carlos}, \text{Diana}\} \cap \{\text{Ana}, \text{Marcos}\} = \emptyset$: no existe ningún p que sea progenitor de ambos.

Respuesta: No son hermanos; el existencial del axioma de Hermano queda insatisfecho, así que $\text{Hermano}(\text{Guillermo}, \text{Zara})$ es falso. (De hecho son **primos**: sus respectivos padres, Carlos y Ana, sí son hermanos entre sí, pues ambos son hijos de Isabel y Felipe.)

6.6.5. Bloque 5: Unificación y Demostración por Resolución

Parte A — Unificación

1. $P(A, B, B)$ y $P(x, y, z)$.
Posición a posición: $x/A, y/B, z/B$.
UMG: $\theta = \{x/A, y/B, z/B\}$.
2. $Q(y, G(A, B))$ y $Q(G(x, x), y)$.
Posición 1: y vs. $G(x, x) \Rightarrow \theta_1 = \{y/G(x, x)\}$ (verificación de ocurrencia OK: y no aparece dentro de $G(x, x)$).
Posición 2 (aplicando θ_1 a ambos términos): $G(A, B)$ vs. $G(x, x) \Rightarrow$ se debe unificar A con x (da x/A) y, en la segunda posición de G , B con x (ya sustituido por A), es decir B vs. A . Como A y B son constantes *distintas*, esta comparación falla.
Respuesta: No se pueden unificar. El intento exige simultáneamente x/A y x/B , lo cual es imposible porque $A \neq B$ (una variable no puede ligarse a dos constantes distintas a la vez).
3. $\text{LeAgrada}(x, y)$ y $\text{LeAgrada}(\text{Joe}, z)$.
Posición 1: x vs. $\text{Joe} \Rightarrow x/\text{Joe}$.
Posición 2: y vs. z (ambas variables) $\Rightarrow y/z$.
UMG: $\theta = \{x/\text{Joe}, y/z\}$ (se deja y ligada a la variable z , no a una constante, para mantener el unificador lo más general posible).
4. $\text{Progenitor}(\text{Padre}(x), x)$ y $\text{Progenitor}(\text{Padre}(\text{Juan}), \text{Juan})$.
Posición 1: $\text{Padre}(x)$ vs. $\text{Padre}(\text{Juan}) \Rightarrow$ se unifican los argumentos internos: x vs. $\text{Juan} \Rightarrow x/\text{Juan}$.
Posición 2 (aplicando x/Juan): x vs. $\text{Juan} \Rightarrow \text{Juan}$ vs. Juan : coincide, sin generar nueva ligadura.
UMG: $\theta = \{x/\text{Juan}\}$.

Parte B — Conversión a FNC

5. $\forall x (\text{Caballo}(x) \Rightarrow \text{Animal}(x))$

Paso a paso:

$$\forall x (\text{Caballo}(x) \Rightarrow \text{Animal}(x)) \equiv \forall x (\neg \text{Caballo}(x) \vee \text{Animal}(x)) \quad (\text{eliminar } \Rightarrow)$$

No hay $\exists$ que skolemizar. Se elimina el cuantificador universal (queda implícito sobre todas las variables libres de la cláusula) y ya es una disyunción de literales.

FNC: $\neg Caballo(x) \vee Animal(x)$

6. $\forall h ((\exists y (CabezaDe(h, y) \wedge Caballo(y))) \Rightarrow (\exists z (CabezaDe(h, z) \wedge Animal(z))))$

Paso 1 (eliminar $\Rightarrow$):

$$\forall h \left(\neg (\exists y (CabezaDe(h, y) \wedge Caballo(y))) \vee \exists z (CabezaDe(h, z) \wedge Animal(z)) \right)$$

Paso 2 (mover $\neg$ hacia adentro; $\neg \exists y \Phi \equiv \forall y \neg \Phi$, De Morgan):

$$\forall h \left(\forall y (\neg CabezaDe(h, y) \vee \neg Caballo(y)) \vee \exists z (CabezaDe(h, z) \wedge Animal(z)) \right)$$

Paso 3 (estandarizar variables): h, y, z ya son distintas, no hace falta renombrar.

Paso 4 (Skolemización): El $\exists z$ está dentro del alcance de $\forall h$ únicamente (no dentro de $\forall y$, que solo gobierna el primer disyunto), así que se reemplaza z por una **función de Skolem de h** : $z \rightarrow S(h)$.

$$\forall h \forall y \left((\neg CabezaDe(h, y) \vee \neg Caballo(y)) \vee (CabezaDe(h, S(h)) \wedge Animal(S(h))) \right)$$

Paso 5 (eliminar cuantificadores universales, quedan implícitos):

$$(\neg CabezaDe(h, y) \vee \neg Caballo(y)) \vee (CabezaDe(h, S(h)) \wedge Animal(S(h)))$$

Paso 6 (distribuir $\vee$ sobre $\wedge$): con $A_1 = \neg CabezaDe(h, y)$, $A_2 = \neg Caballo(y)$, $B_1 = CabezaDe(h, S(h))$, $B_2 = Animal(S(h))$:

$$(A_1 \vee A_2 \vee B_1) \wedge (A_1 \vee A_2 \vee B_2)$$

FNC (dos cláusulas):

$$\neg CabezaDe(h, y) \vee \neg Caballo(y) \vee CabezaDe(h, S(h)) \quad (\text{C-A})$$

$$\neg CabezaDe(h, y) \vee \neg Caballo(y) \vee Animal(S(h)) \quad (\text{C-B})$$

Parte C — Refutación por Resolución **Objetivo:** demostrar que la premisa 5 (“todo caballo es un animal”) implica la afirmación 6 (“la cabeza de un caballo es la cabeza de un animal”), por refutación: se asume la **negación** de la conclusión (sentencia 6), se convierte a FNC, y se resuelve junto con la premisa (sentencia 5) hasta obtener $\perp$.

Paso 1 — Cláusula de la premisa (de la Parte B, ítem 5):

$$C_1 : \neg Caballo(x) \vee Animal(x)$$

Paso 2 — Negar la conclusión (sentencia 6) y convertirla a FNC:

$$\begin{aligned} \neg \left[\forall h (\exists y (CabezaDe(h, y) \wedge Caballo(y)) \Rightarrow \exists z (CabezaDe(h, z) \wedge Animal(z))) \right] &\equiv \exists h \left[\exists y (CabezaDe(h, y) \wedge \neg \exists z (CabezaDe(h, z) \wedge Animal(z))) \right] \\ &\equiv \exists h \exists y \left[CabezaDe(h, y) \wedge \neg \exists z (CabezaDe(h, z) \wedge Animal(z)) \right] \\ &\equiv \exists h \exists y \forall z (\neg CabezaDe(h, z) \vee \neg Animal(z)) \end{aligned}$$

Los cuantificadores $\exists h$ y $\exists y$ son los **más externos** (no están dentro de ningún $\forall$), así que se Skolemizan con **constantes** nuevas H (una cabeza concreta) y Y (un caballo concreto): $h \rightarrow H$, $y \rightarrow Y$. El $\forall z$ se elimina (queda implícito) y la conjunción se separa en cláusulas:

$$\begin{aligned} C_2 : & \text{CabezaDe}(H, Y) \\ C_3 : & \text{Caballo}(Y) \\ C_4 : & \neg \text{CabezaDe}(H, z) \vee \neg \text{Animal}(z) \end{aligned}$$

Paso 3 — Conjunto de cláusulas para la refutación: $\{C_1, C_2, C_3, C_4\}$.

Paso de resolución 1: Resolver C_1 ($\neg \text{Caballo}(x) \vee \text{Animal}(x)$) con C_3 ($\text{Caballo}(Y)$), unificador $\theta = \{x/Y\}$, sobre el par $\text{Caballo}(Y)/\neg \text{Caballo}(x) \Rightarrow$ se deriva $C_5: \text{Animal}(Y)$.

Paso de resolución 2: Resolver C_4 ($\neg \text{CabezaDe}(H, z) \vee \neg \text{Animal}(z)$) con C_2 ($\text{CabezaDe}(H, Y)$), unificador $\theta = \{z/Y\}$, sobre el par $\text{CabezaDe}(H, Y)/\neg \text{CabezaDe}(H, z) \Rightarrow$ se deriva $C_6: \neg \text{Animal}(Y)$.

Paso de resolución 3: Resolver C_5 ($\text{Animal}(Y)$) con C_6 ($\neg \text{Animal}(Y)$), sin unificador adicional (ambos ya están en términos de la constante Y) $\Rightarrow$ se deriva la **cláusula vacía** $\perp$.

Conclusión: Se derivó $\perp$ a partir de $\{\text{premisa}, \neg \text{conclusión}\}$, por lo tanto ese conjunto es insatisfacible. Por refutación, esto confirma que la premisa **implica** la conclusión: la cabeza de un caballo es, en efecto, la cabeza de un animal.

Parte D — Reflexión

7. ¿Por qué la resolución funciona por contradicción?

Para demostrar $BC \models \alpha$ semánticamente habría que revisar *todo* modelo de BC y confirmar que α es verdadera en cada uno — inviable en general en LPO (dominios infinitos). La refutación invierte el problema: asume $\neg \alpha$ y pregunta si $BC \wedge \neg \alpha$ tiene *algún* modelo. Si se logra derivar $\perp$ (la cláusula vacía, sintácticamente insatisfacible), entonces $BC \wedge \neg \alpha$ no tiene ningún modelo, es decir, en **ningún** modelo pueden ser simultáneamente verdaderas BC y $\neg \alpha$. Eso equivale exactamente a decir que en todo modelo donde BC es verdadera, α también lo es — la definición misma de $BC \models \alpha$. La resolución convierte así una pregunta sobre *todos los modelos* (semántica) en una búsqueda de una prueba sintáctica finita de inconsistencia, lo cual es mecánicamente verificable.

8. ¿Por qué la resolución en LPO es solo semi-decidible?

En lógica proposicional el número de símbolos es finito, así que hay exactamente 2^n modelos posibles: TT-Implica? siempre termina (decidible). En LPO los dominios de cuantificación pueden ser infinitos, y la resolución con unificación puede necesitar instanciar variables universales de infinitas maneras distintas, generando una secuencia potencialmente infinita de cláusulas nuevas. El **teorema de completitud** garantiza que si $BC \models \alpha$, la resolución *sí* encontrará $\perp$ en un número **finito** de pasos, tarde o temprano. Pero si $BC \not\models \alpha$, no existe garantía de terminación: el algoritmo puede seguir generando cláusulas para siempre sin nunca confirmar que α *no* se sigue. Por eso el procedimiento es **semi-decidible**: puede confirmar un “sí” en tiempo finito siempre que la respuesta sea sí, pero no puede garantizar confirmar un “no” en tiempo finito.

Parte IV

Planeación

Capítulo 7

Semana 7: Inferencia en LPO y Planeación

Nota 7.0.1 (Eventos esta semana). ■ Publicación **Taller 3**

7.1. Inferencia en LPO: Cierre

Esta clase cierra la unidad de lógica e inferencia (AIMA Cap. 7–9) antes de pasar a planeación: se repasan model checking en lógica proposicional, encadenamiento hacia adelante y hacia atrás sobre cláusulas de Horn, unificación y el occurs check, y resolución por refutación en LPO. Luego se aplica todo el repaso a una guía de ejercicios completa, resuelta paso a paso.

7.1.1. Lógica Proposicional: Entailment y Model Checking

Definición 7.1.1 (Implicación lógica). $KB \models \alpha$ (“ KB implica α ”) si y solo si α es verdadera en *todo* modelo en el que KB es verdadera. Equivalentemente (teorema de refutación):

$$KB \models \alpha \quad \text{si y solo si} \quad KB \wedge \neg\alpha \text{ es insatisfacible.}$$

Model checking (TT-Entails?): para un número finito de símbolos proposicionales, se enumeran las 2^n interpretaciones posibles (tabla de verdad completa), se marcan los **modelos** de KB (filas donde KB es verdadera) y se verifica si α es verdadera en todos ellos. Un solo **contramodelo** (modelo de KB donde α es falsa) basta para refutar $KB \models \alpha$. Es importante distinguir dos nociones que se confunden con frecuencia:

- α es **consistente** con KB : existe *al menos un* modelo de KB donde α es verdadera ($KB \wedge \alpha$ es satisfacible).
- $KB \models \alpha$: α es verdadera en **todos** los modelos de KB .

La primera es mucho más débil que la segunda.

7.1.2. Cláusulas de Horn y Encadenamiento hacia Adelante

Definición 7.1.2 (Cláusula de Horn y cláusula definida). Una **cláusula de Horn** es una disyunción de literales con *a lo sumo un literal positivo* (puede no tener ninguno, en cuyo caso es una **cláusula meta** o de restricción, útil para expresar una contradicción). Una **cláusula definida** es una cláusula de Horn con *exactamente un* literal positivo, y se escribe como implicación

$$P_1 \wedge \cdots \wedge P_n \Rightarrow Q$$

(las premisas P_i deben ser todas verdaderas para concluir Q). Todo el material de esta guía (Ejercicios 2 y 3) usa cláusulas *definidas*: por eso el encadenamiento hacia adelante y hacia atrás son aplicables y completos.

El **encadenamiento hacia adelante** (forward chaining) parte de los hechos conocidos y aplica reglas cuyas premisas ya están satisfechas, derivando nuevos hechos hasta alcanzar la meta o agotar la agenda.

Definición 7.1.3 (PL-FC-ENTAILS?, con agenda y contadores). Se mantiene, para cada regla, un **contador** con el número de premisas aún no confirmadas, y una **agenda** FIFO de símbolos conocidos verdaderos que faltan por procesar. En cada paso se extrae el siguiente símbolo p de la agenda; si p es la meta, se retorna verdadero; si no, se marca p como inferido y, por cada regla donde p aparece en la premisa, se decrementa su contador; si un contador llega a 0, la conclusión de esa regla se agrega a la agenda.

Algorithm 1 PL-FC-ENTAILS?(KB, q) — forward chaining con agenda y contadores

```

1:  $count[c] \leftarrow$  número de símbolos en la premisa de  $c$ , para cada cláusula  $c \in KB$ 
2:  $inferred[s] \leftarrow$  falso, para cada símbolo  $s$ 
3:  $agenda \leftarrow$  símbolos conocidos verdaderos en  $KB$ 
4: while  $agenda$  no está vacía do
5:    $p \leftarrow \text{POP}(agenda)$ 
6:   if  $p = q$  then return verdadero
7:   end if
8:   if  $inferred[p] =$  falso then
9:      $inferred[p] \leftarrow$  verdadero
10:    for cada cláusula  $c$  tal que  $p$  aparece en  $c.PREMISA$  do
11:      decrementar  $count[c]$ 
12:      if  $count[c] = 0$  then
13:        agregar  $c.CONCLUSION$  a  $agenda$ 
14:      end if
15:    end for
16:   end if
17: end while
18: return falso

```

- **Completo** para bases de conocimiento proposicionales de cláusulas de Horn (deriva exactamente los hechos implicados, ni más ni menos).
- **Dirigido por los datos** (*data-driven*): no usa la meta para decidir qué reglas disparar, así que puede derivar hechos verdaderos pero **irrelevantes** para la consulta.

7.1.3. Encadenamiento hacia Atrás

El **encadenamiento hacia atrás** (backward chaining) parte de la meta y busca reglas cuya conclusión coincida con ella; cada premisa de esas reglas se convierte, recursivamente, en una nueva **submeta**. Un hecho de la base resuelve una submeta directamente (hoja del árbol de prueba).

- **Dirigido por la meta** (*goal-driven*): solo explora lo relevante para probar la consulta.
- Cuando una regla no logra probar una submeta (no hay hechos ni otra regla aplicable, o todas las alternativas fallan), ocurre **backtracking**: se descarta el intento y se prueba la siguiente regla cuya conclusión coincida con esa submeta.
- Completo para cláusulas de Horn; es la base operacional de Prolog. Sin control de ciclos puede entrar en bucles infinitos si una submeta depende (directa o indirectamente) de sí misma.

7.1.4. Sustitución, Unificación y Occurs Check

Definición 7.1.4 (Unificador más general, UMG). $\text{UNIFY}(p, q) = \theta$ tal que $\text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$, y θ es el **más general** posible: cualquier otro unificador θ' se obtiene componiendo θ con una sustitución adicional. La unificación procede posición a posición: variable con término (se liga la variable), constante con constante idéntica (coincide sin generar ligadura), constantes distintas o predicados/aridades distintas (falla).

Definición 7.1.5 (Occurs check). Antes de ligar una variable x a un término t , el **occurs check** verifica que x no aparezca dentro de t . Si x ocurriera en t , la ligadura x/t generaría un término **infinito** (p. ej. $x = f(x) = f(f(x)) = \dots$), que no es un objeto válido en LPO. Muchas implementaciones de Prolog omiten el occurs check por eficiencia, arriesgando unificaciones incorrectas que un motor completo rechazaría.

7.1.5. Forma Clausal y Resolución por Refutación en LPO

Definición 7.1.6 (Forma normal conjuntiva (FNC) en LPO — pasos de conversión). Toda sentencia de LPO se convierte a FNC (conjunción de cláusulas, cada una disyunción de literales) siguiendo, en orden, estos pasos:

1. **Eliminar** $\Leftrightarrow$ y $\Rightarrow$: $\alpha \Leftrightarrow \beta$ se reescribe $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$, y $\alpha \Rightarrow \beta$ se reescribe $\neg\alpha \vee \beta$.
2. **Mover $\neg$ hacia adentro** (leyes de De Morgan y de cuantificadores): $\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta$, $\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$, $\neg\forall x \Phi \equiv \exists x \neg\Phi$, $\neg\exists x \Phi \equiv \forall x \neg\Phi$, $\neg\neg\alpha \equiv \alpha$.
3. **Estandarizar variables**: renombrar variables ligadas para que cada cuantificador use un nombre distinto (evita colisiones al combinar cláusulas de sentencias distintas).
4. **Skolemizar** (ver definición siguiente): eliminar cada cuantificador existencial reemplazando su variable por una constante o función de Skolem.
5. **Eliminar los cuantificadores universales** restantes (quedan implícitos: toda variable libre en una cláusula se entiende universalmente cuantificada).
6. **Distribuir $\vee$ sobre $\wedge$** hasta que la fórmula sea una conjunción de disyunciones de literales; cada conjunto separado por $\wedge$ es una cláusula distinta.

Definición 7.1.7 (Skolemización). Si una variable existencial $\exists y$ **no** está dentro del alcance de ningún $\forall$, se reemplaza y por una **constante de Skolem** nueva (un objeto concreto, aunque desconocido, que testifica la existencia). Si $\exists y$ **sí** está dentro del alcance de $\forall x_1, \dots, \forall x_k$, se reemplaza y por una **función de Skolem** nueva $f(x_1, \dots, x_k)$ (el testigo puede depender de esos universales). Por ejemplo, $\forall x \exists y \text{ Padre}(y, x)$ (“todo el mundo tiene un padre”) se skolemiza como $\forall x \text{ Padre}(S(x), x)$: $S(x)$ denota *el* padre de x , que en general depende de x .

■ **Regla de resolución (LPO):**

$$\frac{\ell_1 \vee \dots \vee \ell_k \quad \neg m_1 \vee \dots \vee \neg m_j}{\text{SUBST}(\theta, \ell_1 \vee \dots \vee m_j)} \quad \theta = \text{UNIFY}(\ell_i, m_i)$$

- **Refutación**: para probar $KB \models \alpha$, se agrega $\neg\alpha$ (en FNC) al conjunto de cláusulas de KB y se aplica resolución hasta derivar la **cláusula vacía** $\square$ (contradicción) —lo que confirma que $KB \wedge \neg\alpha$ es insatisfacible— o hasta no poder generar cláusulas nuevas (en cuyo caso $KB \not\models \alpha$).
- La resolución en LPO es **sana y completa por refutación** (teorema de completitud de Gödel), pero solo **semi-decidible**: si $KB \models \alpha$, se garantiza encontrar $\square$ en un número finito de pasos; si $KB \not\models \alpha$, el procedimiento puede no terminar nunca.

7.2. Ejercicios: Guía de Lógica e Inferencia (con solución paso a paso)

Esta sección resuelve, de principio a fin, la *Guía de Ejercicios: Lógica e Inferencia*.

7.2.1. Bloque 1: Preguntas Conceptuales (Verdadero / Falso)

- a. “Si una sentencia α es satisfacible, entonces $KB \models \alpha$ para cualquier base de conocimiento KB consistente con α .”

Falso. Satisfacibilidad de α solo dice que α es verdadera en *algún* modelo; que KB sea consistente con α solo dice que $KB \wedge \alpha$ tiene algún modelo. Ninguna de las dos garantiza que α sea verdadera en **todos** los modelos de KB , que es lo que exige $\models$.

Contraejemplo: $KB = \{P\}$, $\alpha = Q$. α es satisfacible, y KB es consistente con α (el modelo $P=T, Q=T$ satisface ambos), pero $KB \not\models Q$ (el modelo $P=T, Q=F$ también es modelo de KB y no de Q).

- b. “El encadenamiento hacia adelante (forward chaining) es completo para bases de conocimiento proposicionales formadas por cláusulas de Horn.”

Verdadero. Para cláusulas de Horn (cada regla con a lo sumo un literal positivo), el algoritmo con agenda y contadores deriva exactamente todos los símbolos atómicos implicados por KB : es sano (solo deriva lo que se sigue lógicamente) y completo (deriva todo lo que se sigue).

- c. “ $KB \models \alpha$ si y solo si la sentencia $(KB \wedge \neg\alpha)$ es insatisfacible.”

Verdadero. Es el **teorema de refutación**, base de todo método de prueba por contradicción (model checking negativo y resolución): si α fuera falsa en algún modelo de KB , ese modelo satisface simultáneamente KB y $\neg\alpha$; si ningún modelo satisface ambos a la vez, entonces α es verdadera en todo modelo de KB .

- d. “Durante la unificación es válido sustituir un término por una constante de Skolem, de la misma forma en que se sustituye una variable.”

Falso. Las constantes (y funciones) de Skolem se introducen *antes* de la unificación, durante la conversión a FNC, para eliminar cuantificadores existenciales; representan un objeto **fijo pero arbitrario** del dominio, no un comodín instanciable. La unificación solo liga **variables** (símbolos que aún pueden tomar cualquier valor) a términos; una constante de Skolem se comporta como cualquier otra constante: puede aparecer *dentro* de un término al que se liga una variable, pero ella misma no puede recibir una ligadura ni ser tratada como si fuera libre de unificarse con otro término distinto.

7.2.2. Ejercicio 1: El apagón en Paloquemao

Símbolos: P (falló la planta), S (sobrecarga en neveras), T (transformador dañado).

$$R1 : P \vee S \vee T \quad R2 : \neg(P \wedge S) \quad R3 : T \Rightarrow P \quad KB = R1 \wedge R2 \wedge R3$$

P	S	T	$R1$	$R2$	$R3$	KB (i modelo?)
V	V	V	V	F	V	F
V	V	F	V	F	V	F
V	F	V	V	V	V	V (modelo M_1)
V	F	F	V	V	V	V (modelo M_2)
F	V	V	V	V	F	F
F	V	F	V	V	V	V (modelo M_3)
F	F	V	V	V	F	F
F	F	F	F	V	V	F

Cuadro 7.1: Tabla de verdad completa. KB tiene exactamente tres modelos: $M_1 = (V, F, V)$, $M_2 = (V, F, F)$, $M_3 = (F, V, F)$.

a) Tabla de verdad

b) $KB \models (P \vee S)$? En M_1 : $V \vee F = V$. En M_2 : $V \vee F = V$. En M_3 : $F \vee V = V$. Verdadera en los tres únicos modelos de $KB \Rightarrow KB \models (P \vee S)$.

c) $KB \models S$? En M_1 y M_2 , $S = F$. $KB \not\models S$; contramodelo: $M_1 = (P=V, S=F, T=V)$ (o M_2), donde KB es verdadera pero S es falsa.

d) $KB \models \neg(S \wedge T)$? M_1 : $S \wedge T = F \wedge V = F \Rightarrow \neg(\cdot) = V$. M_2 : $F \wedge F = F \Rightarrow V$. M_3 : $V \wedge F = F \Rightarrow V$. Verdadera en los tres modelos $\Rightarrow KB \models \neg(S \wedge T)$.

e) $KB \models (P \vee T)$? M_1 : $V \vee V = V$. M_2 : $V \vee F = V$. M_3 : $F \vee F = \mathbf{F}$. Como M_3 es un modelo de KB donde $P \vee T$ es falsa, $KB \not\models (P \vee T)$: el administrador **no tiene razón**. M_3 muestra que $P \vee T$ es **consistente** con KB (verdadera en M_1, M_2) pero no está **implicada** por KB (falla en M_3): consistencia solo exige un modelo testigo, mientras que implicación exige que se cumpla en *todos* los modelos de KB , sin excepción.

7.2.3. Ejercicio 2: Café de especialidad en el Eje Cafetero

$$\begin{array}{lll}
 R1 : L \wedge H \Rightarrow P & R2 : A \wedge C \Rightarrow PE & R3 : P \wedge PE \Rightarrow CE \\
 R4 : CE \Rightarrow LE & R5 : L \wedge A \Rightarrow SO & \text{Hechos: } L, A, C, H
 \end{array}$$

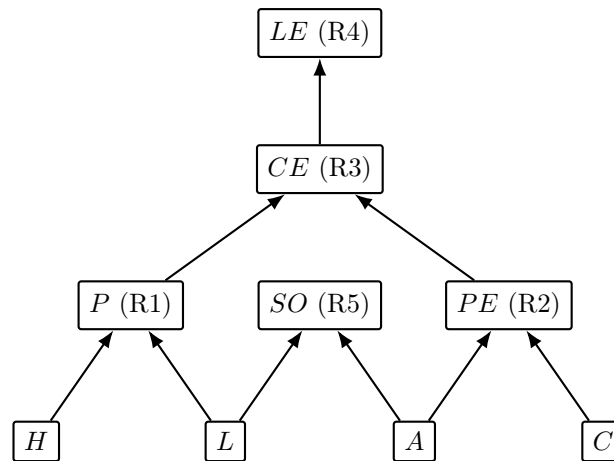


Figura 7.1: Grafo AND-OR: los hechos L, A, C, H están en la base; cada símbolo derivado agrupa (AND, implícito por la regla que lo produce) las premisas indicadas por los dos arcos que entran a él. Nótese que SO no tiene ningún arco de salida: ninguna regla lo usa como premisa.

a) Grafo AND-OR

Paso	Símbolo procesado	Regla que dispara (cuenta a 0)	Agenda al final del paso
1	L	— (decrementa R1: $2 \rightarrow 1$; R5: $2 \rightarrow 1$)	$[A, C, H]$
2	A	R5 dispara $\rightarrow$ agrega SO (decrementa R2: $2 \rightarrow 1$)	$[C, H, SO]$
3	C	R2 dispara $\rightarrow$ agrega PE	$[H, SO, PE]$
4	H	R1 dispara $\rightarrow$ agrega P	$[SO, PE, P]$
5	SO	— (SO no es premisa de ninguna regla)	$[PE, P]$
6	PE	— (decrementa R3: $2 \rightarrow 1$)	$[P]$
7	P	R3 dispara $\rightarrow$ agrega CE	$[CE]$
8	CE	R4 dispara $\rightarrow$ agrega LE	$[LE]$
9	LE	es la consulta $\rightarrow$ se retorna verdadero	$[]$

Cuadro 7.2: Traza completa de PL-FC-ENTAILS? para la consulta LE .

b) Traza de forward chaining (agenda FIFO, orden inicial L, A, C, H)

c) **¿Se deriva LE ?** **Sí.** El hecho derivado SO (paso 2, vía R5: $L \wedge A \Rightarrow SO$) **no contribuye en nada** a probar LE : SO no aparece en la premisa de ninguna regla, así que la cadena que lleva a LE ($P, PE \rightarrow CE \rightarrow LE$) nunca lo utiliza.

d) **¿Por qué se deriva un hecho irrelevante?** Forward chaining es **dirigido por los datos**: en cada paso dispara *toda* regla cuyas premisas ya se satisfacen, sin usar información sobre cuál es la consulta. Como L y A (ya conocidos) satisfacen la premisa de R5 independientemente de que SO sirva o no para responder LE , la regla se dispara igual. Esto refleja que el algoritmo es **sano y completo mirando hacia adelante**: deriva absolutamente todo lo que KB implica, sin ningún mecanismo de poda por relevancia a una meta particular.

7.2.4. Ejercicio 3: La comparsa del Carnaval de Barranquilla

$R1: I \wedge E \Rightarrow H$ $R2: H \wedge V \Rightarrow D$ $R3: G \Rightarrow I$ $R4: B \Rightarrow I$ Hechos: B, E, V

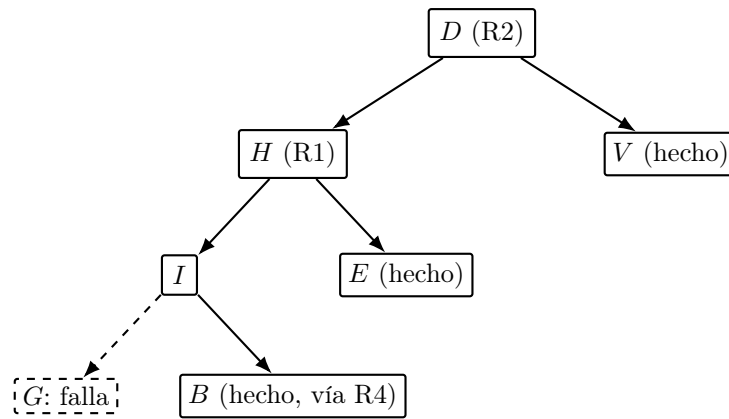


Figura 7.2: Árbol de prueba de backward chaining para D . El intento de probar I vía R3 ($G \Rightarrow I$) falla (rama punteada: G no es hecho ni cabeza de ninguna regla); se hace backtracking y R4 ($B \Rightarrow I$) rescata la prueba, ya que B es un hecho.

a) Árbol de prueba para la meta D

b) Submeta con backtracking El backtracking ocurre en la submeta I : como las reglas se intentan en el orden en que aparecen, primero se prueba **R3** ($G \Rightarrow I$), lo que exige probar G ; pero G no es un hecho ni es la conclusión de ninguna otra regla, así que esa rama **falla** sin alternativas. El algoritmo retrocede a la submeta I y prueba la siguiente regla cuya cabeza es I : **R4** ($B \Rightarrow I$), que exige probar B ; como B es un hecho, R4 **rescata la prueba** y I queda demostrado.

c) Comparación con el Ejercicio 2 Forward chaining en el Ejercicio 2 derivó **5 hechos nuevos**: SO, PE, P, CE, LE (a partir de los 4 hechos iniciales). Backward chaining aquí exploró **7 submetas**: D, H, V, I, G, B, E (una de ellas, G , es una rama fallida). Los números son comparables en este ejemplo pequeño, pero la diferencia cualitativa es la que importa:

- **Dirigido por los datos** (forward chaining): parte de los hechos y aplica todas las reglas disparables, sin mirar la meta; puede derivar información verdadera pero irrelevante (como SO). Es preferible cuando **no hay una única meta fija** y se esperan muchas consultas sobre la misma base (p. ej. un sistema de monitoreo que deriva conclusiones a medida que llegan datos nuevos).
- **Dirigido por la meta** (backward chaining): parte de la consulta y solo explora lo que es relevante para probarla. Es preferible cuando la base de conocimiento es **grande** pero solo una fracción pequeña es relevante para una consulta puntual (p. ej. sistemas expertos de diagnóstico, o Prolog respondiendo una pregunta específica), aunque puede explorar y descartar ramas fallidas (como G) antes de encontrar la regla correcta.

7.2.5. Ejercicio 4: Estación de anillamiento en el páramo

a) Traducción a LPO

a.1. Todas las aves anilladas en Chingaza migran a Sumapaz:

$$\forall x (Ave(x) \wedge Anillada(x, Chingaza)) \Rightarrow Migra(x, Sumapaz)$$

a.2. Existe un ave anillada en Chingaza que cuida su propio nido:

$$\exists x Ave(x) \wedge Anillada(x, Chingaza) \wedge Cuida(x, Nido(x))$$

a.3. Ningún ave migra a la estación donde fue anillada:

$$\forall x \forall e (Ave(x) \wedge Anillada(x, e)) \Rightarrow \neg Migra(x, e)$$

a.4. Toda ave cuida al menos un nido:

$$\forall x Ave(x) \Rightarrow \exists n Cuida(x, n)$$

b) Unificación

b.1. *Anillada(Colibri7, x)* y *Anillada(y, Chingaza)*.

Posición 1: *Colibri7* vs. *y* $\Rightarrow y/Colibri7$. Posición 2: *x* vs. *Chingaza* $\Rightarrow x/Chingaza$.

UMG: $\theta = \{y/Colibri7, x/Chingaza\}$.

b.2. *Migra(x, Nido(x))* y *Migra(Paramuna3, y)*.

Posición 1: *x* vs. *Paramuna3* $\Rightarrow x/Paramuna3$ (occurs check: *x* no ocurre en *Paramuna3*, correcto). Aplicando θ_1 , posición 2: *Nido(Paramuna3)* vs. *y* $\Rightarrow y/Nido(Paramuna3)$ (occurs check: *y* no ocurre en *Nido(Paramuna3)*).

UMG: $\theta = \{x/Paramuna3, y/Nido(Paramuna3)\}$.

b.3. *Vecina(Nido(x), x)* y *Vecina(y, Chingaza)*.

Posición 1: *Nido(x)* vs. *y* $\Rightarrow y/Nido(x)$ (occurs check: *y* $\neq x$, no hay ocurrencia). Posición 2: *x* vs. *Chingaza* $\Rightarrow x/Chingaza$. Componiendo (se sustituye *x* dentro de la ligadura de *y*): *y/Nido(Chingaza)*.

UMG: $\theta = \{x/Chingaza, y/Nido(Chingaza)\}$. Verificación: *Vecina(Nido(Chingaza), Chingaza)* en ambos lados.

b.4. *Anillada(Colibri7, Chingaza)* y *Anillada(Colibri7, Sumapaz)*.

Posición 1: *Colibri7* vs. *Colibri7* (coincide). Posición 2: *Chingaza* vs. *Sumapaz*: dos **constantes distintas**.

No se pueden unificar: la unificación falla porque no existe sustitución que iguale dos constantes diferentes.

b.5. *Cuida(x, Nido(x))* y *Cuida(Nido(y), y)*.

Posición 1: *x* vs. *Nido(y)* $\Rightarrow$ occurs check sobre *x*: *x* no aparece en *Nido(y)* (solo aparece *y*), así que *a priori* pasa: $\theta_1 = \{x/Nido(y)\}$. Aplicando θ_1 a la posición 2: *Nido(x)* se convierte en *Nido(Nido(y))*, que debe unificarse con *y*. Posición 2: *Nido(Nido(y))* vs. *y* $\Rightarrow$ se intentaría *y/Nido(Nido(y))*, pero *y* **ocurre dentro de** *Nido(Nido(y))*.

No se pueden unificar: el occurs check falla en la segunda posición (ligar *y* a un término que contiene a *y* generaría el término infinito $y = Nido(Nido(y)) = Nido(Nido(Nido(y))) = \dots$).

c) Occurs check El **occurs check** es el paso del algoritmo de unificación que, antes de ligar una variable *v* a un término *t*, verifica que *v* **no aparezca dentro de** *t*. De los pares anteriores, solo **b.5** lo requiere (y falla por su causa): al intentar ligar *y* a *Nido(Nido(y))* se crearía un término circular/infinito, que no es un objeto válido en LPO (los términos deben ser finitos). Un motor de inferencia que omita el occurs check (como muchas implementaciones de Prolog, por razones de eficiencia) aceptaría esta unificación y construiría internamente una estructura de datos **cíclica**; esto puede producir inferencias incorrectas (dos términos que en realidad no denotan el mismo objeto se tratan como unificables) o hacer que el motor entre en un **bucle infinito** al intentar recorrer, imprimir o comparar ese término infinito.

7.2.6. Ejercicio 5: El caso de la pieza extraviada

1. $\forall x (TieneLlave(x) \wedge TurnoNocturno(x)) \Rightarrow Accede(x, Boveda)$
2. $\forall x \forall y (Accede(x, Boveda) \wedge Registro(x, y)) \Rightarrow Movio(x, y)$
3. $TieneLlave(Bruno)$ 4. $TurnoNocturno(Bruno)$ 5. $Registro(Bruno, Poporo)$
6. $TieneLlave(Amparo)$ 7. $TurnoNocturno(Celia)$ $\alpha = \exists x Movio(x, Poporo)$

a) Forma clausal (variables estandarizadas) Se aplican los pasos de conversión a FNC de la definición anterior a cada sentencia.

Sentencia 1 ($TieneLlave(x) \wedge TurnoNocturno(x) \Rightarrow Accede(x, Boveda)$):

Paso 1 (eliminar $\Rightarrow$):

$$\forall x \neg(TieneLlave(x) \wedge TurnoNocturno(x)) \vee Accede(x, Boveda)$$

Paso 2 (De Morgan):

$$\forall x (\neg TieneLlave(x) \vee \neg TurnoNocturno(x)) \vee Accede(x, Boveda)$$

Paso 3 (estandarizar): $x \rightarrow x_1$.

Pasos 4–6 (no hay $\exists$ que skolemizar; se elimina $\forall$; ya es disyunción):

$$C_1 : \neg TieneLlave(x_1) \vee \neg TurnoNocturno(x_1) \vee Accede(x_1, Boveda)$$

Sentencia 2 ($Accede(x, Boveda) \wedge Registro(x, y) \Rightarrow Movio(x, y)$): por el mismo procedimiento (eliminar $\Rightarrow$, De Morgan, estandarizar $x, y \rightarrow x_2, y_2$, sin existenciales que skolemizar), se obtiene directamente

$$C_2 : \neg Accede(x_2, Boveda) \vee \neg Registro(x_2, y_2) \vee Movio(x_2, y_2).$$

Hechos (ya son cláusulas unitarias, sin cuantificadores ni conectivos que eliminar):

$$C_3 : TieneLlave(Bruno) \quad C_4 : TurnoNocturno(Bruno) \quad C_5 : Registro(Bruno, Poporo)$$

$$C_6 : TieneLlave(Amparo) \quad C_7 : TurnoNocturno(Celia)$$

Negación de la meta $\alpha = \exists x Movio(x, Poporo)$:

Paso 0 (negar): $\neg\alpha = \neg\exists x Movio(x, Poporo)$.

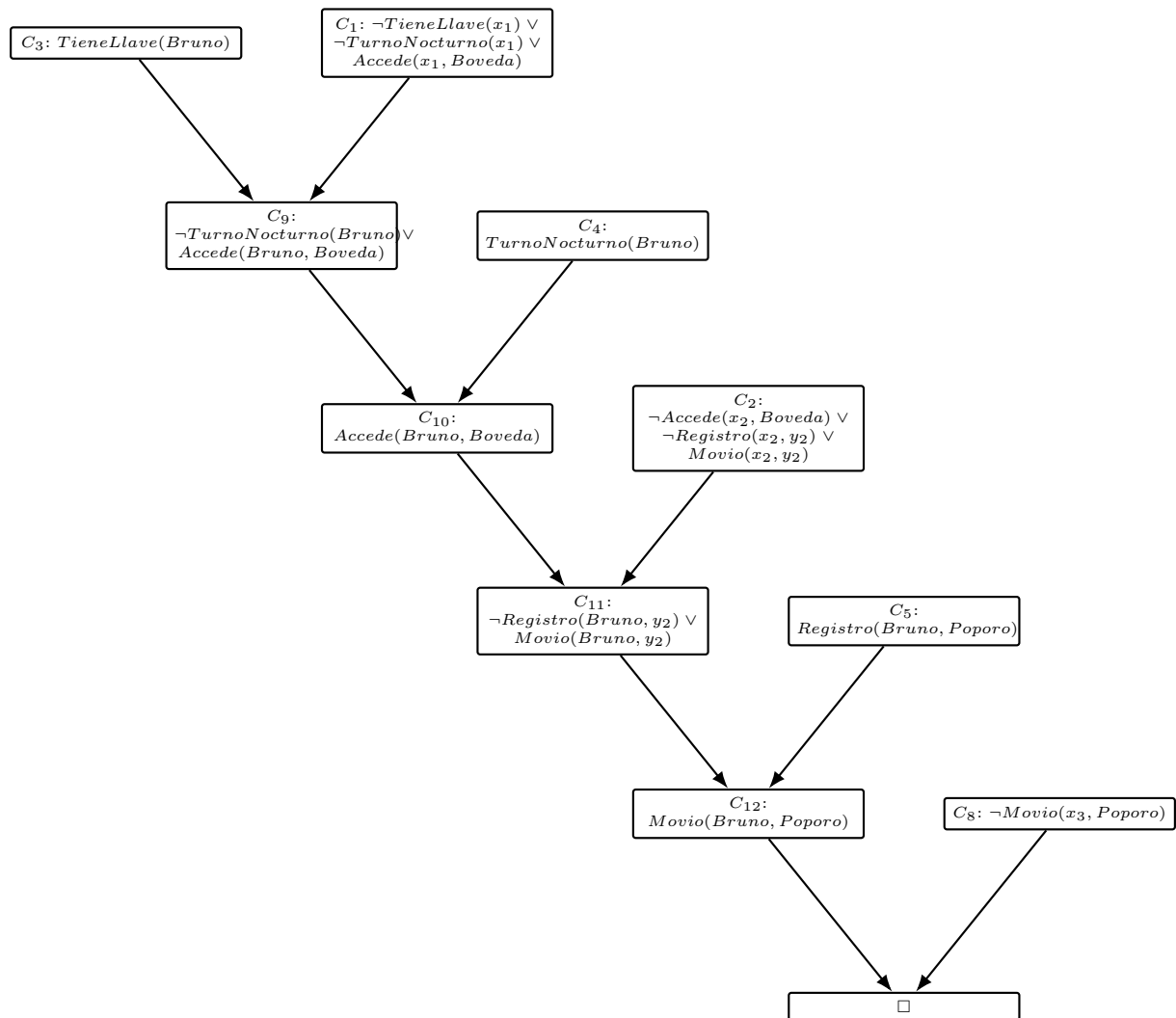
Paso 2 (mover $\neg$ hacia adentro, $\neg\exists x \Phi \equiv \forall x \neg\Phi$): $\forall x \neg Movio(x, Poporo)$.

Paso 3 (estandarizar): $x \rightarrow x_3$.

Pasos 4–6 (no hay $\exists$ que skolemizar; se elimina $\forall$): $C_8 : \neg Movio(x_3, Poporo)$.

Nótese que $\exists x$ en la meta *original* α no se skolemiza directamente: se skolemiza la variable existencial *después de negar*, y aquí la negación la convirtió en universal, así que no aparece ninguna constante de Skolem en C_8 (a diferencia del Ejercicio 4.a.2, donde el $\exists x$ sí quedaría como variable libre estandarizada si se llevara a FNC, sin universales que lo envuelvan).

Paso	Cláusulas resueltas	MGU	Resolvente
1	C_3 con C_1	$\{x_1/Bruno\}$	$C_9 : \neg TurnoNocturno(Bruno) \vee Accede(Bruno, Boveda)$
2	C_4 con C_9	$\{\}$ (ya coincide en $Bruno$)	$C_{10} : Accede(Bruno, Boveda)$
3	C_{10} con C_2	$\{x_2/Bruno\}$	$C_{11} : \neg Registro(Bruno, y_2) \vee Movio(Bruno, y_2)$
4	C_5 con C_{11}	$\{y_2/Poporo\}$	$C_{12} : Movio(Bruno, Poporo)$
5	C_{12} con C_8	$\{x_3/Bruno\}$	$\square$ (cláusula vacía)

Cuadro 7.3: Refutación: se deriva $\square$ a partir de $KB \wedge \neg\alpha$.Figura 7.3: Árbol de refutación completo: cada resolvente combina los dos nodos que apuntan hacia él, hasta llegar a $\square$.

b) Refutación por resolución

c) **Extracción de la respuesta** La variable existencial de la meta original (x en $\exists x Movio(x, Poporo)$) se estandarizó como x_3 en C_8 . Acumulando las sustituciones a lo largo de la refutación, x_3 termina ligada a $Bruno$ (paso 5). **Respuesta: Bruno movió el poporo.**

d) Por qué Amparo y Celia no cierran una refutación alternativa

- **Amparo** ($C_6: TieneLlave(Amparo)$): resolviendo con C_1 ($x_1/Amparo$) se obtiene

$$\neg TurnoNocturno(Amparo) \vee Accede(Amparo, Boveda).$$

Para continuar haría falta resolver el literal $\neg TurnoNocturno(Amparo)$ contra una cláusula unitaria $TurnoNocturno(Amparo)$, pero esa cláusula **no existe** en la KB (el enunciado aclara que Amparo no estuvo en el turno nocturno): la cadena se detiene ahí.

- **Celia** ($C_7: TurnoNocturno(Celia)$): resolviendo con C_1 ($x_1/Celia$) se obtiene

$$\neg TieneLlave(Celia) \vee Accede(Celia, Boveda).$$

Aquí falta una cláusula unitaria $TieneLlave(Celia)$ para resolver $\neg TieneLlave(Celia)$; tampoco existe (el enunciado aclara que Celia no tiene llave). Igual que con Amparo, no hay literal complementario que unifique.

- Incluso si, hipotéticamente, se lograra derivar $Accede(Amparo, Boveda)$ o $Accede(Celia, Boveda)$, el segundo paso exigiría resolver contra C_2 y luego necesitaría una cláusula $Registro(Amparo, y)$ o $Registro(Celia, y)$ para producir *Movio*; la única cláusula *Registro* de la KB es $Registro(Bruno, Poporo)$ así que tampoco hay unificador posible ahí.

e) **¿Por qué probar $KB \wedge \neg\alpha$ insatisfacible en vez de derivar α directamente?** Probar $KB \models \alpha$ “directamente” exigiría verificar que α es verdadera en *todo* modelo de KB —en LPO, con dominios potencialmente infinitos, no hay forma de enumerar todos los modelos. La refutación invierte el problema: por el **teorema de refutación** ($KB \models \alpha \Leftrightarrow KB \wedge \neg\alpha$ insatisfacible), basta con encontrar *una* contradicción sintáctica finita (la cláusula vacía $\square$) para concluir que **ningún** modelo puede satisfacer KB y $\neg\alpha$ a la vez, es decir, que todo modelo de KB satisface α . El principio que conecta ambas cosas es precisamente ese teorema de refutación, sustentado por el **teorema de completitud** de la resolución: convierte una pregunta semántica sobre *todos los modelos* en una búsqueda sintáctica, finita y mecánicamente verificable, de una contradicción.

7.3. Planeación: Definición y Algoritmos Generales

7.3.1. ¿Qué es la Planeación Clásica?

Definición 7.3.1 (Planeación clásica). La **planeación clásica** es la tarea de hallar una secuencia de acciones que lleve a un agente desde un estado inicial hasta un estado que satisface una meta, en ambientes **discretos, determinísticos, estáticos y completamente observables**.

Aproximaciones previas y sus limitaciones:

- **Agente de solución de problemas** (búsqueda genérica): requiere una **función heurística** diseñada a mano para cada nuevo dominio.
- **Agente de lógica proposicional híbrido**: requiere codificar el problema completo como sentencias proposicionales, con una **representación explícita** de un espacio de estados que crece exponencialmente.

Ejemplo 7.3.1 (Por qué la representación importa: comprar libros en línea). Si se define una acción de compra distinta por cada código ISBN de 10 dígitos, un agente sin estructura de dominio debe considerar del orden de 10 mil millones de acciones. Para comprar *cuatro* libros distintos (meta $Tener(A) \wedge Tener(B) \wedge Tener(C) \wedge Tener(D)$), el número de planes de cuatro pasos a examinar sin explotar la estructura del problema es del orden de 10^{40} . Sin embargo, un planificador que sepa que $Tener(c) \Rightarrow Comprar(c)$ es una regla **independiente por cada código** c resuelve el problema de forma trivial. Esto motiva una representación **factorizada e independiente de dominio**: PDDL.

7.3.2. PDDL: Representación de Estados

PDDL (*Planning Domain Definition Language*) representa estados, acciones, metas y dominios de forma factorizada: los mismos algoritmos de planeación funcionan para cualquier problema descrito en PDDL, sin heurísticas ad hoc por dominio.

Definición 7.3.2 (Fluente atómico definido). Un **estado** se representa como una conjunción de **fluentes atómicos definidos**:

- **Fluente**: un aspecto del mundo que puede cambiar con el tiempo.
- **Atómico**: un único predicado aplicado a argumentos (no hay $\wedge$, $\vee$ ni $\Rightarrow$ dentro del fluente mismo).
- **Definido**: los argumentos son **constantes** (sin variables) y el fluente no está negado.

Ejemplos válidos: $Pobre \wedge Desconocido$; $En(Camion_1, Melbourne) \wedge En(Camion_2, Sydney)$.

No permitidos: $En(x, y)$ (tiene variables), $\neg Pobre$ (tiene negación).

PDDL usa una **base de datos semántica**: todo fluente que no aparece explícitamente en la descripción de un estado se asume **falso** (suposición de mundo cerrado), y constantes distintas (p. ej. $Camion_1, Camion_2$) se asumen objetos **distintos entre sí** (suposición de nombres únicos).

7.3.3. PDDL: Schema de Acción

Definición 7.3.3 (Schema de acción). Un **schema de acción** representa toda una **familia** de acciones mediante variables. Sus elementos son: un **nombre**, una **lista de variables**, una **precondición** y un **efecto**, ambos conjunciones de literales (positivos o negativos).

Ejemplo (schema, con variables):

Acción($Volar(p, desde, hacia)$)

PRECOND : $En(p, desde) \wedge Avión(p) \wedge Aeropuerto(desde) \wedge Aeropuerto(hacia)$

EFECTO : $\neg En(p, desde) \wedge En(p, hacia)$

La misma acción, instanciada con constantes:

Acción($Volar(P_1, SFO, JFK)$)

PRECOND : $En(P_1, SFO) \wedge Avión(P_1) \wedge Aeropuerto(SFO) \wedge Aeropuerto(JFK)$

EFECTO : $\neg En(P_1, SFO) \wedge En(P_1, JFK)$

Listing 7.1: El mismo schema de acción en PDDL

```

1 (:action Fly
2   :parameters (?p - Plane ?from ?to - Airport)
3   :precondition (and (At ?p ?from) (Plane ?p)
4                     (Airport ?from) (Airport ?to))
5   :effect (and (At ?p ?to) (not (At ?p ?from))))

```

Definición 7.3.4 (Acción aplicable y RESULTADO). Una acción a es **aplicable** en un estado s si cada literal positivo de su precondición está en s y ningún literal negado de su precondición está en s (equivalentemente: la precondición se determina unificando sus literales con los fluentes de s , sustituyendo variables por constantes). El estado resultante de ejecutar a en s es

$$\text{RESULTADO}(s, a) = (s - \text{DEL}(a)) \cup \text{ADD}(a),$$

donde $\text{DEL}(a)$ son los fluentes que aparecen como literales **negados** en el efecto de a (se eliminan de s) y $\text{ADD}(a)$ son los que aparecen como literales **positivos** (se añaden a s).

Un **dominio de planeación** es un conjunto de schemas de acción. Un **problema** específico en ese dominio agrega un **estado inicial** (INICIAR, conjunción de fluentes definidos) y una **meta** (OBJETIVO, conjunción de literales positivos o negativos que *sí* puede contener variables, interpretadas existencialmente): por ejemplo, $\text{OBJETIVO}(En(C_1, SFO) \wedge \neg En(C_2, SFO) \wedge En(p, SFO))$ se satisface con cualquier avión p que esté en SFO, siempre que C_1 esté en SFO y C_2 no lo esté.

7.3.4. Ejemplos de Dominios Clásicos

Transporte de carga aérea. Predicados: $En(c, p)$ (el cargo c está dentro del avión p), $En(x, a)$ (el objeto x está en el aeropuerto a), $Dentro(c, p)$. Como PDDL no cuenta con un cuantificador universal para expresar “todo el cargo dentro del avión viaja con él”, la estrategia es que un cargo **deja de estar** en su aeropuerto (En) mientras está dentro de un avión, y solo vuelve a tener una ubicación (En) en el aeropuerto de destino.

Acción($Cargar(c, p, a)$)

PRECOND : $En(c, a) \wedge En(p, a) \wedge Cargo(c) \wedge Avión(p) \wedge Aeropuerto(a)$

EFECTO : $\neg En(c, a) \wedge Dentro(c, p)$

Acción($Descargar(c, p, a)$)

PRECOND : $Dentro(c, p) \wedge En(p, a) \wedge Cargo(c) \wedge Avión(p) \wedge Aeropuerto(a)$

EFECTO : $En(c, a) \wedge \neg Dentro(c, p)$

Con el siguiente estado inicial:

INICIAR($En(C_1, SFO) \wedge En(C_2, JFK) \wedge En(P_1, SFO) \wedge En(P_2, JFK)$

$\wedge Cargo(C_1) \wedge Cargo(C_2) \wedge Avión(P_1) \wedge Avión(P_2) \wedge Aeropuerto(JFK) \wedge Aeropuerto(SFO)$)

y la siguiente meta:

OBJETIVO($En(C_1, JFK) \wedge En(C_2, SFO)$)

una solución es

$[Cargar(C_1, P_1, SFO), Volar(P_1, SFO, JFK), Descargar(C_1, P_1, JFK),$
 $Cargar(C_2, P_2, JFK), Volar(P_2, JFK, SFO), Descargar(C_2, P_2, SFO)].$

Neumático de repuesto. Acciones $Quitar(x, y)$ (quitar el objeto x de y), $Colocar(x, y)$, y $DejarDeNoche$: sin precondiciones, con el efecto de **borrar** todo lo que se sabía sobre la ubicación de las ruedas, modelando que en un barrio peligroso las llantas pueden desaparecer si el carro queda sin vigilancia.

Acción($Quitar(Repuesto, Maletero)$), PRECOND : $En(Repuesto, Maletero)$

EFECTO : $\neg En(Repuesto, Maletero) \wedge En(Repuesto, Suelo)$

Acción($Colocar(Repuesto, Eje)$), PRECOND : $En(Repuesto, Suelo) \wedge \neg En(Deshinchada, Eje)$

EFECTO : $\neg En(Repuesto, Suelo) \wedge En(Repuesto, Eje)$

Con el siguiente estado inicial y meta:

INICIAR($En(Deshinchada, Eje) \wedge En(Repuesto, Maletero)$)

OBJETIVO($En(Repuesto, Eje)$)

una solución es

$[Quitar(Repuesto, Maletero), Quitar(Deshinchada, Eje), Colocar(Repuesto, Eje)].$

Mundo de bloques. Predicados: *Sobre*(b, x) (el bloque b está sobre x , que puede ser otro bloque o la mesa), *Despejar*(b) (no hay ningún bloque sobre b), *Bloque*(b). Una sola acción *Mover*(b, x, y) no basta: al mover el *último* bloque de una pila hacia la mesa no hay ningún bloque y concreto que reciba el efecto *Despejar*(y), así que PDDL usa una segunda acción dedicada, *MoverSobreTablero*(b, x), para mover un bloque directamente a la mesa.

Acción(*Mover*(b, x, y))

PRECOND : $Sobre(b, x) \wedge Despejar(b) \wedge Despejar(y) \wedge Bloque(b) \wedge (b \neq x) \wedge (b \neq y) \wedge (x \neq y)$

EFEECTO : $Sobre(b, y) \wedge Despejar(x) \wedge \neg Sobre(b, x) \wedge \neg Despejar(y)$

Acción(*MoverSobreTablero*(b, x))

PRECOND : $Sobre(b, x) \wedge Despejar(b) \wedge Bloque(b) \wedge (b \neq x)$

EFEECTO : $Sobre(b, Mesa) \wedge Despejar(x) \wedge \neg Sobre(b, x)$

Con el siguiente estado inicial:

INICIAR($Sobre(A, Mesa) \wedge Sobre(B, Mesa) \wedge Sobre(C, Mesa)$

$\wedge Bloque(A) \wedge Bloque(B) \wedge Bloque(C) \wedge Despejar(A) \wedge Despejar(B) \wedge Despejar(C)$)

y la siguiente meta:

OBJETIVO($Sobre(A, B) \wedge Sobre(B, C)$)

una solución de dos pasos es [*Mover*($B, Mesa, C$), *Mover*($A, Mesa, B$)].

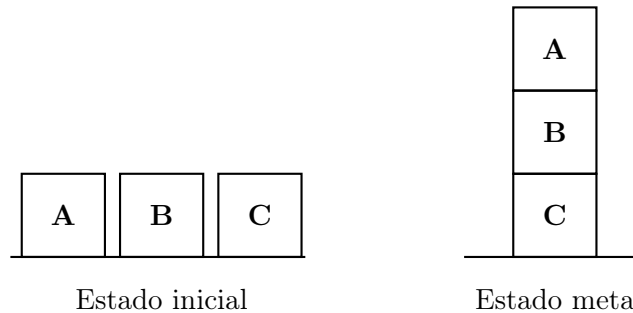


Figura 7.4: Mundo de bloques: en el estado inicial A , B y C están cada uno directamente sobre la mesa; la meta apila A sobre B sobre C . La solución [*Mover*($B, Mesa, C$), *Mover*($A, Mesa, B$)] primero coloca B sobre C y luego A sobre B .

7.3.5. Búsqueda hacia Adelante en el Espacio de Estados (Progresión)

La **planeación de progresión** aplica los algoritmos de búsqueda genéricos (Cap. 3–4) directamente sobre PDDL: parte del estado inicial (conjunto de literales positivos, con suposición de mundo cerrado) y en cada estado s genera ACCIONES(s) unificando las precondiciones de cada schema con los fluentes de s , sustituyendo variables por las constantes que hacen coincidir la unificación (instanciación definida).

- **Problema:** un schema con varios literales en su precondición puede unificar de *muchas* formas distintas. En el dominio del neumático, la precondición *En*(obj, loc) de *Quitar* unifica tanto con $\{obj/Deshinchada, loc/Eje\}$ como con $\{obj/Repuesto, loc/Maletero\}$, generando ramas adicionales por cada instanciación posible.

- **Ejemplo de escala (carga aérea):** con 10 aeropuertos, 5 aviones y 20 piezas de cargo por aeropuerto (50 aviones y 200 cargas en total, cada avión puede volar a 9 aeropuertos y cada carga puede cargarse/descargarse), hay entre 450 y 10.450 acciones aplicables por estado (≈ 2000 en promedio). La meta de mover toda la carga de un aeropuerto a otro requiere una solución de **41 pasos**, lo que da un árbol de búsqueda de hasta 2000⁴¹ nodos: la búsqueda ciega es completamente inviable, y motiva las **heurísticas independientes de dominio** de la Clase 3.

7.3.6. Búsqueda hacia Atrás en el Espacio de Estados (Regresión)

La **planeación de regresión** parte de la meta y aplica acciones “hacia atrás” hasta llegar a un estado que coincide con el inicial, considerando en cada paso únicamente las **acciones relevantes**.

Definición 7.3.5 (Acción relevante). Una acción a es **relevante** para una meta g si el efecto de a unifica con *al menos un* literal de g , **sin negar ninguna otra parte** de g .

Ejemplo: con meta $\neg Pobre \wedge Famoso$, una acción con efecto $Famoso$ es relevante. Una acción con efecto $Pobre \wedge Famoso$ **no** es relevante –aunque también produzca $Famoso$ – porque su efecto haría verdadero $Pobre$, contradiciendo $\neg Pobre$ en la meta.

Dada una meta g y una acción relevante a , la **regresión** de g sobre a produce una nueva meta (estado predecesor) g' :

$$\text{Pos}(g') = (\text{Pos}(g) - \text{ADD}(a)) \cup \text{Pos}(\text{PRECOND}(a))$$

$$\text{NEG}(g') = (\text{NEG}(g) - \text{DEL}(a)) \cup \text{NEG}(\text{PRECOND}(a))$$

Intuición: los literales que a ya garantiza se **remueven** de la meta (ya quedarán satisfechos al ejecutar a), y se **añaden** las precondiciones de a , porque deben cumplirse *antes* de ejecutarla –si no, a no podría haberse ejecutado.

Ejemplo: meta $En(C_2, SFO)$, schema $Descargar$ con efecto $En(c, a)$. Unificando el objetivo con el efecto se obtiene $\theta = \{c/C_2, a/SFO\}$; tras estandarizar $p \rightarrow p'$:

$$\text{Acción}(\text{Descargar}(C_2, p', SFO))$$

$$\text{PRECOND} : \text{Dentro}(C_2, p') \wedge En(p', SFO) \wedge \text{Cargo}(C_2) \wedge \text{Avión}(p') \wedge \text{Aeropuerto}(SFO)$$

$$\text{EFECTO} : En(C_2, SFO) \wedge \neg \text{Dentro}(C_2, p')$$

La nueva meta (estado predecesor) es

$$g' = \text{Dentro}(C_2, p') \wedge En(p', SFO) \wedge \text{Cargo}(C_2) \wedge \text{Avión}(p') \wedge \text{Aeropuerto}(SFO).$$

Comparación con la búsqueda hacia adelante: suponga la meta $Tener(9780134610993)$ con el schema $\text{Acción}(\text{Comprar}(i))$, $\text{PRECOND} : ISBN(i)$, $\text{EFECTO} : Tener(i)$. Sin heurística, la búsqueda hacia **adelante** tendría que enumerar del orden de 10 mil millones de acciones Comprar definidas para encontrar la correcta. La búsqueda hacia **atrás** solo unifica la meta con el efecto ($\theta = \{i'/9780134610993\}$) y obtiene el estado predecesor directamente en un paso: $ISBN(9780134610993)$. Esto ilustra por qué la regresión suele ser más eficiente cuando la meta es específica y el espacio de acciones aplicables hacia adelante es enorme.

7.3.7. Planeación como Satisfacibilidad Booleana (SATPLAN)

Estrategia: fijar un horizonte de tiempo T y codificar *todo* el problema de planeación (con ese horizonte) como una única fórmula proposicional grande, que se entrega a un solucionador SAT; un modelo satisfactible codifica un plan válido.

1. **Proposicionalizar las acciones:** por cada schema, sustituir sus variables por constantes y añadir un superíndice de tiempo t : p. ej. $VolarP_1SFOJFK^1$.
2. **Axiomas de exclusión de acciones:** para cada par de acciones que no pueden ocurrir simultáneamente,

$$\neg(VolarP_1SFOJFK^1 \wedge VolarP_1SFOBUH^1)$$

(el mismo avión no puede volar a dos destinos distintos en el mismo paso de tiempo).

3. **Axiomas de precondition:** por cada acción definida A^t ,

$$A^t \Rightarrow PRE(A)^t$$

(si la acción ocurrió en el tiempo t , sus preconditiones eran verdaderas en t). Por ejemplo, $VolarP_1SFOJFK^1 \Rightarrow En(P_1, SFO)^1 \wedge Avión(P_1)^1 \wedge Aeropuerto(SFO)^1 \wedge Aeropuerto(JFK)^1$.

4. **Axiomas de estado sucesor:** por cada fuente F ,

$$F^{t+1} \Leftrightarrow ACCIONCAUSAF^t \vee (F^t \wedge \neg ACCIONCAUSANOF^t),$$

donde ACCIONCAUSAF es la disyunción de todas las acciones definidas que **añaden** F , y ACCIONCAUSANOF la disyunción de las que lo **eliminan**.

5. **Estado inicial:** se asevera F^0 por cada fuente verdadero en el problema inicial, y $\neg F^0$ por cada fuente no mencionado.
6. **Meta proposicionalizada:** disyunción sobre todas las instancias definidas donde las variables de la meta se reemplazan por constantes. Por ejemplo, la meta $En(A, x) \wedge Bloque(x)$ con objetos A, B, C se convierte en

$$(En(A, A) \wedge Bloque(A)) \vee (En(A, B) \wedge Bloque(B)) \vee (En(A, C) \wedge Bloque(C)).$$

7.3.8. Otras Estrategias de Planeación

- **Grafo de planeación:** estructura de capas alternadas de literales y acciones ($S_0, A_0, S_1, A_1, \dots$) que se construye en tiempo polinomial y acota qué literales son alcanzables en cuántos pasos. Se retoma en la **Clase 3** como base de la heurística FF y de las heurísticas de planeación en general.
- **Cálculo situacional** (*situation calculus*): representa la planeación directamente en LPO mediante un término $do(a, s)$ que denota el estado resultante de ejecutar la acción a en la situación s (p. ej. $do(Move(rob, o109, o103), init)$). Permite razonar con todo el aparato de LPO (cuantificadores, igualdad, teoría de conjuntos), a costa de perder la estructura factorizada –y por tanto buena parte de la eficiencia– que ofrece PDDL.
- **Planeación de orden parcial (POP):** en vez de comprometerse con un **orden total y rígido** entre las acciones del plan (*fixed order*), POP solo impone las restricciones de orden estrictamente necesarias (*least commitment*), dejando sin ordenar entre sí las acciones que no dependen una de otra.

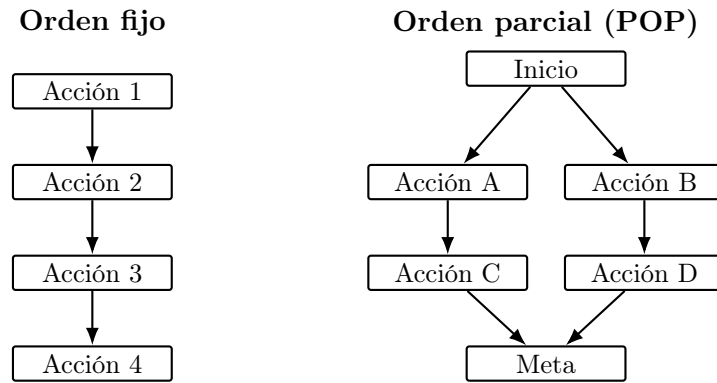


Figura 7.5: Orden fijo vs. orden parcial: en el plan de orden fijo cada acción depende rígidamente de la anterior; en POP, las acciones A/B (y C/D) son mutuamente independientes y solo se ordenan respecto a lo que sí exigen sus dependencias reales, dejando margen para ejecutarlas en cualquier orden entre sí o en paralelo.

7.4. Planeación: Heurísticas y Planeación Jerárquica

La Clase 2 mostró que la búsqueda ciega sobre PDDL no escala (hasta 2000⁴¹ nodos en el ejemplo de carga aérea). Esta clase repasa qué hace admisible a una heurística y luego construye heurísticas **independientes de dominio** para planeación por relajación del problema, un conjunto de técnicas de **podado independiente de dominio**, y finalmente la **planeación jerárquica (HTN)**, que ataca el problema de escala organizando acciones en niveles de abstracción en vez de heurísticas.

7.4.1. Repaso: Admisibilidad y Monotonidad de Heurísticas

Definición 7.4.1 (Heurística). Una **heurística** $h(n)$ es una función que estima qué tan cerca está un estado (nodo n) de la meta, y sirve para guiar la búsqueda hacia el camino con mayor promesa. Ejemplos clásicos: distancia Manhattan, distancia euclidiana.

Evaluar una heurística no es solo verificar que *encuentre* una solución, sino que encuentre el **camino más corto**. Tres preguntas clave:

- **Admisibilidad:** ¿la heurística nunca sobrestima el costo real? Formalmente,

$$0 \leq h(n) \leq h^*(n),$$

donde $h^*(n)$ es el costo real óptimo desde n hasta la meta. Una heurística admisible garantiza encontrar el camino más corto (cuando existe solución).

- **Informedness:** dadas dos heurísticas admisibles, ¿cuál domina a la otra (da estimados más ajustados, y por tanto expande menos nodos)?
- **Monotonidad / consistencia:** si ya se expandió un estado, ¿se garantiza que no se va a encontrar el mismo estado después por un camino más corto? Una heurística es **consistente** si, para todo nodo n_i y sucesor n_j alcanzado con una acción de costo $\text{COSTO}(n_i, n_j)$,

$$h(n_i) - h(n_j) \leq \text{COSTO}(n_i, n_j).$$

7.4.2. Heurísticas en Planeación: Relajación de Problemas

Diferencia clave frente a búsqueda genérica: en planeación, las heurísticas pueden construirse de forma **independiente del dominio**, porque PDDL ya factoriza el problema en flujos y schemas de acción –no hace falta que un experto diseñe la heurística a mano para cada dominio nuevo.

Un problema de búsqueda se modela como un grafo: nodos = estados, arcos = acciones, objetivo = estado meta. La estrategia general es la **relajación**: construir una versión más fácil del problema cuyo costo exacto (o una aproximación de él) sirve como heurística admisible para el problema original. Dos formas de relajar:

- **Agregar más arcos al grafo** (permitir más transiciones entre estados) –heurística de ignorar precondiciones.
- **Agrupar múltiples nodos en uno solo** (menos estados) –abstracción de estado.

7.4.3. Heurística de Ignorar Precondiciones

Definición 7.4.2 (Ignorar precondiciones). Se remueven todas las precondiciones de las acciones: toda acción queda aplicable en cualquier estado. En el problema relajado, cada fuente de la meta se alcanza en un solo paso (o el problema es imposible). El número de pasos para resolver el problema relajado es, aproximadamente, el número de metas todavía insatisfechas.

Estrategia general: contar el mínimo número de acciones tal que la unión de sus efectos satisfaga la meta. Esto es exactamente una **instancia del problema de cobertura de conjuntos** (*set cover*): dado un universo $\{1, \dots, m\}$ y n conjuntos cuya unión cubre el universo, hallar el menor número de conjuntos cuya unión también lo cubre.

Ejemplo 7.4.1 (Cobertura de conjuntos). Sea $U = \{1, 2, 3, 4, 5\}$ y $S = \{\{1, 2, 3\}, \{4, 5\}\}$: basta escoger los dos conjuntos de S para cubrir U .

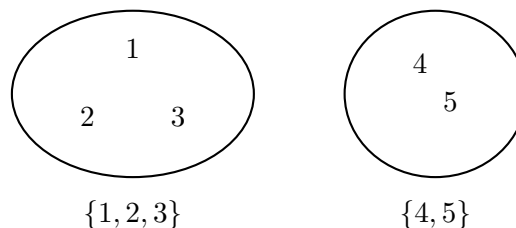


Figura 7.6: $U = \{1, 2, 3, 4, 5\}$ cubierto por $S = \{\{1, 2, 3\}, \{4, 5\}\}$: la cobertura mínima requiere ambos conjuntos.

El problema de cobertura mínima es **NP-Hard** en general, pero un algoritmo voraz simple (cobertura por conjuntos) devuelve una cobertura dentro de un factor $\log n$ de la mínima real, donde n es el número de literales en la meta –suficiente para una heurística útil, aunque no exacta.

Paso a paso del algoritmo voraz sobre el ejemplo anterior:

1. Elementos sin cubrir: $\{1, 2, 3, 4, 5\}$.
2. Escoger el conjunto de S que cubre **más** elementos sin cubrir todavía: $\{1, 2, 3\}$ cubre 3 elementos, $\{4, 5\}$ cubre 2; se escoge $\{1, 2, 3\}$.
3. Elementos sin cubrir restantes: $\{4, 5\}$.

4. Escoger de nuevo el conjunto que más cubre: solo queda $\{4, 5\}$, que cubre los 2 restantes; se escoge.
5. Elementos sin cubrir: $\emptyset \Rightarrow$ se detiene. Cobertura hallada: $\{\{1, 2, 3\}, \{4, 5\}\}$, de tamaño 2 –que en este ejemplo coincide con la cobertura mínima real.

Ejemplo 7.4.2 (8-puzzle). Con el schema

$$\text{Acción}(\text{Mover}(t, s_1, s_2))$$

$$\text{PRECOND} : \text{En}(t, s_1) \wedge \text{Baldosa}(t) \wedge \text{Vacía}(s_2) \wedge \text{Adyacente}(s_1, s_2)$$

$$\text{EFECTO} : \text{En}(t, s_2) \wedge \text{Vacía}(s_1) \wedge \neg \text{En}(t, s_1) \wedge \neg \text{Vacía}(s_2)$$

dos relajaciones distintas producen dos heurísticas clásicas:

- Remover $\text{Vacía}(s_2) \wedge \text{Adyacente}(s_1, s_2)$: cualquier baldosa se puede mover a cualquier casilla en una sola acción $\Rightarrow$ heurística del **número de baldosas fuera de lugar**.
- Remover solo $\text{Vacía}(s_2)$ (se conserva Adyacente): cada baldosa solo se puede mover a casillas adyacentes, pero sin bloqueos $\Rightarrow$ heurística de **distancia Manhattan**.

Ejemplo numérico (informedness): supóngase un estado con exactamente 3 baldosas fuera de su posición meta.

1. Heurística h_1 (baldosas fuera de lugar): cuenta directamente las baldosas mal ubicadas, sin importar qué tan lejos estén $\Rightarrow h_1 = 3$.
2. Heurística h_2 (distancia Manhattan): suma, para cada baldosa fuera de lugar, el número de casillas que la separan (en horizontal + vertical) de su posición meta. Si esas 3 baldosas están, respectivamente, a 1, 2 y 2 pasos de su posición meta, $h_2 = 1 + 2 + 2 = 5$.
3. Como $h_2 \geq h_1$ siempre (mover una baldosa fuera de lugar toma al menos 1 paso, y puede tomar más si no está adyacente a su destino), h_2 **domina** a h_1 : es una heurística más informada –ambas son admisibles, pero h_2 da estimados más ajustados y por tanto expande menos nodos.

7.4.4. Heurística de Ignorar Listas de Borrado (Delete Lists)

Definición 7.4.3 (Ignorar listas de borrado). Se asume que metas y precondiciones están compuestas **solo por literales positivos**, y se remueven las listas de borrado de las acciones (los literales negados de sus efectos): ningún fluente se elimina nunca. El problema de planeación relajado resultante es resoluble en **tiempo polinomial** mediante ascenso de colinas (*hill climbing*) sobre el espacio de estados, donde los arcos son acciones y la altura de cada punto es el valor heurístico.

Grafo de Planeación y Heurística FF

Un **grafo de planeación** alterna capas de literales y acciones:

- Capa de literales S_i : literales alcanzables en i pasos.
- Capa de acciones A_i : acciones aplicables en el paso i .

Se construye en tiempo polinomial (a diferencia del espacio de estados completo) y la heurística **FF** (*Fast-Forward*) usa la longitud del plan hallado sobre el grafo relajado (ignorando listas de borrado) como estimador del costo restante.

7.4.5. Podado Independiente de Dominio

Además de relajar el problema para obtener una heurística, se pueden podar ramas del árbol de búsqueda sin perder –o aceptando perder– la garantía de optimalidad.

Reducción por simetría. Se podan todas las ramas simétricas del árbol de búsqueda salvo una. Dos acciones son **simétricas** si generan dos estados equivalentes en el contexto del problema: mismas precondiciones y efectos estructurales, difiriendo solo en objetos que la meta no distingue entre sí.

Ejemplo 7.4.3 (Mundo de bloques). Con estado inicial $Sobre(A, Mesa) \wedge Sobre(B, Mesa) \wedge Sobre(C, Mesa) \wedge Despejar(A) \wedge Despejar(B) \wedge Despejar(C)$ y meta $Sobre(A, B)$, las acciones $Mover(A, Mesa, B)$ y $Mover(A, Mesa, C)$ tienen la misma estructura (mover A a la mesa hacia otro bloque despejado): como la meta no distingue entre B y C , ambas ramas son simétricas y basta explorar una.

Verificación paso a paso de la simetría:

1. **Mismas precondiciones:** $Mover(A, Mesa, B)$ exige $Despejar(B)$; $Mover(A, Mesa, C)$ exige $Despejar(C)$ –ambas se cumplen en el estado inicial, con la misma estructura (mover A hacia un bloque despejado cualquiera).
2. **Mismos efectos estructurales:** ambas acciones producen $Sobre(A, \cdot)$ sobre un bloque que queda despejado antes de mover, difiriendo solo en si ese bloque se llama B o C .
3. **La meta no distingue:** $Sobre(A, B)$ menciona a B explícitamente, pero nada en el problema obliga a preferir B sobre C antes de decidir –ambos objetos son intercambiables en este punto de la búsqueda.
4. Como las tres condiciones se cumplen, se declara la simetría y se **descarta** una de las dos ramas (p. ej. $Mover(A, Mesa, C)$) sin perder la solución óptima, reduciendo a la mitad el factor de ramificación efectivo en este paso.

Acciones preferidas. Se asume el riesgo de podar una solución óptima a cambio de guiar la búsqueda rápidamente: se define una versión **relajada** del problema, se resuelve para obtener un **plan relajado**, y se llama **acción preferida** a cualquier acción que sea un paso de ese plan relajado o que logre alguna de sus precondiciones. La búsqueda hacia adelante explora primero las acciones preferidas.

Ejemplo 7.4.4. Con el mismo estado inicial y meta $Sobre(A, B)$ del ejemplo anterior, las seis acciones posibles son $Mover(A, Mesa, B)$, $Mover(A, Mesa, C)$, $Mover(B, Mesa, A)$, $Mover(B, Mesa, C)$, $Mover(C, Mesa, A)$, $Mover(C, Mesa, B)$. La relajación (ignorar precondiciones/borrado) resuelve la meta en un paso con $Mover(A, Mesa, B)$, así que esa es la **acción preferida**: se explora antes que las otras cinco.

Paso a paso de la relajación:

1. Construir el problema relajado: se ignoran las precondiciones (todas las 6 acciones quedan aplicables de inmediato) y las listas de borrado de sus efectos.
2. Revisar el efecto de cada una de las 6 acciones: solo $Mover(A, Mesa, B)$ tiene $Sobre(A, B)$ –el fuente exacto de la meta– en su lista de agregado.
3. El plan relajado óptimo es, entonces, $[Mover(A, Mesa, B)]$, de un solo paso.
4. Como $Mover(A, Mesa, B)$ es un paso de ese plan relajado, se marca como **acción preferida** y la búsqueda hacia adelante la explora antes que las otras cinco.

Remover interacciones negativas (submetas serializables). Un problema tiene **submetas serializables** si existe un orden en el que el planificador puede lograrlas una por una **sin deshacer** ninguna submeta ya alcanzada (sin necesidad de backtracking).

Ejemplo 7.4.5. En el mundo de bloques, para construir una torre A sobre B sobre C sobre la mesa, lograr primero “ C sobre la mesa” no requiere deshacerse después: las submetas son serializables en ese orden. Un contraejemplo clásico son n interruptores que controlan cada uno una luz separada, con meta “todas las luces encendidas”: si se define un orden particular de submetas, ninguna acción posterior interfiere con las anteriores.

Paso a paso (torre A sobre B sobre C):

1. Lograr $Sobre(C, Mesa)$: ya es verdadero en el estado inicial (o se logra sin depender de A ni B).
2. Lograr $Sobre(B, C)$: la acción $Mover(B, Mesa, C)$ no toca a A ni deshace $Sobre(C, Mesa)$ –sigue siendo verdadero después.
3. Lograr $Sobre(A, B)$: la acción $Mover(A, Mesa, B)$ no deshace $Sobre(B, C)$ ni $Sobre(C, Mesa)$.
4. Como ningún paso deshizo una submeta ya lograda, el orden C, B, A es una **serialización válida** –no hizo falta backtracking.

Paso a paso (n interruptores): con $Prender(I_i)$ afectando únicamente a su luz L_i , lograr L_1 , luego $L_2, \dots$, luego L_n en cualquier orden nunca deshace una luz ya encendida, porque cada interruptor es independiente de los demás –las n submetas son serializables en *cualquier* orden.

Abstracción de estado. Muchos problemas reales tienen 10^{100} estados o más; relajar las *acciones* (como arriba) no reduce el número de *estados*, así que evaluar la heurística sigue siendo costoso. La **abstracción de estado** ataca esto **ignorando algunos fluentes** por completo, reduciendo drásticamente el tamaño del espacio de estados sobre el que se calcula la heurística.

Ejemplo 7.4.6 (Carga aérea). Con 10 aeropuertos, 50 aviones y 200 piezas de cargo, el espacio de estados completo tiene del orden de 10^{405} estados. Si se supone que todos los paquetes están en 5 aeropuertos y todos comparten destino, y se **ignoran los fluentes** salvo los que registran un avión y un paquete en cada uno de esos 5 aeropuertos, el espacio abstracto se reduce a apenas 10^{11} estados.

De dónde salen estas magnitudes (orden de magnitud, no cálculo exacto):

1. **Espacio completo:** cada uno de los 200 cargos puede estar en cualquiera de ≈ 60 ubicaciones (10 aeropuertos o dentro de alguno de los 50 aviones), y cada uno de los 50 aviones puede estar en cualquiera de los 10 aeropuertos: el producto de todas estas posibilidades independientes ($\approx 60^{200} \times 10^{50}$) ya está en el orden de 10^{405} .
2. **Espacio abstracto:** si solo interesa, para cada uno de los 5 aeropuertos relevantes, si hay o no un avión presente y si hay o no un paquete presente (dos posibilidades cada uno), el número de combinaciones relevantes se reduce en muchísimos órdenes de magnitud –hasta quedar en el orden de 10^{11} , todavía grande, pero $\approx 10^{394}$ veces más pequeño que el espacio completo.
3. **Consecuencia:** calcular (o aproximar) una heurística sobre 10^{11} estados abstractos es factible; hacerlo sobre 10^{405} estados reales no lo es.

Ideas clave para construir heurísticas: descomposición. Se divide el problema en partes, se resuelve cada parte por separado, y se combinan los resultados, bajo la **suposición de independencia de submetas**: el costo de resolver una conjunción de submetas es, aproximadamente, la suma de los costos de resolver cada submeta por separado. Esta suposición puede fallar en dos direcciones:

- **Optimista:** existen interacciones negativas entre los subplanes de cada submeta (un subplan deshace el progreso de otro), así que el costo real es **mayor** que la suma.
- **Pesimista:** los subplanes contienen acciones redundantes (dos acciones se pueden reemplazar por una sola que sirva a ambas submetas), así que el costo real es **menor** que la suma.

Costo de heurísticas por descomposición: sea una meta con fluentes G dividida en submetas $G_1, \dots, G_n$, y sean $P_1, \dots, P_n$ los planes óptimos que resuelven cada submeta por separado, con $\text{COSTO}(P_i)$ como estimado heurístico de cada una.

- Combinar los estimados con el **máximo**, $\max_i \text{COSTO}(P_i)$, produce una heurística **admisible**.
- Combinar los estimados con la **suma**, $\sum_i \text{COSTO}(P_i)$, en general **no** es admisible (por las interacciones negativas de arriba).
- Es necesario escoger las abstracciones con cuidado, de modo que el costo de calcular la heurística sea menor que el costo de resolver el problema original –si no, la heurística no aporta nada.

Ejemplo 7.4.7 (Máximo vs. suma). Sea $G = G_1 \wedge G_2$, con $\text{COSTO}(P_1) = 3$ y $\text{COSTO}(P_2) = 4$ resolviendo cada submeta por separado.

1. **Combinando con el máximo:** $\max(3, 4) = 4$. Esto nunca sobrestima el costo real, porque resolver *ambas* submetas juntas cuesta, como mínimo, lo que cuesta la más difícil de las dos por separado $\Rightarrow$ admisible.
2. **Combinando con la suma:** $3 + 4 = 7$. Si existe una única acción que logra un fluente de G_1 y un fluente de G_2 a la vez (redundancia entre los subplanes), el costo real de lograr $G_1 \wedge G_2$ juntas podría ser menor que 7 (p. ej. 5) $\Rightarrow$ la suma **sobrestimaría** el costo real, violando la admisibilidad.

Ejemplo 7.4.8 (Aplicación real: NASA Deep Space 1). La sonda *Deep Space 1* de la NASA usó restricciones temporales y de compatibilidad codificadas en un lenguaje similar a PDDL (*Remote Agent*) para planear y ejecutar maniobras autónomamente, ilustrando que estas técnicas de podado y relajación escalan a dominios reales de control de naves espaciales.

7.4.6. Planeación Jerárquica (HTN)

Ni siquiera las heurísticas anteriores evitan que la planeación de bajo nivel sea inviable cuando la solución requiere **millones o miles de millones de acciones primitivas** (p. ej. “mover la rodilla 5 grados” repetido para cada micro-ajuste de un viaje completo). La **planeación jerárquica** (*Hierarchical Task Network*, HTN) ataca el problema organizando las acciones en **niveles de abstracción**, en vez de (o además de) usar heurísticas.

Definición 7.4.4 (HTN). Una red jerárquica de tareas descompone **tareas abstractas** en **subtareas** más concretas, hasta llegar a **tareas primitivas** (acciones directamente ejecutables). Se asume observabilidad completa y determinismo.

- **Tareas primitivas:** acciones directamente ejecutables, con schema estándar (nombre, pre-condición, efecto).
- **Tareas abstractas (acciones de alto nivel, HLA):** se descomponen mediante **métodos de refinamiento** en secuencias de subtareas.
- **Ventaja:** permite incorporar conocimiento de dominio (qué refinamientos probar primero) para guiar la búsqueda, y –como se ve más adelante– reduce drásticamente el costo computacional frente a un planificador plano.

Ejemplo 7.4.9 (Refinamiento recursivo). La acción de alto nivel $Ir(Casa, SFO)$ admite (al menos) dos refinamientos:

REFINAMIENTO($Ir(Casa, SFO)$),

PASOS : [$Manejar(Casa, ParqueaderoSFO)$, $Bus(ParqueaderoSFO, SFO)$]

REFINAMIENTO($Ir(Casa, SFO)$), PASOS : [$Taxi(Casa, SFO)$]

y una acción de alto nivel puede definirse **recursivamente**: por ejemplo, $Navegar([a, b], [x, y])$ (ir de la celda (a, b) a la celda (x, y) en una cuadrícula) se refina en el caso base (ya se llegó) y en dos casos recursivos (moverse a la celda de la izquierda o de la derecha si está conectada):

REFINAMIENTO($Navegar([a, b], [x, y])$), PRECOND : $a=x \wedge b=y$, PASOS : []

REFINAMIENTO($Navegar([a, b], [x, y])$), PRECOND : $Conectados([a, b], [a-1, b])$,

PASOS : [$Izquierda$, $Navegar([a-1, b], [x, y])$]

(y análogamente para $Derecha$ con $[a+1, b]$).

Traza paso a paso de $Navegar([0, 0], [2, 0])$, suponiendo que $[0, 0]$ – $[1, 0]$ – $[2, 0]$ están conectadas en línea recta:

1. $Navegar([0, 0], [2, 0])$: $a=0$, $b=0$, $x=2$, $y=0$. Como $a \neq x$, no aplica el caso base; se cumple $Conectados([0, 0], [1, 0])$, así que se usa el refinamiento hacia la derecha: PASOS = [$Derecha$, $Navegar([1, 0], [2, 0])$].
2. $Navegar([1, 0], [2, 0])$: de nuevo $a=1 \neq x=2$; se cumple $Conectados([1, 0], [2, 0])$: PASOS = [$Derecha$, $Navegar([2, 0], [2, 0])$].
3. $Navegar([2, 0], [2, 0])$: ahora $a=x=2$ y $b=y=0$, se cumple la precondición del **caso base**: PASOS = [].
4. Concatenando las tres capas de refinamiento, el plan primitivo final es [$Derecha$, $Derecha$].

Búsqueda para Soluciones Primitivas

Formulación: se define una acción de alto nivel raíz, $Actuar$, cuyo objetivo es encontrar una implementación que lleve a la meta. Para cada acción primitiva a_i del problema, se añade un refinamiento de $Actuar$ con pasos $[a_i, Actuar]$ (una definición recursiva que va añadiendo acciones); el ciclo se detiene añadiendo un refinamiento con una lista de pasos **vacía** cuya precondición es la meta del problema.

Algorithm 2 HIERARCHICAL-SEARCH(*problem*, *hierarchy*) — búsqueda jerárquica en anchura

```

1: frontier  $\leftarrow$  cola FIFO con [Actuar] como único elemento
2: while verdadero do
3:   if frontier está vacía then return fracaso
4:   end if
5:   plan  $\leftarrow$  POP(frontier) ▷ el plan menos refinado de la frontera
6:   hla  $\leftarrow$  la primera HLA en plan (nulo si no hay ninguna)
7:   prefijo, sufijo  $\leftarrow$  subsecuencias de acciones antes/después de hla en plan
8:   resultado  $\leftarrow$  RESULTADO(problem.INICIAL, prefijo)
9:   if hla es nulo then ▷ el plan es primitivo; resultado es su efecto
10:    if problem.ESMETA(resultado) then return plan
11:    end if
12:   else
13:     for cada secuencia en REFINAMIENTOS(hla, resultado, hierarchy) do
14:       agregar CONCATENAR(prefijo, secuencia, sufijo) a frontier
15:     end for
16:   end if
17: end while

```

Traza paso a paso (meta: llegar a SFO, a partir de $Ir(Casa, SFO)$ con los dos refinamientos del Ejemplo 4.8):

1. $frontier = [Ir(Casa, SFO)]$.
2. Se extrae $[Ir(Casa, SFO)]$: $hla = Ir(Casa, SFO)$ (no nulo); $prefijo = [], sufijo = []$; $resultado = RESULTADO(Casa, []) = Casa$.
3. Como *hla* no es nulo, se buscan sus refinamientos: REFINAMIENTOS($Ir(Casa, SFO)$, *Casa*, *hierarchy*) da las dos secuencias $[Manejar(Casa, ParqueaderoSFO), Bus(ParqueaderoSFO, SFO)]$ y $[Taxi(Casa, SFO)]$, y ambas se agregan a la frontera: $frontier = [[Manejar, Bus], [Taxi]]$ (en notación abreviada).
4. Se extrae el primer plan, $[Manejar, Bus]$ (orden FIFO): ya no contiene ninguna HLA ($hla = \text{nulo}$), así que es **primitivo**; $resultado = RESULTADO(Casa, [Manejar, Bus]) = EstarEn(SFO)$.
5. Como *problem*.ESMETA($EstarEn(SFO)$) es verdadero, el algoritmo **retorna** el plan

$$[Manejar(Casa, ParqueaderoSFO), Bus(ParqueaderoSFO, SFO)]$$

—sin necesidad de examinar la rama $[Taxi(Casa, SFO)]$, que queda sin explorar en la frontera.

Propiedades: sea d el número de acciones primitivas de la solución y b el número de acciones aplicables por estado. Un planificador **no jerárquico** tiene costo $\mathcal{O}(b^d)$. Un planificador **jerárquico** donde cada acción no primitiva admite r refinamientos, cada uno con k acciones en el siguiente nivel, puede construir hasta $r^{(d-1)/(k-1)}$ árboles de descomposición distintos —mucho menor que b^d cuando los refinamientos capturan bien la estructura del dominio. Además, el conocimiento de dominio también puede introducirse directamente en las precondiciones (PRECOND) de los refinamientos, y la estructura jerárquica resultante es más fácil de interpretar para humanos.

Ejemplo numérico: con $b = 10$ acciones aplicables por estado y una solución de $d = 20$ pasos primitivos, un planificador **no jerárquico** enfrenta hasta 10^{20} nodos posibles. Si la jerarquía

ofrece $r = 3$ refinamientos alternativos por HLA, cada uno con $k = 4$ acciones en el siguiente nivel, el planificador **jerárquico** construye a lo sumo $3^{(20-1)/(4-1)} = 3^{19/3} \approx 3^{6.3}$, del orden de **unos pocos miles** de árboles de descomposición –muchísimos órdenes de magnitud menos que 10^{20} .

Búsqueda para Soluciones Abstractas: Semántica Angelical

La búsqueda anterior solo reconoce una solución cuando el plan ya es **completamente primitivo**, lo cual contradice el sentido común: debería ser posible afirmar que el plan

$$[\text{Manejar}(\text{Casa}, \text{ParqueaderoSFO}), \text{Bus}(\text{ParqueaderoSFO}, \text{SFO})]$$

llega al aeropuerto sin refinarlo hasta el nivel de presionar pedales. La solución es escribir descripciones de **precondición-efecto para las acciones de alto nivel**, igual que para las primitivas.

Definición 7.4.5 (No determinismo demoníco vs. angelical). Cuando una acción de alto nivel tiene múltiples implementaciones posibles, hay dos formas de interpretar esa variabilidad:

- **Demónico**: un adversario escoge la implementación (como en juegos adversarios).
- **Angelical**: el **agente** escoge la implementación que más le convenga –entre más refinamientos tenga una acción de alto nivel, más poderosa es bajo esta semántica.

Definición 7.4.6 (Set alcanzable, REACH). Dado un estado s , el **set alcanzable** de una acción de alto nivel h , escrito $\text{REACH}(s, h)$, es el conjunto de todos los estados alcanzables por *alguna* implementación primitiva de h . El set alcanzable de una secuencia se compone:

$$\text{REACH}(s, [h_1, h_2]) = \bigcup_{s' \in \text{REACH}(s, h_1)} \text{REACH}(s', h_2).$$

Un plan de alto nivel llega a la meta exactamente cuando su set alcanzable **interseca** el conjunto de estados meta.

Notación de efectos posiblemente inciertos (para acciones de alto nivel cuya implementación exacta aún no se fija):

- $\tilde{+}A$: posiblemente añade A .
- $\tilde{-}A$: posiblemente remueve A .
- $\tilde{\pm}A$: posiblemente añade o remueve A .

Ejemplo 7.4.10. Si $\text{Ir}(\text{Casa}, \text{SFO})$ tiene dos refinamientos (manejar+bus, o taxi) y el agente solo toma taxi si tiene dinero, entonces la acción de alto nivel *posiblemente* remueve *Dinero* (efecto $\tilde{-}\text{Dinero}$). Con schemas abstractos para h_1 y h_2 :

$$\text{Acción}(h_1), \quad \text{PRECOND} : \neg A, \quad \text{EFECTO} : A \wedge \tilde{-}B$$

$$\text{Acción}(h_2), \quad \text{PRECOND} : \neg B, \quad \text{EFECTO} : \tilde{+}A \wedge \tilde{\pm}C$$

Como puede haber **infinitas** implementaciones distintas de una acción de alto nivel, se usan dos aproximaciones del set alcanzable real:

$$\text{REACH}^-(s, h) \subseteq \text{REACH}(s, h) \subseteq \text{REACH}^+(s, h),$$

donde REACH^+ (descripción **optimista**) **sobreaproxima** el set alcanzable real, y REACH^- (descripción **pesimista**) lo **subaproxima**.

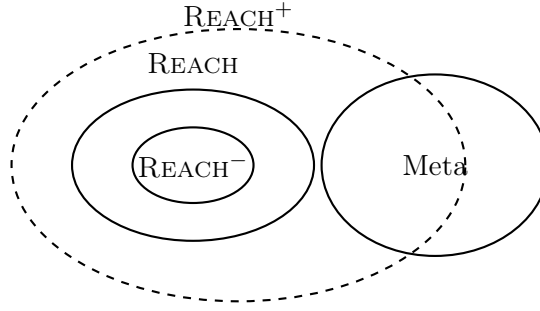


Figura 7.7: $REACH^- \subseteq REACH \subseteq REACH^+$: si $REACH^-$ ya interseca la meta, hay solución garantizada; si ni siquiera $REACH^+$ la interseca, no hay solución; si solo $REACH^+$ la interseca, el resultado es ambiguo y hace falta refinar más.

Verificación:

- Si $REACH^+(s, plan)$ **no** interseca el set de metas $\Rightarrow$ no existe solución por ese plan (ni ninguna de sus implementaciones puede llegar a la meta).
- Si $REACH^-(s, plan)$ **sí** interseca el set de metas $\Rightarrow$ existe solución garantizada (al menos una implementación llega a la meta).
- Si solo $REACH^+$ interseca (y $REACH^-$ no) $\Rightarrow$ el resultado es **ambiguo**: hace falta refinar el plan más para decidir.

Algorithm 3 ANGELIC-SEARCH(*problem*, *hierarchy*, *planInicial*)

```

1: frontier  $\leftarrow$  cola FIFO con planInicial como único elemento
2: while verdadero do
3:   if frontier está vacía then return fracaso
4:   end if
5:   plan  $\leftarrow$  POP(frontier)
6:   if  $REACH^+(problem.INICIAL, plan)$  interseca problem.META then
7:     if plan es primitivo then return plan
8:     end if
9:     garantizado  $\leftarrow REACH^-(problem.INICIAL, plan) \cap problem.META$ 
10:    if garantizado  $\neq \emptyset$  y HAYPROGRESO(plan, planInicial) then
11:      sf  $\leftarrow$  cualquier elemento de garantizado
12:      return DESCOMPONER(hierarchy, problem.INICIAL, plan, sf)
13:    end if
14:    hla  $\leftarrow$  alguna HLA en plan
15:    prefijo, sufijo  $\leftarrow$  subsecuencias antes/después de hla en plan
16:    resultado  $\leftarrow$  RESULTADO(problem.INICIAL, prefijo)
17:    for cada secuencia en REFINAMIENTOS(hla, resultado, hierarchy) do
18:      insertar CONCATENAR(prefijo, secuencia, sufijo) en frontier
19:    end for
20:  end if
21: end while

```

HAYPROGRESO verifica que la búsqueda no esté en un ciclo infinito de refinamientos; una vez se garantiza que *alguna* implementación llega a la meta, DESCOMPONER reconstruye la solución completa dividiendo el problema en subproblemas más pequeños, resolviendo cada uno de forma recursiva con ANGELICSEARCH.

Traza paso a paso (meta: estar en SFO, $planInicial = [Ir(Casa, SFO)]$, retomando el Ejemplo 4.8, donde *ambos* refinamientos de $Ir(Casa, SFO)$ terminan en SFO):

1. $frontier = [[Ir(Casa, SFO)]]$.
2. Se extrae $[Ir(Casa, SFO)]$. Se calcula $REACH^+(Casa, [Ir(Casa, SFO)])$: la unión de los efectos de *ambos* refinamientos posibles (manejar+bus, o taxi) es $\{SFO\}$, que **interseca** la meta.
3. El plan $[Ir(Casa, SFO)]$ **no** es primitivo (contiene una HLA), así que se calcula

$$REACH^-(Casa, [Ir(Casa, SFO)]) :$$

como los *dos* refinamientos posibles llegan a SFO sin excepción (no hay incertidumbre sobre el destino final), $REACH^- = \{SFO\}$ también.

4. $garantizado = REACH^- \cap META = \{SFO\} \neq \emptyset$, y HAYPROGRESO confirma que no hay ciclo: se cumple la condición de la línea 10 del Algoritmo 3.
5. Se retorna $DESCOMPONER(hierarchy, Casa, [Ir(Casa, SFO)], SFO)$, que reconstruye una implementación primitiva concreta –por ejemplo,

$$[Manejar(Casa, ParqueaderoSFO), Bus(ParqueaderoSFO, SFO)]$$

– **sin** haber tenido que enumerar y comparar explícitamente ambos refinamientos acción por acción, como sí lo hacía HIERARCHICAL-SEARCH.

Ejemplo 7.4.11 (Ventaja de la búsqueda angelical: mundo de aspiradora con D piezas). Sea un mundo con muchas piezas (habitaciones) conectadas por corredores, cada una de $w \times h$ casillas, con la jerarquía “Navegar $\rightarrow$ por pieza LimpiarTodaLaPieza $\rightarrow$ limpiar cada fila”. Con D el tamaño de la solución mínima:

- BFS plano: $\mathcal{O}(5^D)$ nodos.
- Búsqueda jerárquica (sin semántica angelical): mejora sobre BFS, pero sigue intentando encontrar **todas** las formas consistentes de limpiar cada pieza.
- Búsqueda angelical: escala **aproximadamente lineal** en el número de casillas, porque $REACH^-/REACH^+$ permiten confirmar o descartar una acción de alto nivel completa sin enumerar sus implementaciones primitivas una por una.

Ejemplo numérico: sea un edificio de 5 piezas de 3×3 casillas cada una, donde limpiar una pieza completa toma $D_{pieza} \approx 9$ pasos primitivos (uno por casilla), para un total de $D \approx 45$ pasos primitivos.

- BFS plano: hasta 5^{45} nodos –completamente inviable.
- Búsqueda jerárquica (sin semántica angelical): reduce la búsqueda a nivel de *LimpiarTodaLaPieza* por pieza, pero sigue enumerando las distintas formas concretas de recorrer las 9 casillas de cada una.
- Búsqueda angelical: usando $REACH^+/REACH^-$, *LimpiarTodaLaPieza* se confirma o descarta como un solo paso de alto nivel –sin enumerar las 9 casillas internamente– así que el costo escala con el número de **piezas** (5), no con D (45).

7.5. Planeación en Ambientes No Deterministas

7.5.1. Ambientes No Deterministas y Condicionales

Cuando las acciones tienen efectos inciertos, se necesitan **planes condicionales** que incluyan ramificaciones según el resultado de las acciones.

- **Plan de búsqueda AND-OR:** los nodos AND representan contingencias del entorno; los nodos OR representan elecciones del agente.
- Un plan condicional es una solución si cubre todas las ramas.

7.5.2. Planeación Parcialmente Observable

Con observabilidad parcial el agente mantiene un **estado de creencia** (belief state) y ejecuta acciones de recopilación de información.

7.5.3. Planeación Continua y de Ejecución

- **Monitoreo de ejecución:** detectar discrepancias entre el plan y la realidad.
- **Replaneación:** adaptar el plan ante fallos o nueva información.
- **Agentes reactivos-deliberativos:** combinan planeación a largo plazo con respuesta reactiva.

Parte V

IA Moderna, Aplicaciones y Ética

Capítulo 8

Semana 8: Aplicaciones, Ética y Repaso Final

Eventos esta semana

- Entrega Taller 3
- Examen Final

8.1. Aplicaciones en IA

8.1.1. Machine Learning

Paradigma donde los sistemas **aprenden de datos** en lugar de ser programados explícitamente.

- **Aprendizaje supervisado:** clasificación, regresión.
- **Aprendizaje no supervisado:** clustering, reducción de dimensionalidad.
- **Aprendizaje por refuerzo (RL):** aprender mediante interacción con el entorno y recompensas.

8.1.2. Deep Learning

Redes neuronales profundas con múltiples capas de abstracción.

- CNN (Convolutional Neural Networks): visión computacional.
- RNN / Transformers: procesamiento de lenguaje natural (NLP).
- LLMs: modelos de lenguaje de gran escala (ej. GPT, Claude).

8.1.3. Áreas de Aplicación

- **NLP:** traducción automática, chatbots, análisis de sentimientos.
- **Visión computacional:** detección de objetos, reconocimiento facial, diagnóstico médico por imágenes.
- **Robótica:** percepción, planeación de movimiento, manipulación.

8.2. Ética en IA y Aplicaciones Futuras

8.2.1. Riesgos y Desafíos Éticos

- **Sesgo algorítmico:** sistemas entrenados con datos sesgados perpetúan o amplifican discriminación.
- **Privacidad:** recolección masiva de datos personales.
- **Seguridad:** ataques adversariales, manipulación de sistemas autónomos.
- **Desplazamiento laboral:** automatización de tareas cognitivas y manuales.
- **Concentración de poder:** dominio de pocas empresas sobre infraestructura de IA.

8.2.2. Marcos Regulatorios

- **AI Act** (Unión Europea): regulación por niveles de riesgo.
- Lineamientos de uso de IA generativa en contextos académicos.

8.2.3. Niveles de Uso de IA Generativa (Uniandes)

Nivel	Uso	Descripción
1	Sin IA generativa	No debe usarse en ningún aspecto
2	Generar/estructurar ideas	No incluir material generado en el trabajo
3	Editar y refinar	Explicar uso en nota al pie o anexo
4	Completar tareas (revisión humana)	Distinguir material IA; incluir transcripción de prompts
5	Uso pleno	Sin restricciones de distinción

8.2.4. Futuro de la IA

8.3. Repaso para Examen Final

Nota

Repaso de todos los temas del curso en preparación para el **Examen Final**. El examen cubre: búsqueda, incertidumbre, CSP, lógica y planeación.

8.3.1. Resumen de Temas

1. **Agentes inteligentes:** PEAS, tipos de agente, naturaleza del entorno.
2. **Búsqueda no informada:** BFS, DFS, IDDFS, UCS.
3. **Búsqueda informada:** A*, IDA*, RBFS; admisibilidad y consistencia.
4. **Búsqueda local:** Hill climbing, simulated annealing, algoritmos genéticos.
5. **Búsqueda adversaria:** Minimax, poda α - β , MCTS.
6. **Incertidumbre:** probabilidad, Bayes, redes bayesianas, inferencia, MDPs.
7. **CSP:** backtracking, AC-3, MRV, min-conflicts.
8. **Lógica:** proposicional, LPO, resolución, encadenamiento.
9. **Planeación:** PDDL, búsqueda en espacio de planes, heurísticas, HTN.